

Лабораторная работа № 9

«Расчет центробежного насоса»

Аннотация

Центробежный насос представляет собой динамический лопастной насос, который непрерывно перемещает рабочее тело за счет центробежной силы. Внутри корпуса (спирального отвода) находится рабочее колесо. Оно может быть открытого, полуоткрытого и закрытого типов. Лопасты обычно изогнуты в противоположную сторону от направления вращения. При вращении рабочего колеса жидкость под действием центробежной силы начинает отбрасываться от центра к периферии, давление в центре понижается, создается разрежение, благодаря которому, жидкость всасывается снова.

План занятия

- расчет рабочего колеса центробежного насоса;
- реализация расчета рабочего колеса на python;
- построение рабочего колеса;
- изменение кода построения рабочего колеса для Salome;
- расчет спирального отвода;
- реализация расчета спирального отвода на python;
- построение спирального отвода;
- подготовка геометрии;
- автоматизация построения на python;
- внесение дополнений в построение спирального отвода;
- создание границ;
- построение сетки;
- настройка OpenFOAM.

Форма представления выполненных заданий

Полученные в рамках данной лабораторной работы результаты прикладываются в качестве ответа в том же разделе LMS.

1 РАСЧЕТ РАБОЧЕГО КОЛЕСА ЦЕНТРОБЕЖНОГО НАСОСА

Расчет рабочего колеса центробежного насоса выполняется согласно алгоритму и формулам, приведенным в учебно-методическом пособии «Расчет лопастных машин и гидродинамических передач».

Входными параметрами являются: подача Q ($\text{м}^3/\text{с}$), напор H (м), число оборотов n (об/мин). Частота вращения принимается равной $\omega = n\pi/30$, давление выражается формулой $p = \rho gH$ (кг/м^3). В качестве жидкости принимается вода, предполагается одноступенчатый насос.

Последовательность расчета:

1. Коэффициент быстроходности:

$$n_s = \frac{3.65n\sqrt{Q}}{H^{3/4}}, \text{ об/мин} \quad (1)$$

2. Приведенный диаметр рабочего колеса:

$$D_{1пр} = (4 - 4.5) \sqrt[3]{\frac{Q}{n}}, \text{ м} \quad (2)$$

3. Гидравлический КПД:

$$\eta_{\Gamma} = 1 - \frac{0.42}{(\lg D_{1пр} - 0.172)^2} \quad (3)$$

Значение $D_{1пр}$ задается в мм.

4. Теоретический напор:

$$H_T = \frac{H}{\eta_{\Gamma}}, \text{ м} \quad (4)$$

5. Объемный КПД насоса:

$$\eta_0 = \frac{1}{1 + 0.68n_s^{-2/3}} \quad (5)$$

6. Подача колеса:

$$Q_K = \frac{Q}{\eta_0}, \text{ м}^3/\text{с} \quad (6)$$

7. Предварительная скорость перед входом в рабочее колесо насоса:

$$C_0 = (0.06 - 0.08)\sqrt[3]{Q_K n^2} \quad (7)$$

8. Механический КПД насоса выбирается:

$$\eta_M = 0.96 - 0.98 \quad (8)$$

9. Общий КПД:

$$\eta = \eta_\Gamma \eta_0 \eta_M \quad (9)$$

10. Мощность для перекачки жидкости:

$$N = \frac{pQ}{1000\eta}, \text{ кВт} \quad (10)$$

11. Мощность с учетом перегрузок:

$$N_M = k_3 N, \text{ кВт}, \quad (11)$$

где k_3 (коэффициент запаса) равен:

1.05 – 1.3, если $N = 1 - 50$ кВт,

1.03, если $N > 50$ кВт

12.Максимальный крутящий момент на приводном валу рабочего колеса:

$$M_M = \frac{9554 N_M}{n}, \text{ Нм} \quad (12)$$

13.Диаметр вала насоса:

$$d_B = \sqrt[3]{\frac{M_M}{0.2[\tau]}}, \text{ м}, \quad (13)$$

где $[\tau]$ (допускаемое напряжение) для стали равно:

$$(15 - 20) \cdot 10^6, \text{ Н/м}^2$$

14.Диаметр втулки рабочего колеса:

$$d_{BT} = (1.2 - 1.25)d_B, \text{ м} \quad (14)$$

15.Диаметр входа в рабочее колесо:

$$D_o = \sqrt{\frac{4Q_k}{\pi C_0} + d_{BT}^2}, \text{ м} \quad (15)$$

16.Окружная скорость входной кромки рабочей лопатки колеса:

$$U_1 = \frac{\pi D_1 n}{60}, \frac{\text{м}}{\text{с}}, \quad (16)$$

где $D_1 = (0.8 - 0.9)D_0$, если $n_s > 90$

$D_1 = (1.03 - 1.05)D_0$, если $n_s \leq 90$

17.Скорость жидкости на входе:

$$C_0 = \frac{4Q_K}{\pi(D_0^2 - d_{\text{вт}}^2)} \quad (17)$$

18.Меридианная скорость потока перед входом в рабочее колесо без учета стеснения потока:

$$C_{m1} \approx C_0 \quad (18)$$

19.Меридианная скорость на входном радиусе с учетом коэффициента стеснения потока:

$$C_{m1} = k_1 C_{m1}, \quad (19)$$

где k_1 (коэффициент стеснения потока) = 1.05 – 1.15

20.Угол безударного входа потока:

$$\beta_{1\text{пот}} = \arctg(C_{m1} / U_1), \text{град} \quad (20)$$

21.Угол атаки:

$$\beta_\alpha = 3^\circ - 10^\circ \quad (21)$$

22.Угол наклона лопатки на входе:

$$\beta_1 = \beta_{1\text{пот}} + \beta_\alpha \quad (22)$$

23.Ширина канала на входе в меридиональном сечении:

$$b_1 = Q_K / (\pi D_1 C_{m1}), \text{ м} \quad (23)$$

24.Коэффициент окружной скорости на выходе из колеса:

$$K_{u2} = (1.87 - 2.0)n_s^{-0.28} \quad (24)$$

25.Окружная скорость на наружном диаметре колеса, первое приближение:

$$U_2 = \sqrt{gH_T / K_{u2}}, \text{ м/с} \quad (25)$$

26. Наружный диаметр рабочего колеса:

$$D_2 = \frac{2U_2}{\omega}, \text{ м} \quad (26)$$

27. Коэффициент стеснения на выходе колеса:

$$K_2 = 1.05 - 1.1 \quad (27)$$

28.Меридианная составляющая абсолютной скорости на выходе из рабочего колеса:

$$0.5C_0 \leq C_{m2} \leq C_0 \quad (28)$$

29.Отношение W_1/W_2 относительных скоростей на входе и выходе колеса:

$$\frac{w_1}{w_2} = 5.65 \cdot 10^{-5} \cdot n_s^2 - 18.23 \cdot 10^{-3} \cdot n_s + 2.65 \quad (29)$$

30. Выходной угол:

$$\beta_2 = \arcsin\left(\frac{W_1 K_2 C_{m2}}{W_2 K_1 C_{m1}} \sin\beta_1\right), \text{град} \quad (30)$$

31. Число лопастей:

$$z = 6.5 \frac{D_2 + D_1}{D_2 - D_1} \sin \frac{\beta_1 + \beta_2}{2} \quad (31)$$

32. Поправка на конечное число лопастей:

$$\mu = \frac{1}{1 + p}, \quad (32)$$

где $p = 2\psi/[z(1 - (D_1/D_2)^2)]$,
где $\psi = (0.55 - 0.65) + 0.6\sin\beta_2$

33. Теоретический напор при бесконечно большом числе лопастей:

$$H_{T\infty} = H_T/\mu, \text{ м} \quad (33)$$

34. Меридиональная скорость на выходе с учетом стеснения потока:

$$C_{m2} = k_2 C_{m2}, \text{ м/с} \quad (34)$$

35. Окружная скорость на выходе колеса (приближение 2):

$$U_2 = 0.5 \frac{C_{m2}}{\operatorname{tg}\beta_2} + 0.5 \sqrt{\left(\frac{C_{m2}}{\operatorname{tg}\beta_2}\right)^2 + 4H_{T\infty}g}, \text{ м/с} \quad (35)$$

36. Наружный диаметр колеса (приближение 2):

$$D_2 = \frac{2U_2}{\omega}, \text{ м} \quad (36)$$

37. Ширина канала на выходе:

$$b_2 = Q_K / (\pi D_2 C_{m2}), \text{ м} \quad (37)$$

38. Толщина лопаток рабочего колеса (для стали):

$$\delta_{л1} = \delta_{л2} = 3 - 6, \text{ мм} \quad (38)$$

39. Проверка коэффициента стеснения на выходе:

$$K'_2 = 1 / (1 - \frac{z\delta_{л2}}{\pi D_2 \sin\beta_2}) \quad (39)$$

При несовпадении коэффициентов K'_2 и K_2 (точность можно принять 5 – 10 %), выбирается другое K_2 и расчет повторяется с п. 30.

40. Проверка коэффициента стеснения на входе:

$$K'_1 = 1 / (1 - \frac{z\delta_{л1}}{\pi D_1 \sin\beta_1}) \quad (40)$$

При несовпадении коэффициентов K'_1 и K_1 с выбранной точностью выбирается другое K_1 и расчет повторяется с п. 19.

2 РЕАЛИЗАЦИЯ РАСЧЕТА РАБОЧЕГО КОЛЕСА НА PYTHON

Создайте в рабочей директории файл с расширением .py для выполнения расчета, используя представленные выше формулы. Назовите его calculation.py. В начале импортируйте библиотеку math для вычислений, объявите входные данные, укажите плотность воды.

Листинг 1 – Входные данные для расчета

```
import math

Q = 0.0138 #м3/с
H = 70.0 #м
n = 2950.0 #rpm

rho = 1000.0 #kg/m3
g = 9.8 #м/с
omega = math.pi * n / 30.0
p1 = rho * g * H
```

В тех формулах, в которых дан диапазон возможных значений, будут выбраны начальные. Полученные данные должны быть округлены. Используйте round. При вычислении гидравлического КПД значение $D_{1пр}$ задается в мм. Не используйте целочисленные типы, иначе расчет будет произведен неверно. Для вывода результатов используйте функцию print.

Листинг 2 – Вычисление коэффициента быстроходности, приведенного диаметра и гидравлического КПД

```
14 n_s = round(3.65 * n * math.sqrt(Q)/math.pow(H, 0.75), 1)
15 print('n_s', n_s)
16
17 D_1pr = round(4.0 * math.pow((Q/n), (1.0/3.0)), 3)
18 print('D_1pr', D_1pr)
19
20 n_g = round(1.0 - (0.42 / math.pow((math.log(D_1pr * 1000.0, 10)
    - 0.172), 2)), 3)
21 print('n_g', n_g)
```

Откройте терминал из каталога, в котором расположен файл, введите python и название файла.

Листинг 3 – Вывод в терминал

```
('n_s', 52.3)
('D_1pr', 0.067)
('n_g', 0.846)
```

Введите формулы для расчета теоретического напора, объемного КПД насоса и подачи колеса.

Листинг 4 – Вычисление теоретического напора, объемного КПД насоса, подачи

```
23 H_T = round(H / n_g, 3)
24 print('H_T', H_T)
25
26 n0 = round(1.0 / (1.0 + 0.68 * math.pow(n_s, (-2.0/3.0))), 3)
27 print('n0', n0)
28
29 Q_K = round(Q / n0, 3)
30 print('Q_K', Q_K)
```

Для предварительной скорости перед входом в рабочее колесо был выбран коэффициент 0.06. Механический КПД был принят 0.96. Общий КПД и мощность вычисляются по соответствующим формулам.

Листинг 5 – Вычисление предварительной скорости, механического и общего КПД, мощности

```
32 c0 = round(0.06 * math.pow((Q_K * n**2), (1.0/3.0)))
33 print('c0', c0)
34
35 n_m = 0.96
36 print('n_m', n_m)
37
38 n_kpd = round(n_g * n0 * n_m, 3)
39 print('n_kpd', n_kpd)
40
41 N = round(p1 * Q / (1000 * n_kpd), 3) # kvт
42 print('N', N)
```

Коэффициент запаса принимает значение в зависимости от мощности. Было выбрано значение 1.05 для второго условия.

Листинг 6 – Вычисление коэффициента запаса, мощности с учетом перегрузок

```
44 if N > 50.0:
45     k3 = 1.03
46 else:
47     k3 = 1.05
48 print('k3', k3)
49
50 N_M = round(k3 * N, 3)
51 print('N_M', N_M)
```

Напишите формулы для вычислений максимального крутящего момента на приводном валу рабочего колеса, диаметра вала насоса, диаметра втулки рабочего колеса, диаметра входа в рабочее колесо. Множитель в допускаемом напряжении был выбран 15, в диаметре втулки 1.2.

Листинг 7 – Вычисление максимального крутящего момента, диаметра вала насоса, диаметра втулки, диаметра входа

```

53 M_M = round(9554.0 * N_M / n, 3)
54 print('M_M', M_M)
55
56 tay = 15.0 * 10**6 # H/m2
57 print('tay', tay)
58
59 d_v = round(math.pow((M_M/0.2/tay), (1.0/3.0)), 4)
60 print('d_v', d_v)
61
62 d_vt = round(1.2 * d_v, 3)
63 print('d_vt', d_vt)
64
65 D0 = round(math.sqrt(4.0 * Q_K / math.pi / C0 + d_vt ** 2), 3)
66 print('D0', D0)

```

Напишите условие расчета диаметра входа по средней линии тока на входной кромке лопатки. Множители были выбраны 0.8 и 1.03 соответственно. Вычислите окружную скорость входной кромки рабочей лопатки колеса, скорость жидкости на входе и меридианную скорость потока перед входом в рабочее колесо без учета стеснения потока.

Листинг 8 – Вычисление диаметра входа по средней линии тока, окружной скорости входной кромки, скорости на входе и меридианной скорости потока

```

68 if n_s > 90.0:
69     D1 = round(0.8 * D0, 3)
70 else:
71     D1 = round(1.03 * D0, 3)
72 print('D1', D1)
73
74 U1 = round(math.pi * D1 * n / 60.0)
75 print('U1', U1)
76
77 C0 = round(4.0 * Q_K / (math.pi * (D0 ** 2 - d_vt ** 2)))
78 print('C0', C0)
79
80 C_m1 = C0
81 print('C_m1', C_m1)

```

Так как часть расчета может выполняться не один раз (при несовпадении коэффициентов K'_1 и K_1 или K'_2 и K_2), вынесем его в отдельные функции. Первую назовите func1, на вход она будет принимать коэффициент K_1 . Объявите использование глобальных переменных и напишите формулы для вычислений меридианной скорости на входном радиусе с учетом коэффициента стеснения, угла безударного входа потока (при радиальном входе $\alpha_1 = 90^\circ$), угла наклона лопатки на входе, ширины канала на входе в меридиональном сечении. Величину угла переведите в градусы, используйте функцию math.degrees.

Листинг 9 – Функция func1

```

112 def func1(k1_local):
113     global k1, C_m1, U1, betta1_pot, betta1, b1, Q_K, D1
114
115     k1 = k1_local
116     print('k1', k1)
117
118     C_m1 = round(k1 * C_m1, 2)
119     print('C_m1', C_m1)
120
121     betta1_pot = round(math.degrees(math.atan(C_m1/U1)))
122     print('betta1_pot', betta1_pot)
123
124     betta_a = 3.0
125     print('betta_a', betta_a)
126
127     betta1 = betta1_pot + betta_a
128     print('betta1', betta1)
129
130     b1 = round(Q_K / (math.pi * D1 * C_m1), 3)
131     print('b1', b1)

```

Выше данной функции напишите формулы для вычисления коэффициента окружной скорости на выходе из колеса (был выбран множитель 1.87), окружной скорости на наружном диаметре колеса в первом приближении, наружного диаметра рабочего колеса. Объявите и ширину лопасти на данном этапе. Было выбрано значение 4 мм.

Листинг 10 – Вычисление коэффициента окружной скорости на выходе, окружной скорости на наружном диаметре, наружного диаметра колеса, ширины

```
84 K_u2 = round(1.87 * math.pow(n_s, (-0.28)),3)
85 print('K_u2', K_u2)
86
87 U2 = round(math.sqrt(g * H_T / K_u2), 3)
88 print('U2', U2)
89
90 D2 = round(2.0 * U2 / omega, 3)
91 print('D2', D2)
92
93 width = 0.004
94 print('width', width)
```

Объявите глобальные переменные.

Листинг 11 – Глобальные переменные

```
96 # Vars
97 betta1 = 0.0,
98 k1_ = 0.0,
99 C_m2 = 0.0,
100 W12 = 0.0,
101 betta2 = 0.0,
102 z = 0.0,
103 psi = 0.0,
104 p = 0.0,
105 mu = 0.0,
106 H_T_inf = 0.0,
107 k2_ = 0.0,
108 k1 = 0.0,
109 k2 = 0.0,
```

Создайте вторую функцию func2, принимающую на вход коэффициент K_2 . Объявите использование глобальных переменных, присвойте входное значение переменной K_2 .

Листинг 12 – Создание функции func2

```
135 def func2(k2_local):
136     global k2, C_m2, C0, W12, n_s, betta2, k1, C_m1, betta1,
137     z, D2, D1, psi, p, mu, H_T_inf, H_T, U2, D2, omega, Q_K, b2,
138     width, k1_, k2_
139     k2 = k2_local
140     print('k2', k2)
```

Меридианная составляющая абсолютной скорости на выходе из рабочего колеса была принята $0.8C_0$. Отношение относительных скоростей на входе и выходе колеса вычисляется по соответствующей формуле.

Листинг 13 – Вычисление меридианной составляющей абсолютной скорости на выходе и отношения относительных скоростей

```

141     C_m2 = round(0.8 * C0, 1)
142     print('C_m2', C_m2)
143
144     W12 = round(5.65 * math.pow(10.0, -5) * n_s ** 2 - 18.23
145 * math.pow(10.0, -3) * n_s + 2.65, 2)
146     print('W12', W12)

```

Согласно формулам вычислите угол выхода (не забывайте переводить в радианы для расчета тригонометрических функций), число лопастей, поправку на конечное число лопастей, теоретический напор. Для ψ был выбран множитель 0.55.

Листинг 14 – Вычисление угла выхода, числа лопастей, поправки на конечное число лопастей, теоретического напора

```

147     betta2 = round(math.degrees(math.asin(W12 * k2 / k1 *
148 C_m2 / C_m1 * math.sin(math.radians(betta1))))
149     print('betta2', betta2)
150
151     z = round(6.5 * ((D2 + D1)/(D2 - D1)) *
152 math.sin((math.radians(betta1) + math.radians(betta2))/2))
153     print('z', z)
154
155     psi = round(0.55 + 0.6 * math.sin(math.radians(betta2)),
156 3)
157     print('psi', psi)
158
159     p = round(2.0 * psi / (z * (1.0 - (D1 / D2)**2)), 3)
160     print('p', p)
161
162     mu = round(1.0 / (1.0 + p), 3)
163     print('mu', mu)
164
165     H_T_inf = round(H_T / mu, 3)
166     print('H_T_inf', H_T_inf)

```

Вычислите меридиональную скорость на выходе с учетом стеснения потока, окружную скорость на выходе колеса во втором приближении,

наружный диаметр колеса во втором приближении, ширину канала на выходе, коэффициенты стеснения на входе и выходе.

Листинг 15 – Окончание функции func2

```
165         C_m2 = round(k2 * C_m2, 3)
166         print('C_m2', C_m2)
167
168         U2 = round(0.5 * (C_m2 /
169         (math.tan(math.radians(betta2))) + math.sqrt((C_m2 /
170         (math.tan(math.radians(betta2))) ** 2 + 4.0 * g * H_T_inf))), 3)
169         print('U2', U2)
170
171         D2 = round(2.0 * U2 / omega, 3)
172         print('D2', D2)
173
174         b2 = round(Q_K / (math.pi * D2 * C_m2), 3)
175         print('b2', b2)
176
177         k1_ = round(1.0 / (1.0 - (z * width)/(math.pi * D1 *
178         math.sin(math.radians(betta1)))), 3)
179         print('k1_', k1_)
180
181         k2_ = round(1.0 / (1.0 - (z * width)/(math.pi * D2 *
182         math.sin(math.radians(betta2)))), 3)
183         print('k2_', k2_)
```

Создайте третью функцию по проверке коэффициентов K'_1 и K_1 , K'_2 и K_2 . Назовите ее funcErr. Объявите использование глобальных переменных. Точность примем равной 10%. В условие проверки коэффициентов добавьте проверку на их максимальное значение. При несовпадении с заданной точностью соответствующий коэффициент увеличивается на 0.01, функции вызываются снова. В терминал выводится результат выполнения функции.

Листинг 16 – Функция funcErr начало

```
184 def funcErr():
185     global k2, k1
186
187     err_k1 = k1_ * 100.0 / k1
188     print('err_k1', err_k1)
189
190     if (math.fabs(100.0 - err_k1) > 10.0 and k1 < 1.15):
191         print('k1 again')
192         k1 = k1 + 0.01
193         func1(k1)
194         func2(k2)
195         funcErr()
196     elif k1 > 1.15:
197         print('k1 more than 1.15')
```

Листинг 17 – Функция funcErr продолжение

```

198         else:
199             print('k1 done')
200
201         err_k2 = 100.0 * k2_ / k2
202         print('err_k2', err_k2)
203
204         if (math.fabs(100.0 - err_k2) > 10.0 and k2 < 1.1):
205             print('k2 again')
206             k2 = k2 + 0.01
207             func2(k2)
208             funcErr()
209         elif k2 > 1.1:
210             print('k2 more than 1.1')
211         else:
212             print('k2 done')

```

Объявите вызов функций с начальными значениями.

Листинг 18 – Вызов функций

```

215 func1(1.05)
216 func2(1.05)
217 funcErr()

```

Вызовите скрипт в терминале.

Листинг 19 – Результат расчета

('n_s', 52.3)	('betta1', 17.0)	('z', 5.0)
('D_lpr', 0.067)	('b1', 0.017)	('psi', 0.784)
('n_g', 0.846)	('k2', 1.05)	('p', 0.36)
('H_T', 82.742)	('C_m2', 2.4)	('mu', 0.735)
('n0', 0.954)	('W12', 1.85)	('H_T_inf', 112.574)
('Q_K', 0.014)	('betta2', 24.0)	('C_m2', 2.52)
('C0', 3.0)	('z', 5.0)	('U2', 36.236)
('n_m', 0.96)	('psi', 0.794)	('D2', 0.235)
('n_kpd', 0.775)	('p', 0.364)	('b2', 0.008)
('N', 12.215)	('mu', 0.733)	('k1_', 1.35)
('k3', 1.05)	('H_T_inf', 112.881)	('k2_', 1.074)
('N_M', 12.826)	('C_m2', 2.52)	('err_k1', 127.35849056603773)
('M_M', 41.539)	('U2', 36.138)	k1 again
('tay', 15000000.0)	('D2', 0.234)	('k1', 1.07)
('d_v', 0.024)	('b2', 0.008)	('C_m1', 3.57)
('d_vt', 0.029)	('k1_', 1.35)	('betta1_pot', 15.0)
('D0', 0.082)	('k2_', 1.072)	('betta_a', 3.0)
('D1', 0.084)	('err_k1', 128.57142857142856)	('betta1', 18.0)
('U1', 13.0)	k1 again	('b1', 0.015)
('C0', 3.0)	('k1', 1.06)	('k2', 1.05)
('C_m1', 3.0)	('C_m1', 3.34)	('C_m2', 2.4)
('K_u2', 0.618)	('betta1_pot', 14.0)	('W12', 1.85)
('U2', 36.223)	('betta_a', 3.0)	('betta2', 22.0)
('D2', 0.235)	('betta1', 17.0)	('z', 5.0)
('width', 0.004)	('b1', 0.016)	('psi', 0.775)
('k1', 1.05)	('k2', 1.05)	('p', 0.355)
('C_m1', 3.15)	('C_m2', 2.4)	('mu', 0.738)
('betta1_pot', 14.0)	('W12', 1.85)	('H_T_inf', 112.117)
('betta_a', 3.0)	('betta2', 23.0)	('C_m2', 2.52)

Листинг 20 – Результат расчета продолжение

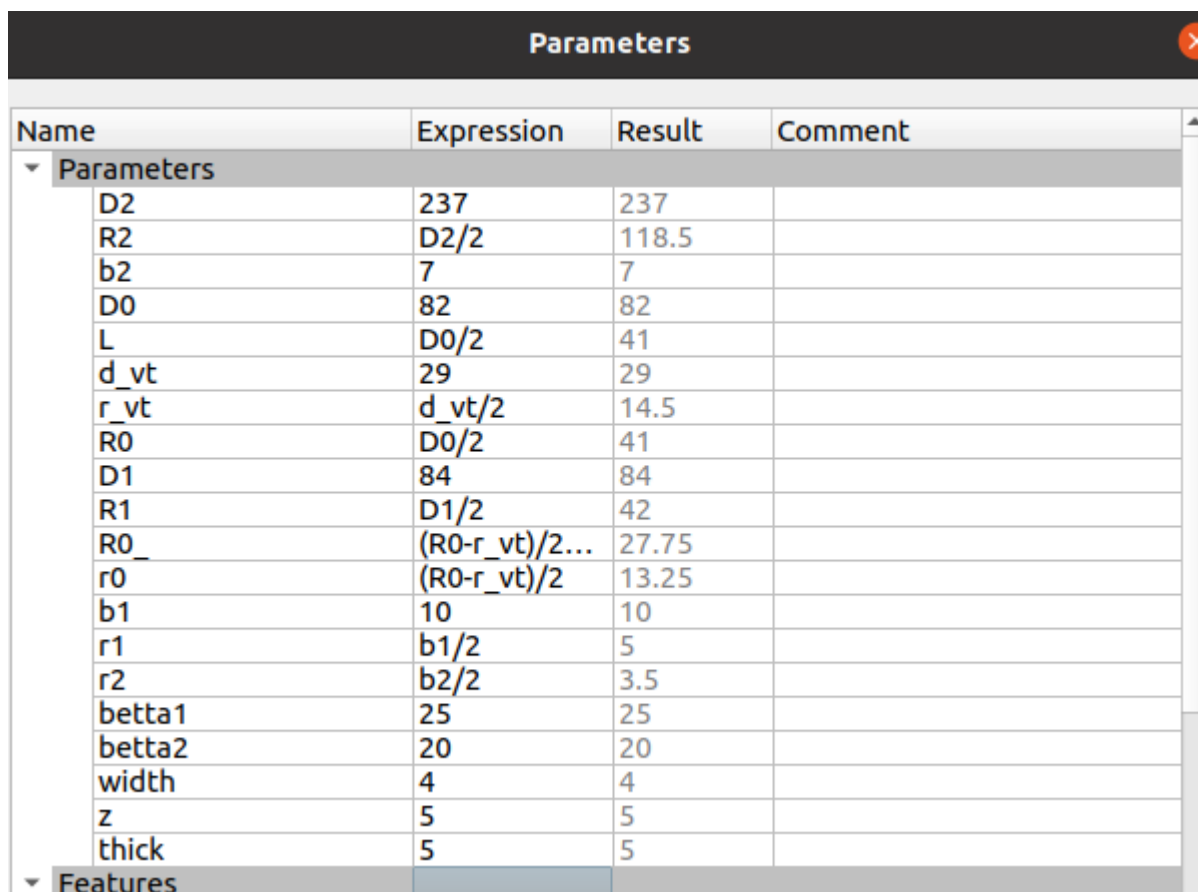
```

('U2', 36.324) ('b1', 0.013) ('D2', 0.236)
('D2', 0.235) ('k2', 1.05) ('b2', 0.007)
('b2', 0.008) ('C_m2', 2.4) ('k1_', 1.241)
('k1_', 1.325) ('W12', 1.85) ('k2_', 1.081)
('k2_', 1.078) ('betta2', 21.0) ('err_k1', 112.81818181818181)
('err_k1', 123.83177570093457) ('z', 5.0) k1 again
k1 again ('psi', 0.765) ('k1', 1.11)
('k1', 1.08) ('p', 0.351) ('C_m1', 5.14)
('C_m1', 3.86) ('mu', 0.74) ('betta1_pot', 22.0)
('betta1_pot', 17.0) ('H_T_inf', 111.814) ('betta_a', 3.0)
('betta_a', 3.0) ('C_m2', 2.52) ('betta1', 25.0)
('betta1', 20.0) ('U2', 36.449) ('b1', 0.01)
('b1', 0.014) ('D2', 0.236) ('k2', 1.05)
('k2', 1.05) ('b2', 0.007) ('C_m2', 2.4)
('C_m2', 2.4) ('k1_', 1.268) ('W12', 1.85)
('W12', 1.85) ('k2_', 1.081) ('betta2', 20.0)
('betta2', 22.0) ('err_k1', 116.33027522935778) ('z', 5.0)
('z', 5.0) k1 again ('psi', 0.755)
('psi', 0.775) ('k1', 1.1) ('p', 0.346)
('p', 0.355) ('C_m1', 4.63) ('mu', 0.743)
('mu', 0.738) ('betta1_pot', 20.0) ('H_T_inf', 111.362)
('H_T_inf', 112.117) ('betta_a', 3.0) ('C_m2', 2.52)
('C_m2', 2.52) ('betta1', 23.0) ('U2', 36.569)
('U2', 36.324) ('b1', 0.011) ('D2', 0.237)
('D2', 0.235) ('k2', 1.05) ('b2', 0.007)
('b2', 0.008) ('C_m2', 2.4) ('k1_', 1.219)
('k1_', 1.285) ('W12', 1.85) ('k2_', 1.085)
('k2_', 1.078) ('betta2', 21.0) ('err_k1', 109.81981981981981)
('err_k1', 118.98148148148147) ('z', 5.0) k1 done
k1 again ('psi', 0.765) ('err_k2', 103.33333333333333)
('k1', 1.09) ('p', 0.35) k2 done
('C_m1', 4.21) ('mu', 0.741)
('betta1_pot', 18.0) ('H_T_inf', 111.663)
('betta_a', 3.0) ('C_m2', 2.52)
('betta1', 21.0) ('U2', 36.427)

```

3 ПОСТРОЕНИЕ РАБОЧЕГО КОЛЕСА

Согласно вычисленным значениям выполним построение рабочего колеса. Откройте Salome, перейдите в модуль Shaper. Выберите Part => Parameters. Добавьте переменные, необходимые для построения (Рисунок 1). Их вычисление будет рассмотрено позже.



Name	Expression	Result	Comment
Parameters			
D2	237	237	
R2	$D2/2$	118.5	
b2	7	7	
D0	82	82	
L	$D0/2$	41	
d_vt	29	29	
r_vt	$d_vt/2$	14.5	
R0	$D0/2$	41	
D1	84	84	
R1	$D1/2$	42	
R0_	$(R0-r_vt)/2...$	27.75	
r0_	$(R0-r_vt)/2$	13.25	
b1	10	10	
r1	$b1/2$	5	
r2	$b2/2$	3.5	
betta1	25	25	
betta2	20	20	
width	4	4	
z	5	5	
thick	5	5	
Features			

Рисунок 1 – Объявление переменных

3.1 Построение канала рабочего колеса

В основном меню выберите Part => New Part. Создайте эскиз на плоскости YOZ, нажмите Set plane view, чтобы перпендикулярно установить плоскость, уберите видимость (чекбокс Visible).

Выберите инструмент Линия, активируйте чекбокс Auxiliary, перпендикулярно оси OY проведите прямую линию, убедитесь, что присутствует зависимость вертикальности (Рисунок 2).

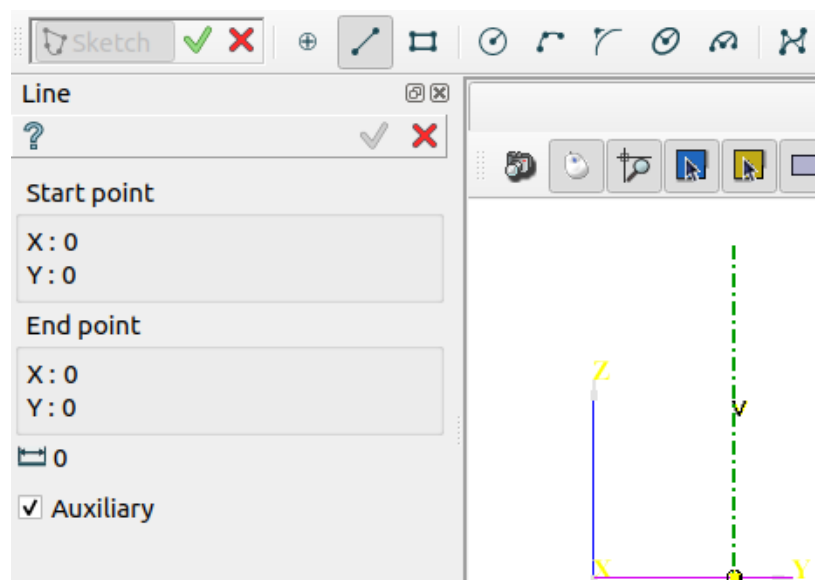


Рисунок 2 – Создание линии

Установите расстояние до оси OY, используйте инструмент Vertical Distance (Рисунок 3).

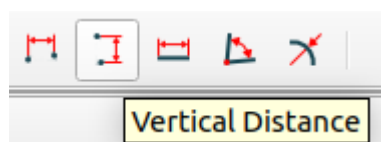


Рисунок 3 – Установка вертикального расстояния

Выберите значение R2 из переменных, которое равно половине наружного диаметра рабочего колеса (Рисунок 4). Примите изменения.

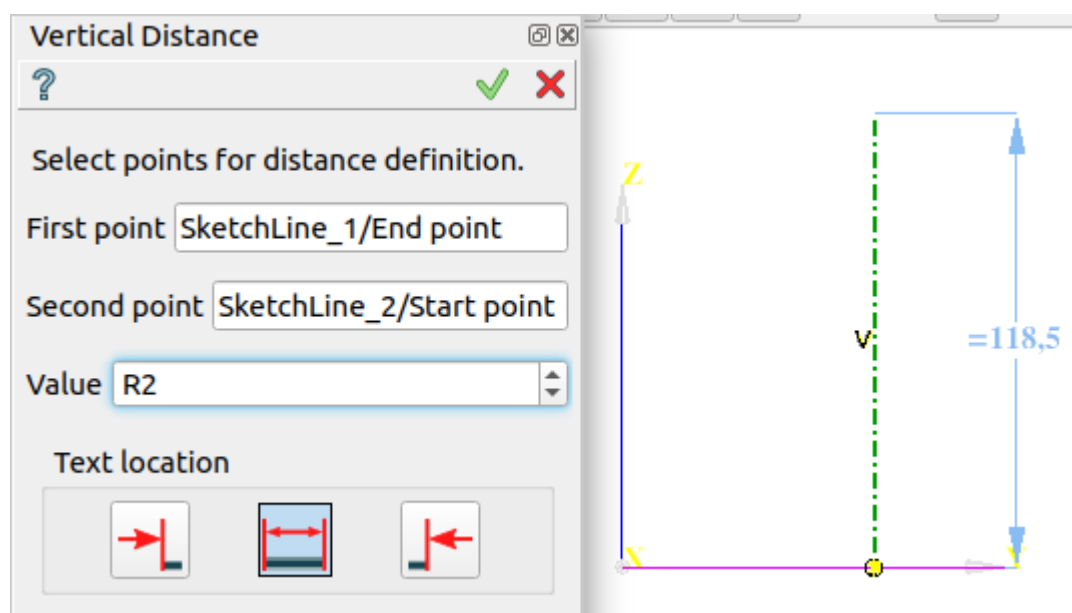


Рисунок 4 – Установка размера

Влево от построенной оси отложите отрезок, равный ширине канала на выходе (b_2). Для установки размера можете использовать Length или Horizontal Distance (Рисунок 5).

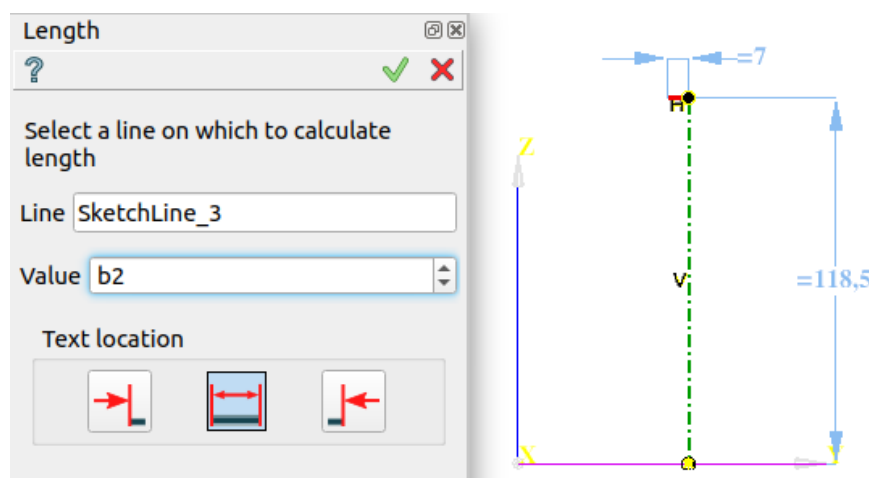


Рисунок 5 – Установка размера ширины канала на выходе

Отложите еще один отрезок длиной L , который равен половине диаметра входа в рабочее колесо (Рисунок 6), на расстоянии половины диаметра втулки рабочего колеса (Рисунок 7). L – наибольшая ширина входа.

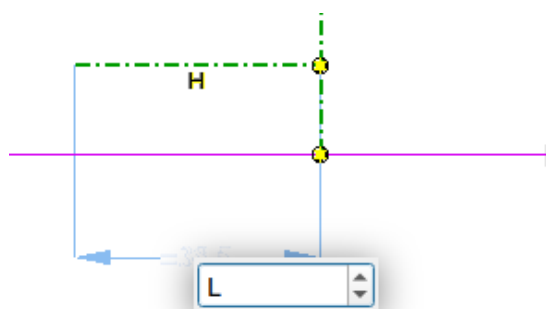


Рисунок 6 – Установка размера L

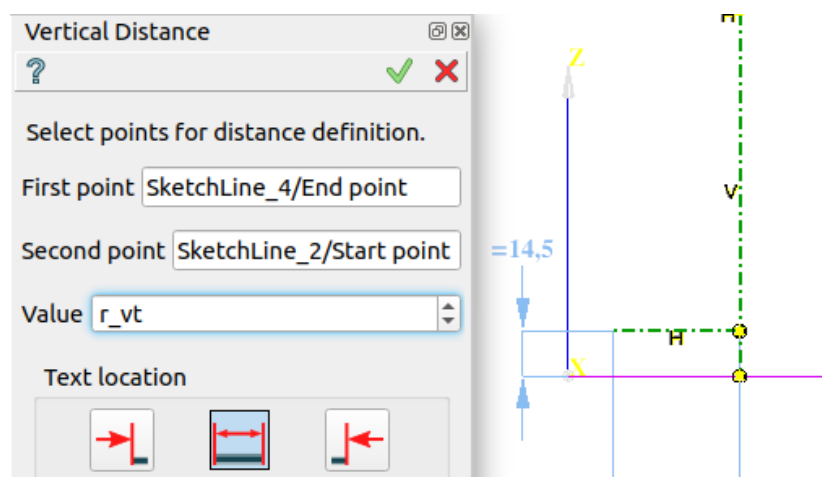


Рисунок 7 – Установка вертикального размера

На расстоянии R0 от оси OY отложите отрезок (Рисунок 8).

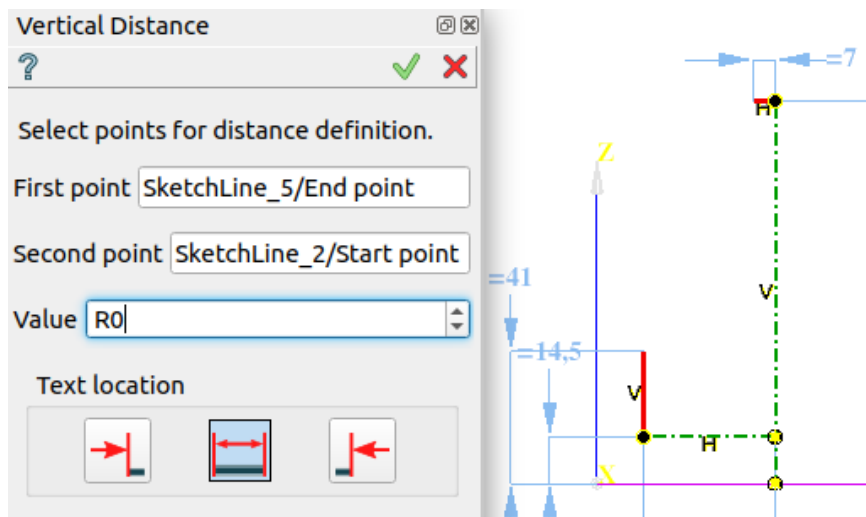


Рисунок 8 – Установка вертикального расстояния

Используйте зависимость Coincident для соединения эскиза с осью OZ (Рисунок 9).

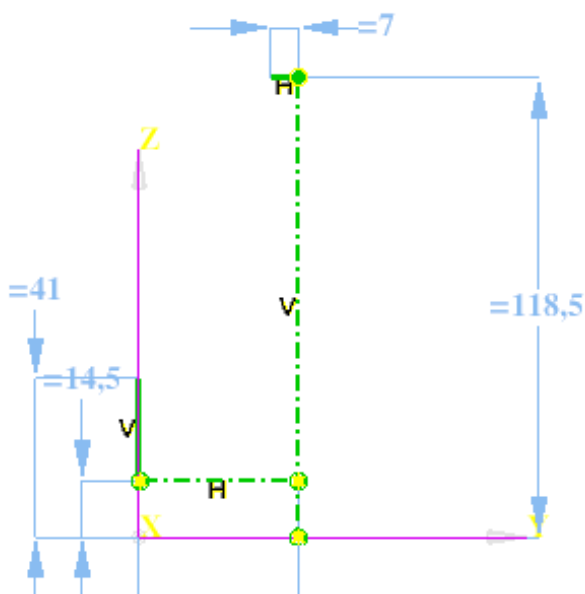


Рисунок 9 – Зависимость Coincident

Создайте 4 горизонтальных отрезка и 4 точки (Рисунок 10).

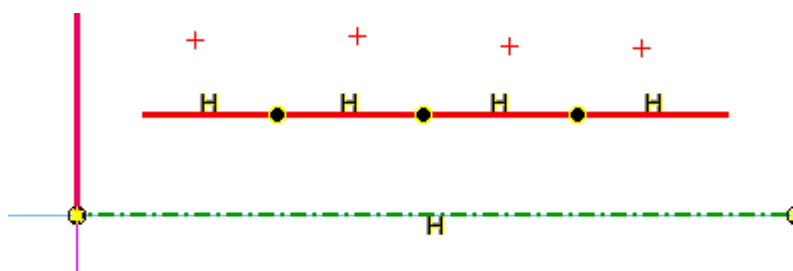


Рисунок 10 – Создание отрезков и точек

С помощью зависимости Equal сделайте все отрезки равными (Рисунок 11).

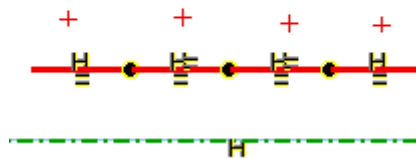


Рисунок 11 – Зависимость Equal

Выберите зависимость Middle point и соедините точки с отрезками. Должно получиться 8 равных отрезков (Рисунок 12).

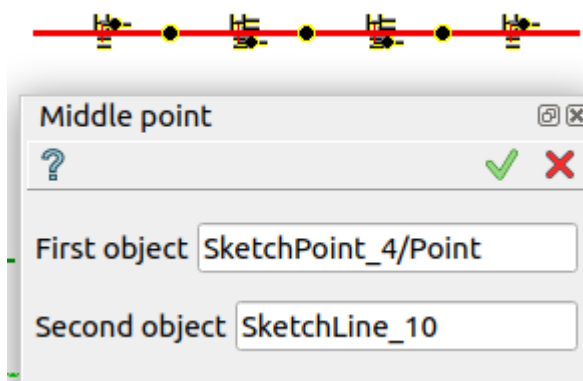


Рисунок 12 – Зависимость Middle point

Соедините концы созданной линии с отрезком длиной L. Должен получиться полностью определенный эскиз (Рисунок 13).

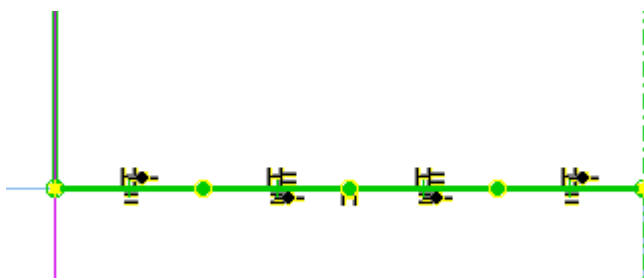


Рисунок 13 – Соединение линий

Теперь те же действия нужно повторить и в вертикальном направлении, общая длина L (Рисунок 14).

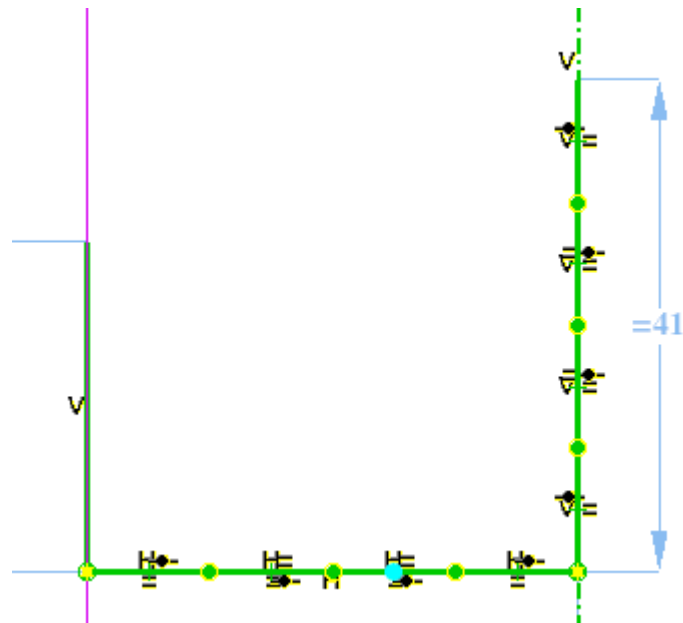


Рисунок 14 – Создание вертикальных отрезков

Точки на горизонтальном и вертикальном отрезке соединяются линиями (Рисунок 15).

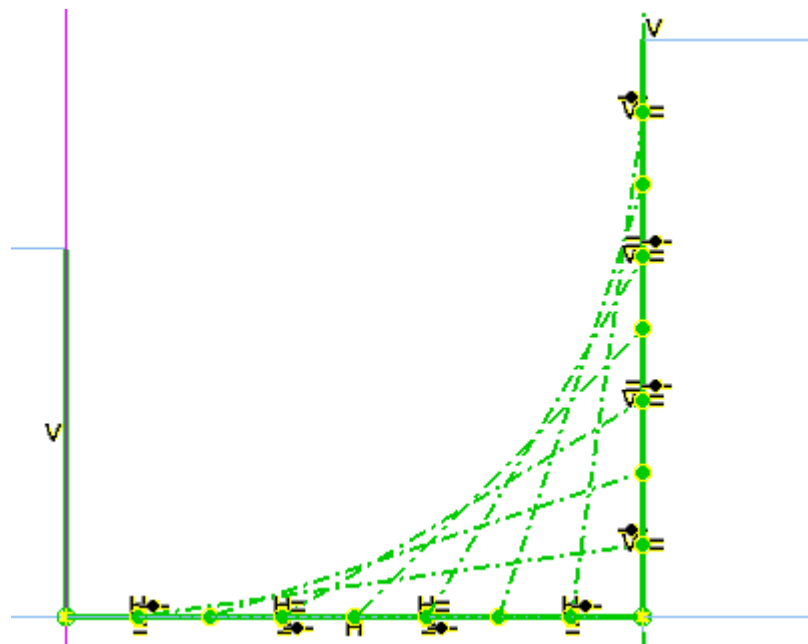


Рисунок 15 – Соединение точек

Постройте 6 точек, которые являются точками пересечения линий, используйте инструмент Coincident для соединения точки с двумя отрезками (Рисунок 16).

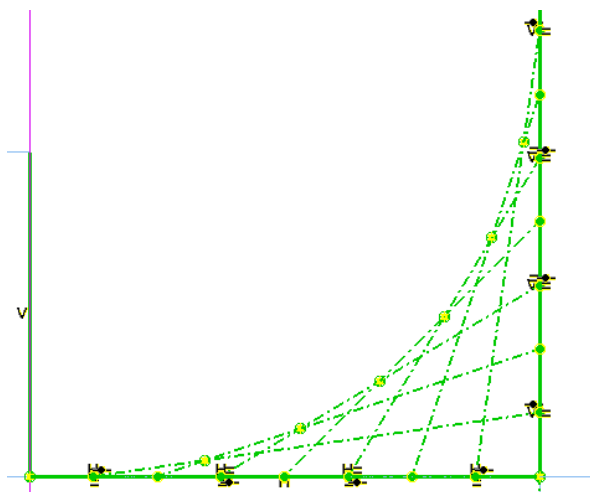


Рисунок 16 – Создание точек пересечения

Теперь строится огибающая линия, которая является приближенным меридианным сечением контура ведущего диска. Используйте инструмент Arc по трем точкам (Рисунок 17).

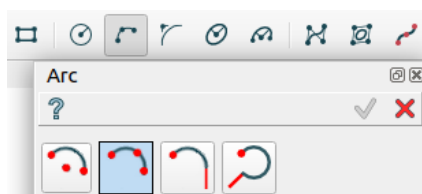


Рисунок 17 – Инструмент Arc

Последовательно выбирайте начало, конец затем среднюю точку (Рисунок 18). Повторите для всех точек.

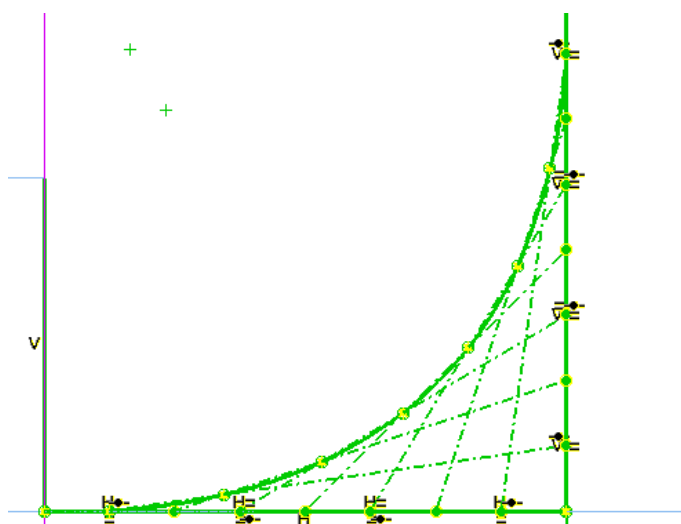


Рисунок 18 – Меридианное сечение контура ведущего диска

Постройте две точки на входном отрезке на расстоянии $R0_{_}$ и $R1$ от точки 0 (Рисунок 19).

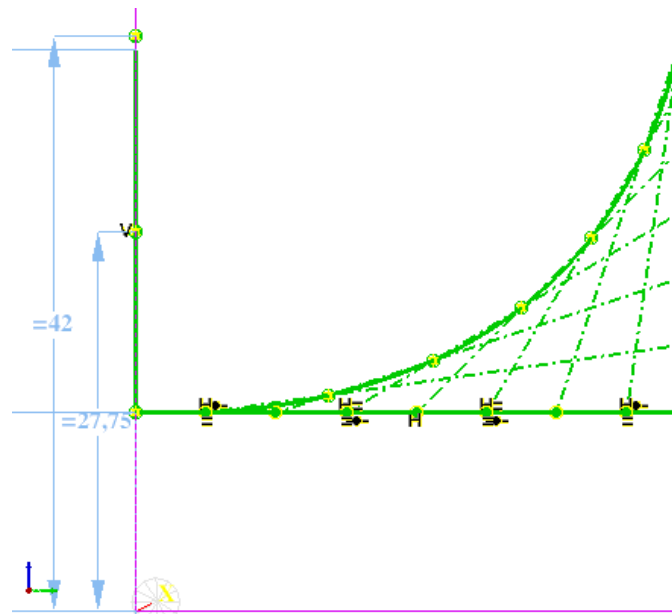


Рисунок 19 – Построение точек

На горизонтальных линиях, проведенных из этих точек, описываются окружности радиусами r_0 , r_1 , окружность радиусом r_2 описывается на выходном отрезке b_2 . Можете указывать радиусы, можете использовать зависимости касательной и средней точки линии. Все окружности должны касаться линии, являющейся приближенным меридианным сечением контура ведущего диска (Рисунок 20).

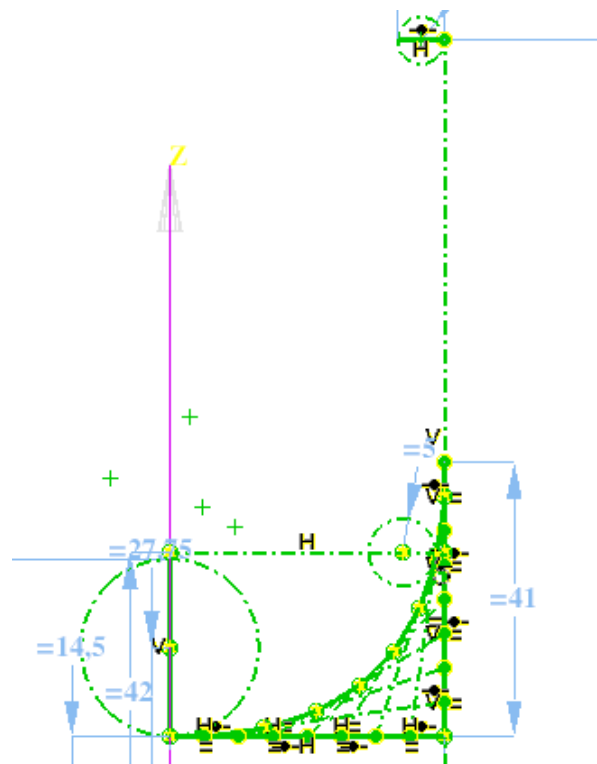


Рисунок 20 – Создание окружностей

Проведите линию, касающуюся окружностей, которая является приближенным меридианным сечением контура ведомого диска.

Выберите дугу по трем точкам, установите зависимость касательной линии для окружностей (Рисунок 21).

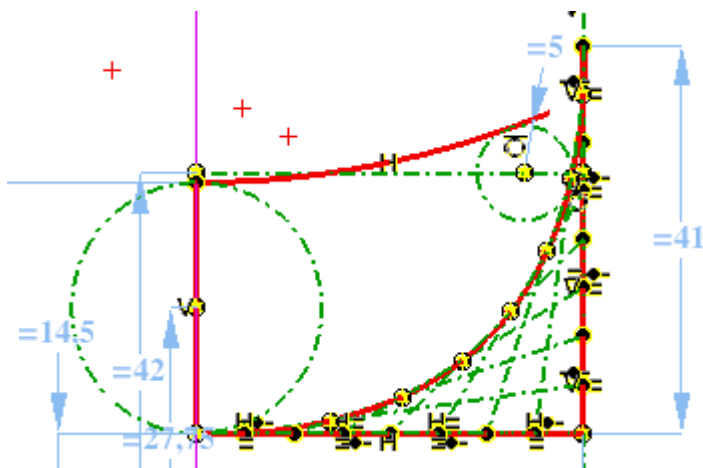


Рисунок 21 – Создание касательной

Проведите прямую линию и установите угол 3° (Рисунок 22).

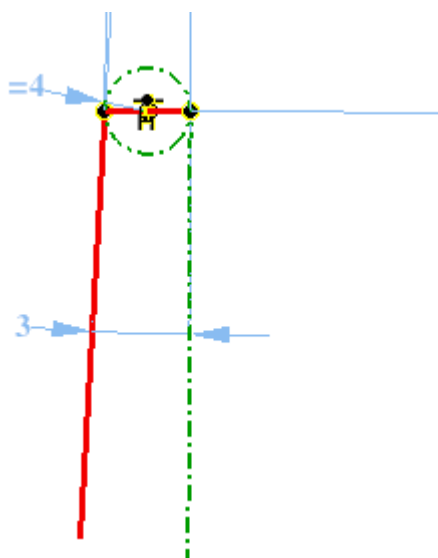


Рисунок 22 – Создание линии

Соедините точки касательной и прямой. Используйте инструмент fillet для скругления. Выберите полученную точку и задайте радиус 5° . Соедините оставшуюся часть меридианного сечения прямой линией. Сделайте линии, необходимые только на этапе построения сечения, вспомогательными (Рисунок 23).

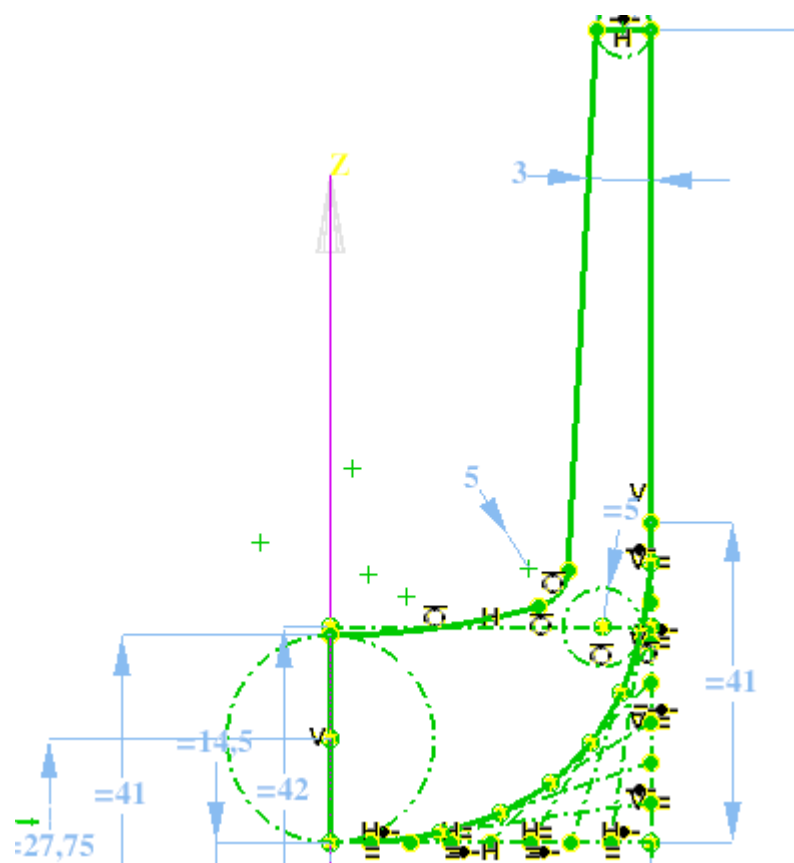


Рисунок 23 – Создание скругления

Выходите из эскиза, сохраняйте изменения. Перейдите в раздел Features, выберите операцию Revolution. Укажите ось OY, угол 360 (Рисунок 24).

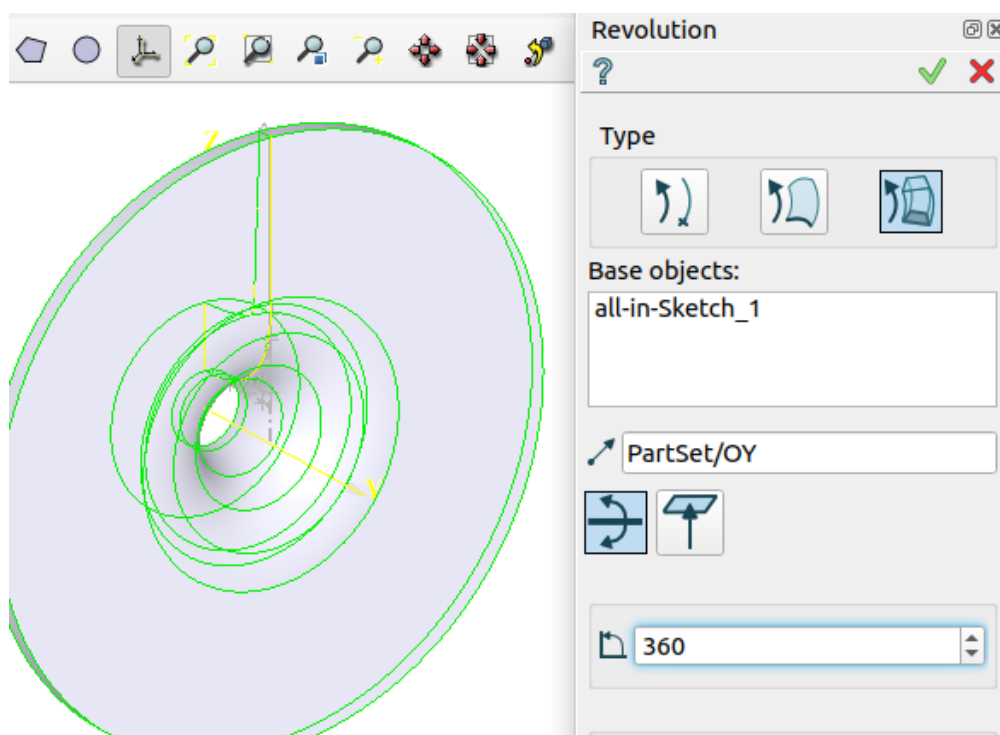


Рисунок 24 – Операция Revolution

Сохраните промежуточный файл, назовите его wheel, также выполните команду File => Dump Study, сформируется скрипт, назовите его тоже wheel.

Откройте скрипт по расчету рабочего колеса центробежного насоса и измените входные параметры. Установите $Q = 0.008 \text{ м}^3/\text{с}$, $H = 9.7 \text{ м}$, $n = 1000 \text{ об/мин}$. Выполните расчет путем запуска скрипта calculation.py в терминале. Полученные результаты используйте для кода wheel.py.

Листинг 21 – Входные параметры

```
model.addParameter(partSet, "D2", "271")
model.addParameter(partSet, "R2", "D2/2")
model.addParameter(partSet, "b2", "11")
model.addParameter(partSet, "D0", "103")
model.addParameter(partSet, "L", "D0/2")
model.addParameter(partSet, "d_vt", "18")
model.addParameter(partSet, "r_vt", "d_vt/2")
model.addParameter(partSet, "R0", "D0/2")
model.addParameter(partSet, "D1", "106")
model.addParameter(partSet, "R1", "D1/2")
model.addParameter(partSet, "R0_", "(R0-r_vt)/2+r_vt")
model.addParameter(partSet, "r0", "(R0-r_vt)/2")
model.addParameter(partSet, "b1", "15")
model.addParameter(partSet, "r1", "b1/2")
model.addParameter(partSet, "r2", "b2/2")
model.addParameter(partSet, "betta1", "27")
model.addParameter(partSet, "betta2", "15")
model.addParameter(partSet, "width", "4")
model.addParameter(partSet, "z", "4")
model.addParameter(partSet, "thick", "5")
```

Загрузите в Salome измененный скрипт wheel.py. Геометрия должна перестроиться (Рисунок 25).

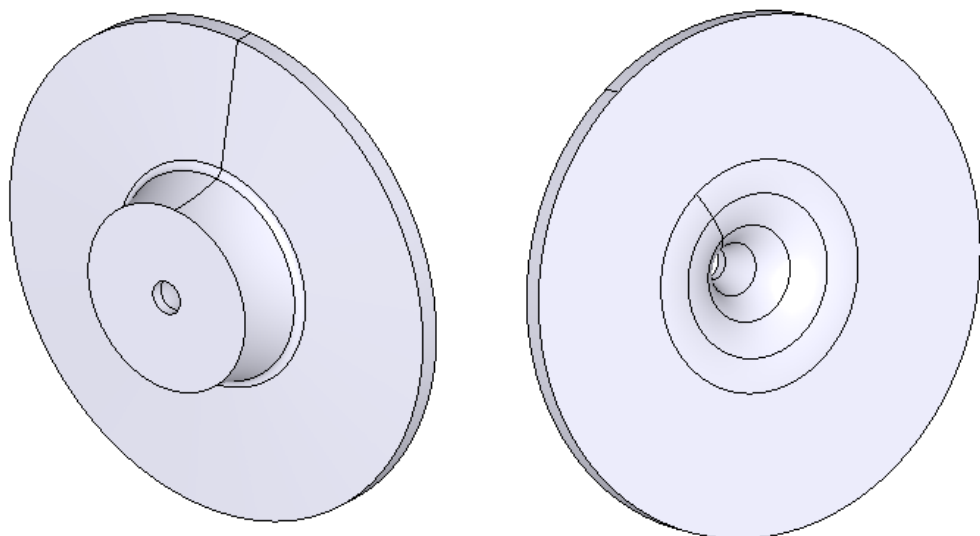


Рисунок 25 – Канал рабочего колеса

3.2 Построение лопаток колеса

В меню Construction выберите Plane, задайте построение плоскости на расстоянии L от XOZ (Рисунок 26).

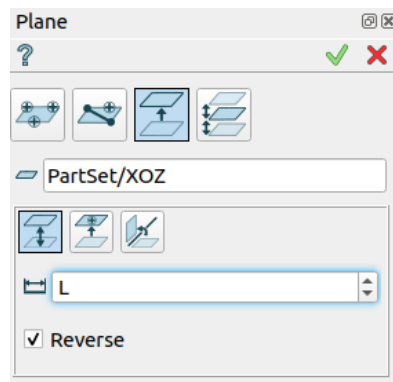


Рисунок 26 – Создание плоскости

На данной плоскости постройте эскиз. Измените видимость sketch, plane, revolution (Рисунок 27).

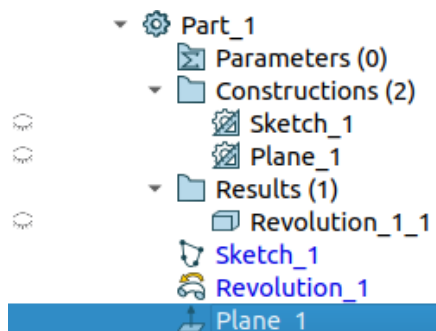


Рисунок 27 – Изменение видимости

Для начала проведите две окружности радиусами R_1 и R_2 соответственно (Рисунок 28).

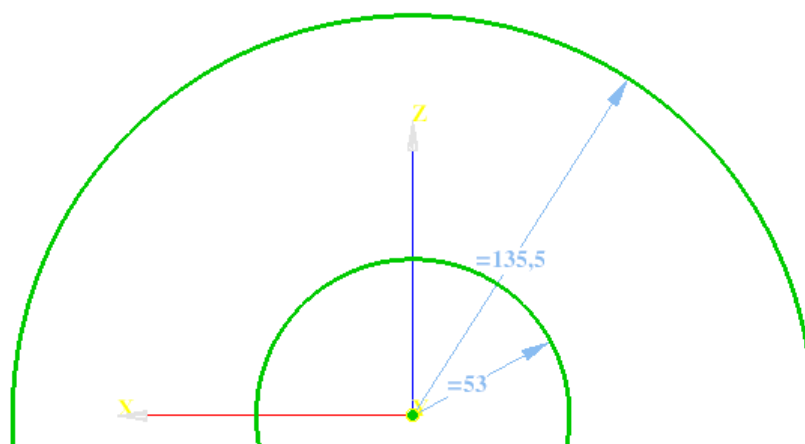


Рисунок 28 – Создание окружностей

Постройте осевыми линиями угол $\beta_1 + \beta_2$ с вершиной в точке 0 (Рисунок 29).

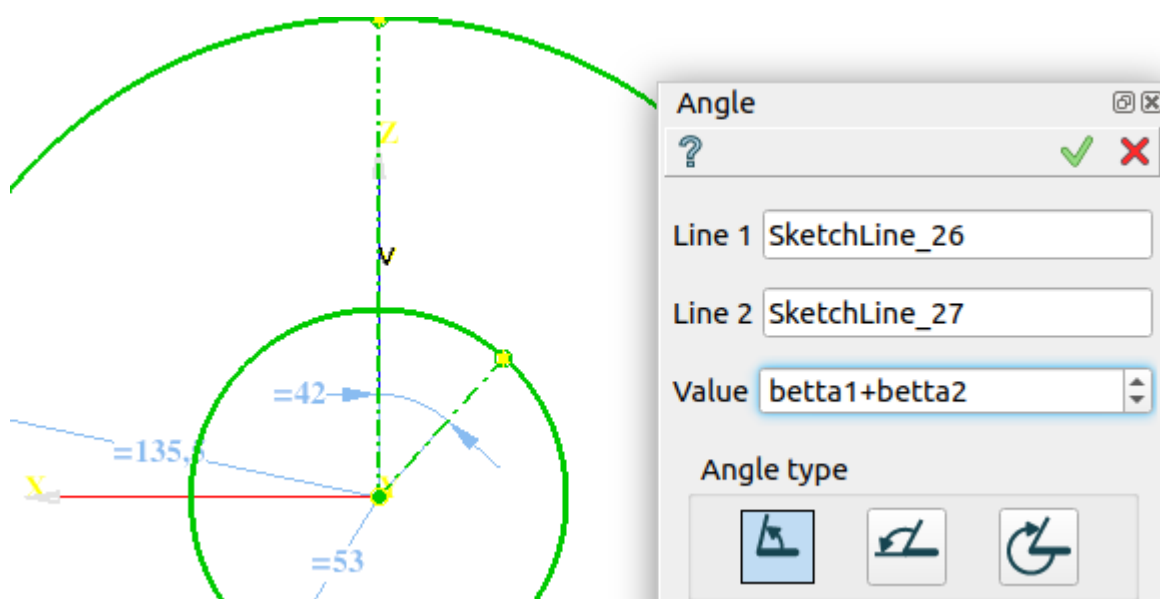


Рисунок 29 – Построение угла

Проведите еще одну линию из точки пересечения угла с окружностью радиуса R_2 , к ней постройте перпендикуляр к средней точке.

Соедините построенную линию с окружностью радиуса R_1 , затем точку пересечения угла $\beta_1 + \beta_2$ с окружностью радиуса R_1 и построенную линию. Должен получиться полностью определенный эскиз (Рисунок 30).

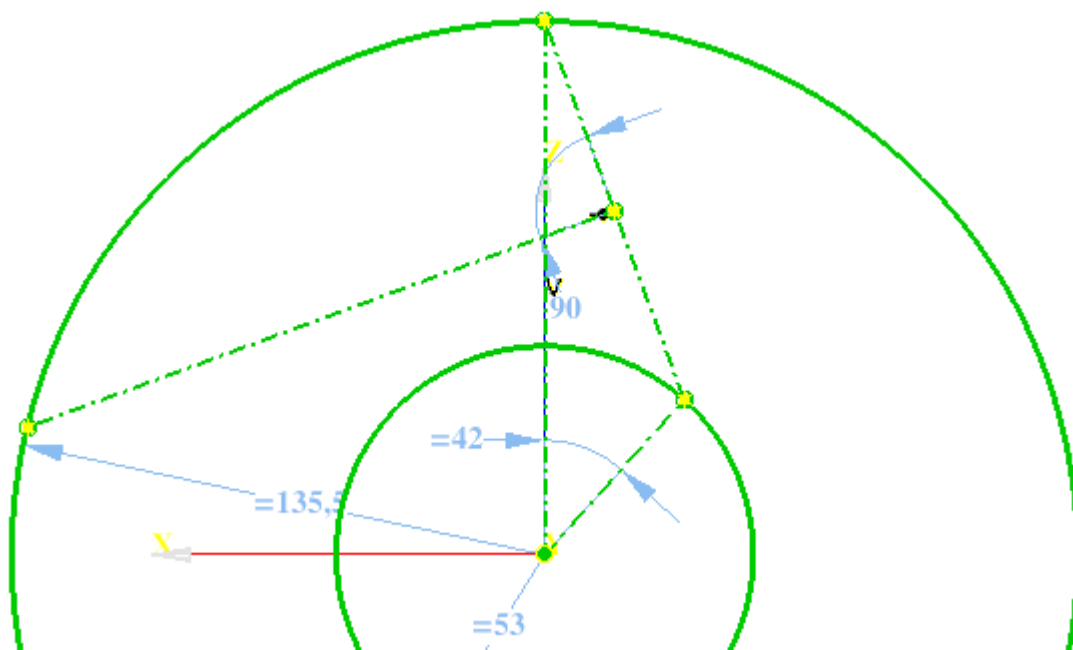


Рисунок 30 – Построение треугольника скоростей

A diagram of a three-lobed green shape. Internal dashed lines connect vertices. A red line labeled 'X' points to a central vertex. Angles are labeled: 15, 42, 53, 90, and 135. A yellow arrow points to a vertex.



Постройте еще две произвольные дуги. Соедините концы дуг осевыми линиями (Рисунок 33).

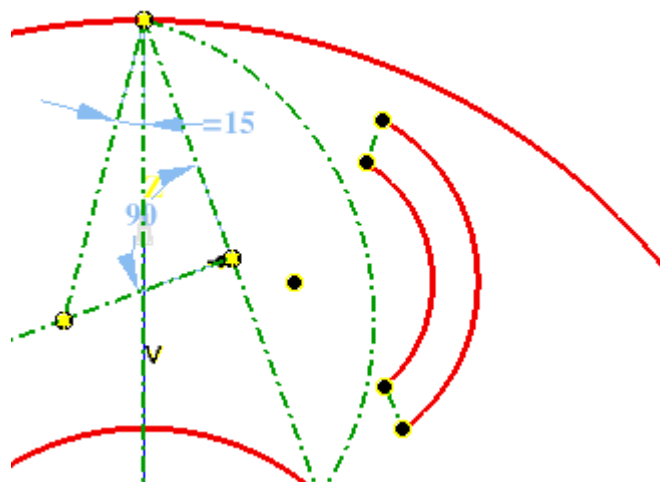


Рисунок 33 – Создание дуг

Постройте еще две дуги. Соедините точки с помощью зависимости Coincident, используйте зависимость касательной (Рисунок 34).

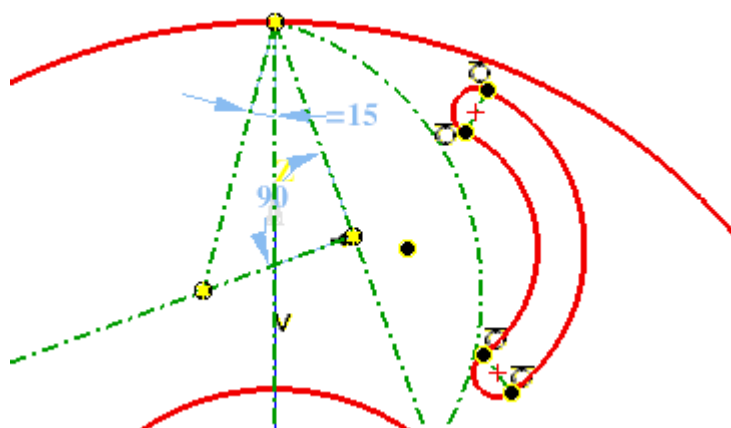


Рисунок 34 – Создание замкнутого контура

Ширину линии установите равной переменной width (Рисунок 35).

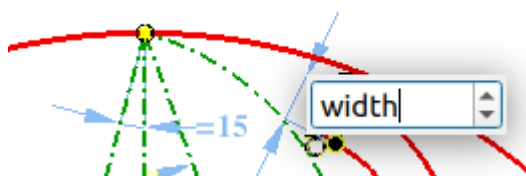


Рисунок 35 – Установка ширины лопатки

Соедините центры дуг с центром дуги лопатки, установите зависимость касательной с окружностями. Должен получиться профиль лопатки. Ранее проведенные окружности сделайте осевыми (Рисунок 36).

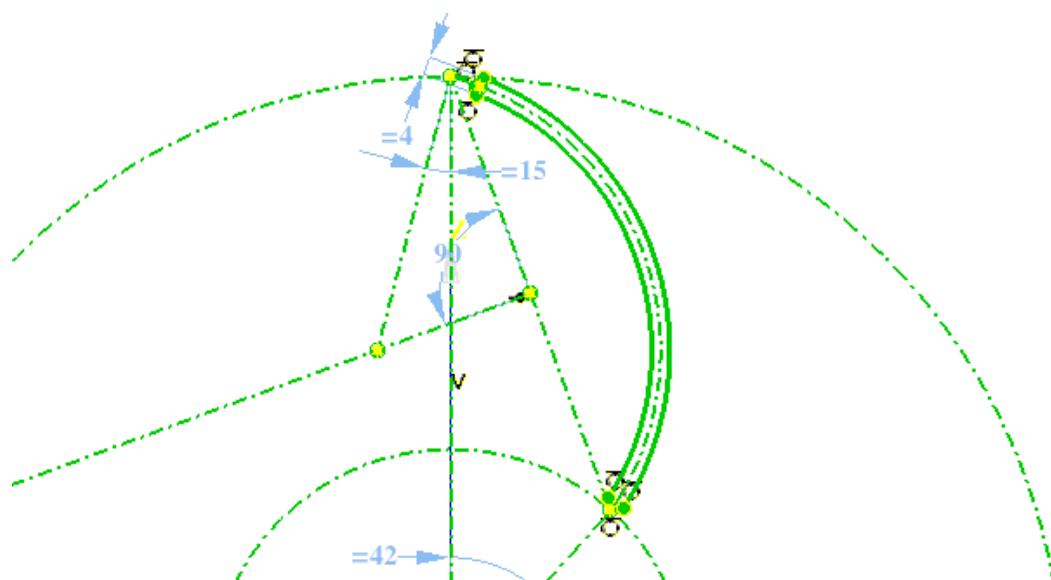


Рисунок 36 – Профиль лопатки

Создайте массив лопаток, используйте Angular copy (Рисунок 37).

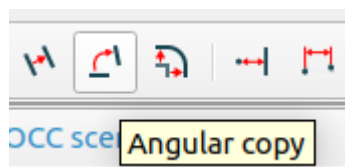


Рисунок 37 – Операция Angular copy

Выберите геометрию, центр вращения точку 0, для количества лопаток используйте переменную z, угол 360° (Рисунок 38).

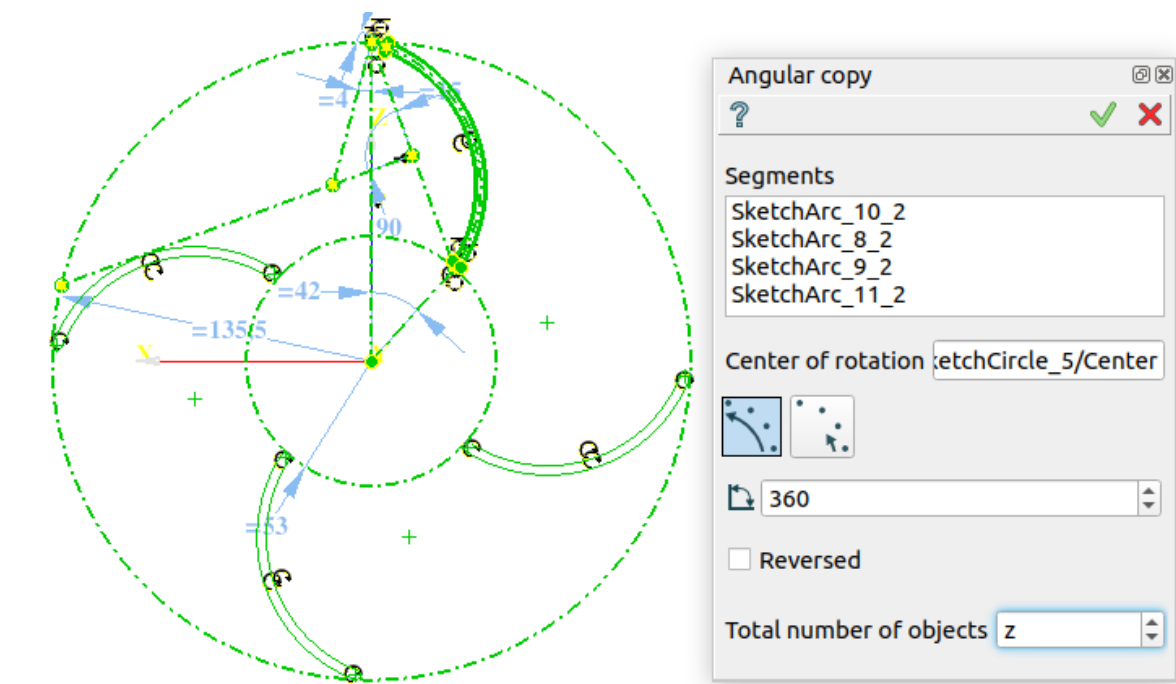


Рисунок 38 – Создание массива

Выходите из эскиза, сохраняйте изменения. Теперь выдавите вычитанием все, что находится в эскизе 2. В меню Features выберите ExtrusionCut. В качестве геометрии выберите эскиз 2, длину установите $0.9L$, вычитать будем из Revolution_1_1 (Рисунок 39).

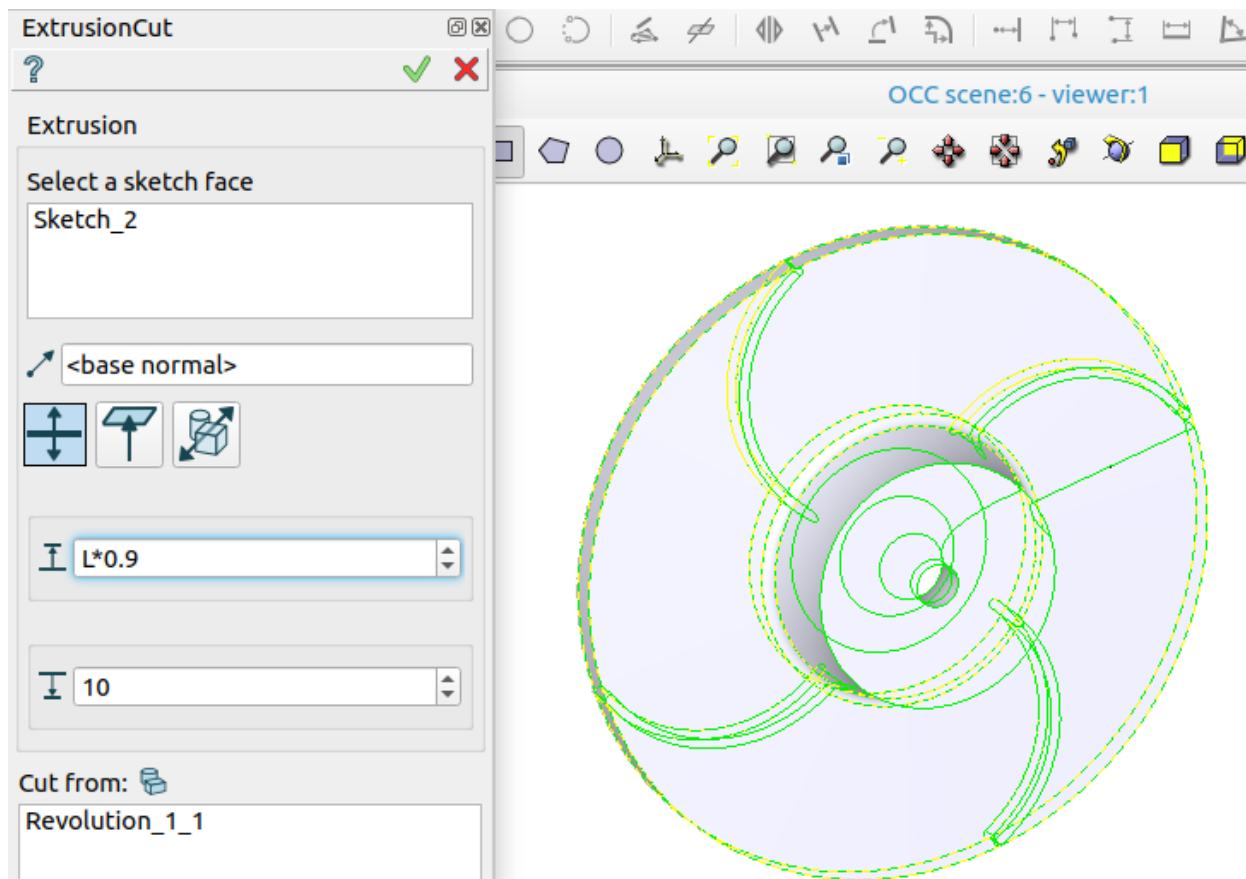


Рисунок 39 – Операция ExtrusionCut

Сохраните изменения. Создайте новый эскиз на плоскости Plane_1. Добавьте две окружности: окружность радиуса R_2 и окружность, удаленную от окружности радиуса R_2 на величину thick (Рисунок 40).

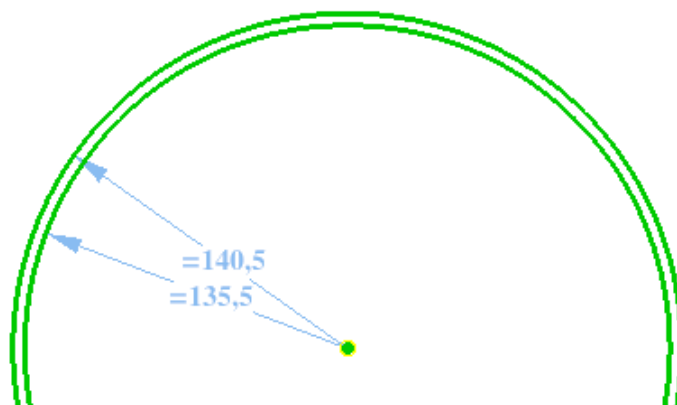


Рисунок 40 – Эскиз 3

Выполните операцию Extrusion на величину b2 (Рисунок 41).

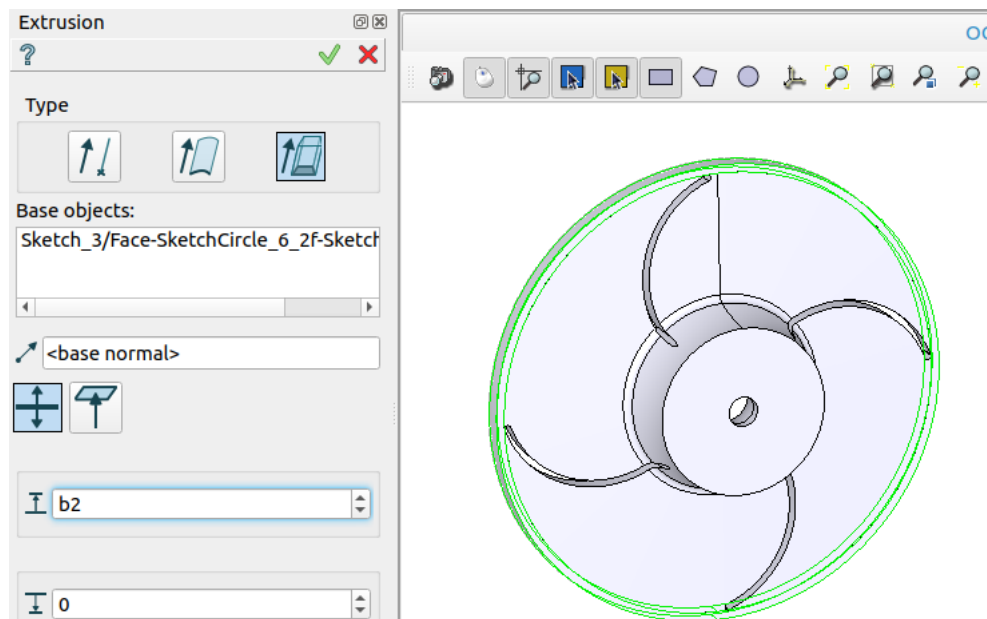


Рисунок 41 – Операция Extrusion

Объедините полученные результаты в один с помощью операции Fuse (Рисунок 42).

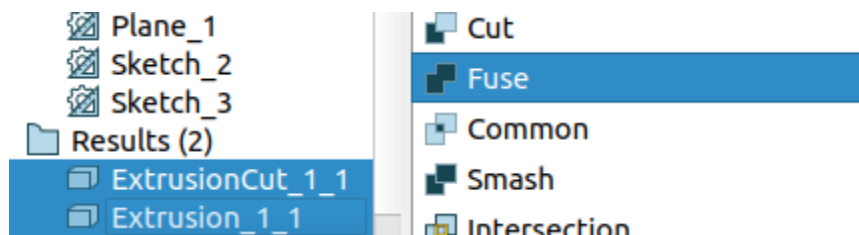


Рисунок 42 – Операция Fuse

Переименуйте результат (Рисунок 43).

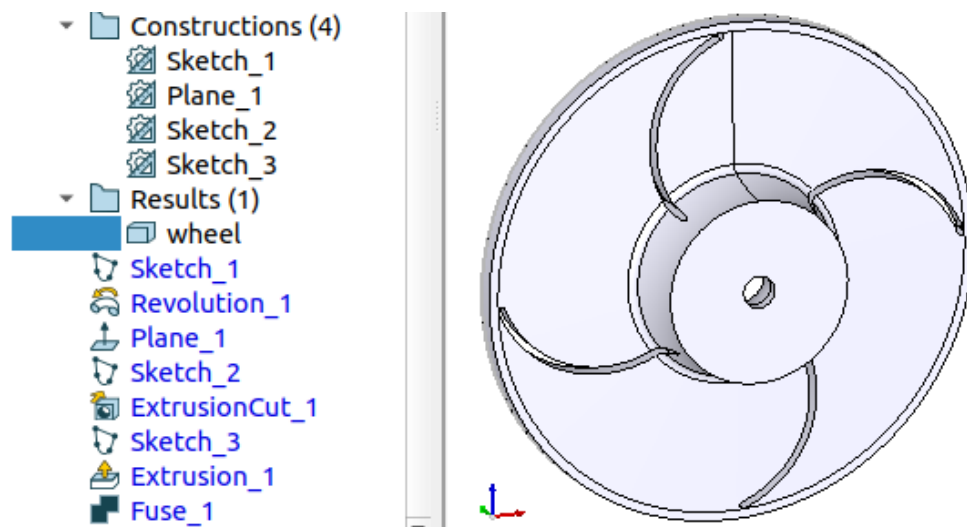


Рисунок 43 – Результат построения рабочего колеса

Сохраните файл, назовите его part1. Выберите Part => Parameters. Внесите изменения во входные параметры, используйте данные из первого расчета. Геометрия рабочего колеса должна построиться без ошибок (Рисунок 44).

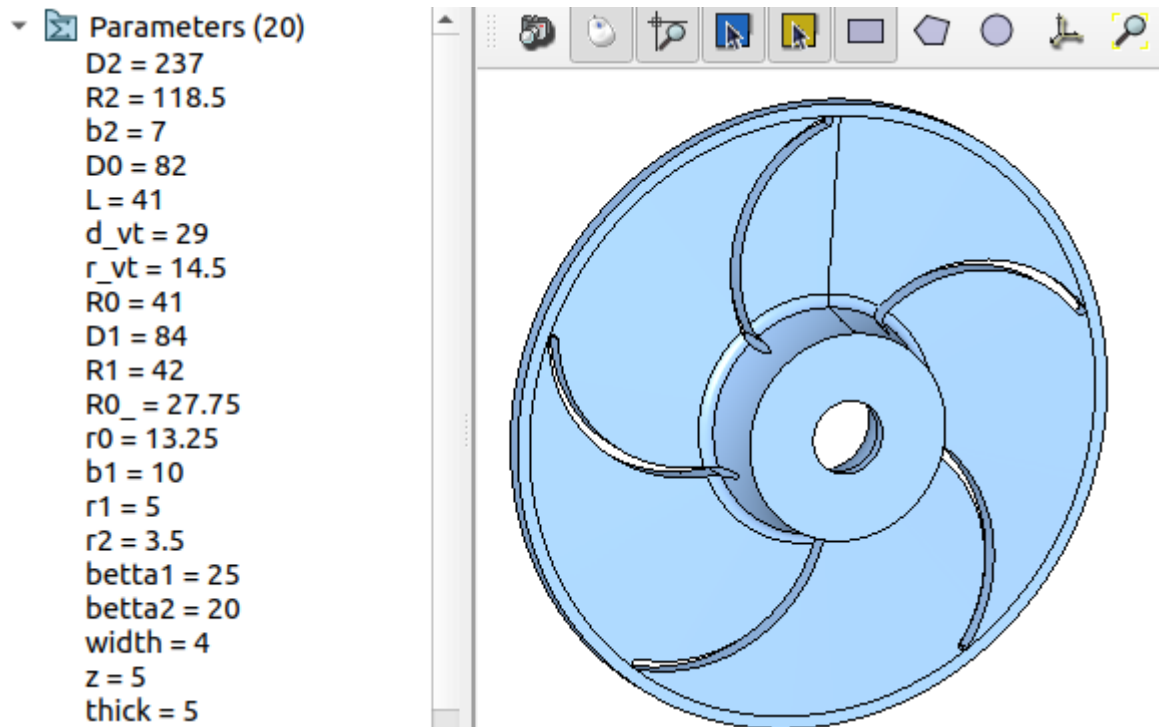


Рисунок 44 – Результат построения

Выполните команду File => Dump Study, сформируется скрипт, назовите его part1.py.

4 ИЗМЕНЕНИЕ КОДА ПОСТРОЕНИЯ РАБОЧЕГО КОЛЕСА ДЛЯ SALOME

Откройте файл part1.py, его необходимо изменить для параметрического построения рабочего колеса, предварительно сохраните исходный файл в part1_orig.py. При изменении только входных данных могут появляться ошибки из-за некорректных координат начальных точек. Добавьте в начало файла импорт библиотеки math.

Листинг 22 – Импорт библиотеки math

```
7 import sys
8 import salome
9 import math
```

Добавьте входные параметры для расчета из файла calculation.py.

Листинг 23 – Входные параметры

```
16 Q = 0.008 #м3/с
17 H = 9.7 #м
18 n = 1000.0 #rpm
19
20 rho = 1000.0 #kg/м3
21 g = 9.8 #м/с
22 omega = math.pi * n / 30.0
23 p1 = rho * g * H
```

Перенесите весь расчет из calculation.py. Используйте переменные для входных параметров в Salome, переведите необходимые в мм.

Листинг 23 – Использование переменных

```
239 partSet = model.moduleDocument()
240 model.addParameter(partSet, "D2", "D2*1000")
241 model.addParameter(partSet, "R2", "D2/2")
242 model.addParameter(partSet, "b2", "b2*1000")
243 model.addParameter(partSet, "D0", "D0*1000")
244 model.addParameter(partSet, "L", "D0/2")
245 model.addParameter(partSet, "d_vt", "d_vt*1000")
246 model.addParameter(partSet, "r_vt", "d_vt/2")
247 model.addParameter(partSet, "R0", "D0/2")
248 model.addParameter(partSet, "D1", "D1*1000")
249 model.addParameter(partSet, "R1", "D1/2")
250 model.addParameter(partSet, "R0_", "(R0-r_vt)/2+r_vt")
251 model.addParameter(partSet, "r0", "(R0-r_vt)/2")
252 model.addParameter(partSet, "b1", "b1*1000")
253 model.addParameter(partSet, "r1", "b1/2")
254 model.addParameter(partSet, "r2", "b2/2")
255 model.addParameter(partSet, "betta1", "betta1")
256 model.addParameter(partSet, "betta2", "betta2")
257 model.addParameter(partSet, "width", "width*1000")
258 model.addParameter(partSet, "z", "z")
259 model.addParameter(partSet, "thick", "5")
```

Найдите строки, отвечающие за формирование массива лопаток.
Удалите объявление дуг и их порядок.

Листинг 24 – Создание массива

```
668 ### Create SketchMultiRotation
669 SketchMultiRotation_1_objects = [SketchArc_10.results()[1],
    SketchArc_8.results()[1], SketchArc_9.results()[1],
    SketchArc_11.results()[1]]
670 SketchMultiRotation_1 =
    Sketch_2.addRotation(SketchMultiRotation_1_objects,
    SketchCircle_5.center(), 360, "z", True)
671 model.do()
```

Сохраните изменения, загрузите скрипт в Salome, убедитесь, что все строится без ошибок.

Обеспечим параметрическое построение первого эскиза.

Первую осевую вертикальную линию будем строить на расстоянии $L+10$ от оси X. Объявите новую переменную L, которая равна половине диаметра входа в рабочее колесо, переведите в мм.

Листинг 25 – Изменение координаты осевой линии

```
261 L = D0/2*1000
262
263 ### Create Part
264 Part_1 = model.addPart(partSet)
265 Part_1_doc = Part_1.document()
266
267 ### Create Sketch
268 Sketch_1 = model.addSketch(Part_1_doc,
    model.defaultPlane("YOZ"))
269
270 ### Create SketchLine
271 SketchLine_1 = Sketch_1.addLine(L+10, 0, L+10, 118.5)
272 SketchLine_1.setAuxiliary(True)
```

Начало линии b2 будет в точке $L+10$, окончание можно поставить любую точку левее, можем принять L. Значения на другой оси менять не обязательно, так как далее следует зависимость соединения.

Листинг 26 – Построение линии 3

```
281 ### Create SketchLine
282 SketchLine_3 = Sketch_1.addLine(L+10, 118.5, L, 118.5)
283 Sketch_1.setCoincident(SketchLine_1.endPoint(),
    SketchLine_3.startPoint())
284 Sketch_1.setHorizontal(SketchLine_3.result())
285 Sketch_1.setLength(SketchLine_3.result(), "b2")
```

Линия 4 должна начинаться в точке $L+10$, конец также можно установить в точке L . Координаты по оси Z изменять не обязательно. Далее происходит установка длины отрезка и расстояния до оси OX .

Листинг 27 – Построение линии 4

```
287 ### Create SketchLine
288 SketchLine_4 = Sketch_1.addLine(L+10, 14.5, L, 14.5)
289 SketchLine_4.setAuxiliary(True)
290 Sketch_1.setCoincident(SketchLine_4.startPoint(),
    SketchLine_1.result())
291 Sketch_1.setHorizontal(SketchLine_4.result())
292 Sketch_1.setLength(SketchLine_4.result(), "L")
293 Sketch_1.setVerticalDistance(SketchLine_4.endPoint(),
    SketchAPI_Line(SketchLine_2).startPoint(), "r_vt")
```

Начало линии 5 можно оставить в точке 0, значения по оси Z могут быть любыми, но первое должно быть меньше второго, так как далее отрезок будет соединяться зависимостями, учитывая начальную и конечную точки.

Листинг 28 – Построение линии 5

```
295 ### Create SketchLine
296 SketchLine_5 = Sketch_1.addLine(0, 14.5, 0, 41)
297 Sketch_1.setCoincident(SketchLine_4.endPoint(),
    SketchLine_5.startPoint())
298 Sketch_1.setVertical(SketchLine_5.result())
299 Sketch_1.setVerticalDistance(SketchLine_5.endPoint(),
    SketchAPI_Line(SketchLine_2).startPoint(), "R0")
```

Далее создаются 4 отрезка и 4 точки в горизонтальном и вертикальном направлении, координаты их построения можно оставить без изменения, благодаря зависимостям, построение должно проходить корректно.

Внесем изменения в построение окружностей радиусами r_1 и r_2 . Найдите соответствующие строки. Задайте координату по оси X , максимально приближенную к точке 0. Это необходимо для того, чтобы окружность не построилась касательной с другой стороны. Установите координаты по оси Z . Для окружности радиуса r_2 используйте $R2$, для окружности радиуса r_1 $L + r_vt$. Объявите использование данных переменных. Переведите их значения в мм.

Листинг 29 – Построение окружности 2 и 3

```
510 ### Create SketchCircle
511 SketchCircle_2 = Sketch_1.addCircle(1.0, R2, 5.5)
512 SketchCircle_2.setAuxiliary(True)
513
514 ### Create SketchCircle
515 SketchCircle_3 = Sketch_1.addCircle(1.0, L+r_vt, 7.5)
516 SketchCircle_3.setAuxiliary(True)
517 Sketch_1.setCoincident(SketchCircle_2.center(),
    SketchLine_3.result())
518 Sketch_1.setRadius(SketchCircle_2.results()[1], "r2")
519 Sketch_1.setRadius(SketchCircle_3.results()[1], "r1")
520 Sketch_1.setTangent(SketchCircle_2.results()[1],
    SketchLine_1.result())
521 Sketch_1.setTangent(SketchArc_4.results()[1],
    SketchCircle_3.results()[1])
```

При построении дуги можем использовать следующие точки для обеспечения корректной установки зависимости касательной.

Листинг 30 – Построение дуги 5

```
532 ### Create SketchArc
533 SketchArc_5 = Sketch_1.addArc(0, R1*1.5, -10.0, R1*1.5, 10.0,
    R1*1.5, False)
534 Sketch_1.setCoincident(SketchLine_6.result(),
    SketchArc_5.center())
535 Sketch_1.setCoincident(SketchLine_5.endPoint(),
    SketchArc_5.startPoint())
```

Следующую точку линии нужно строить на расстоянии L-b2 и R2 соответственно.

Листинг 31 – Построение линии 23

```
533 ### Create SketchLine
534 SketchLine_23 = Sketch_1.addLine(L-b2, R2, L-2*b2, R2-2*b2)
535 Sketch_1.setCoincident(SketchLine_3.endPoint(),
    SketchLine_23.startPoint())
```

Добавьте объявление переменных.

Листинг 32 – Объявление переменных

```
261 L = D0/2*1000
262 R2 = D2/2*1000
263 r_vt = d_vt/2*1000
264 R1 = D1/2*1000
265 b2 = b2*1000
```

Измените координаты линии 24. Убедитесь в корректной установке зависимостей соединения.

Листинг 33 – Построение линии 24

```
541 ### Create SketchLine
542 SketchLine_24 = Sketch_1.addLine(L, R2, L, R2-1.0)
543 Sketch_1.setCoincident(SketchLine_1.endPoint(),
    SketchLine_24.startPoint())
544 Sketch_1.setCoincident(SketchLine_14.endPoint(),
    SketchLine_24.endPoint())
```

Измените координаты дуги скругления ведомого диска. Сначала записывается центральная точка, затем остальные две. Убедитесь в корректном порядке зависимостей, иначе могут возникнуть ошибки. Центральная точка строится максимально близко к оси, затем с помощью зависимостей устанавливается.

Листинг 34 – Построение дуги 6

```
556 ### Create SketchArc
557 SketchArc_6 = Sketch_1.addArc(L/10, R1*1.5, L/10, R1*1.4, L/9,
    R1*1.5, False)
558 Sketch_1.setCoincident(SketchArc_6.startPoint(),
    SketchArc_5.endPoint())
559 Sketch_1.setCoincident(SketchArc_6.endPoint(),
    SketchLine_23.endPoint())
560 Sketch_1.setTangent(SketchArc_6.results()[1],
    SketchLine_23.result())
561 Sketch_1.setTangent(SketchArc_6.results()[1],
    SketchArc_5.results()[1])
562 Sketch_1.setRadius(SketchArc_6.results()[1], 5)
563 model.do()
```

Теперь внесите изменения во второй эскиз. В нем могут возникнуть ошибки при построении профиля лопатки. Созданные дуги не должны выходить за пределы окружностей радиусов R_1 и R_2 . Первая точка дуги 8 должна лежать справа от оси X, остальные построим на оси OZ. Объявите переменную width.

Листинг 35 – Объявление переменной

```
266 width = width*1000
```

Для корректной установки зависимостей дуги будут построены на оси OZ радиусом можно принять 2 ширины лопатки. Центр сместите вправо на величину width. Координата по оси Z находится между радиусами окружностей. По аналогии установите координаты второй окружности.

Листинг 36 – Построение дуг

```
640 ### Create SketchArc
641 SketchArc_8 = Sketch_2.addArc(width, R1+(R2-R1)/2, 0.0, R1+(R2-R1)/
    2-2.0*width, 0.0, R1+(R2-R1)/2+2.0*width, True)
642 SketchArc_9 = Sketch_2.addArc(width, R1+(R2-R1)/2, -width, R1+(R2-R1)/
    2-2.0*width, -width, R1+(R2-R1)/2+2.0*width, True)
```

Соедините крайние точки дуг осевыми линиями. Измените координаты, согласно построенным выше дугам. Проверьте порядок установки зависимостей.

Листинг 37 – Построение отрезков

```
645 ### Create SketchLine
646 SketchLine_31 = Sketch_2.addLine(-width, R1+(R2-R1)/
    2+2.0*width, 0.0, R1+(R2-R1)/2+2.0*width)
647 SketchLine_31.setAuxiliary(True)
648 Sketch_2.setCoincident(SketchArc_9.startPoint(),
    SketchLine_31.startPoint())
649 Sketch_2.setCoincident(SketchArc_8.startPoint(),
    SketchLine_31.endPoint())
650
651 ### Create SketchLine
652 SketchLine_32 = Sketch_2.addLine(-width, R1+(R2-R1)/
    2-2.0*width, 0.0, R1+(R2-R1)/2-2.0*width)
653 SketchLine_32.setAuxiliary(True)
654 Sketch_2.setCoincident(SketchArc_9.endPoint(),
    SketchLine_32.startPoint())
655 Sketch_2.setCoincident(SketchArc_8.endPoint(),
    SketchLine_32.endPoint())
```

Измените координаты создания двух соединяющих дуг. Они должны располагаться максимально близко к созданным ранее дугам. Обратите внимание на направление построения (true, false).

Листинг 38 – Построение дуг

```
657 ### Create SketchArc
658 SketchArc_10 = Sketch_2.addArc(-width/2, R1+(R2-R1)/
    2+2.0*width, -width, R1+(R2-R1)/2+2.0*width, 0.0, R1+(R2-R1)/
    2+2.0*width, True)
659
660 ### Create SketchArc
661 SketchArc_11 = Sketch_2.addArc(-width/2, R1+(R2-R1)/
    2-2.0*width, -width, R1+(R2-R1)/2-2.0*width, 0.0, R1+(R2-R1)/
    2-2.0*width, False)
```

Проверьте зависимости. При построении профиля лопатки могут возникнуть ошибки, если зависимости будут применены в отличном от представленного порядке.

Листинг 39 – Установка зависимостей

```
733 Sketch_2.setTangent(SketchArc_10.results()[1],  
    SketchArc_9.results()[1])  
734 Sketch_2.setTangent(SketchArc_8.results()[1],  
    SketchArc_10.results()[1])  
735 Sketch_2.setTangent(SketchArc_9.results()[1],  
    SketchArc_11.results()[1])  
736 Sketch_2.setTangent(SketchArc_11.results()[1],  
    SketchArc_8.results()[1])  
737 Sketch_2.setCoincident(SketchArc_10.startPoint(),  
    SketchArc_9.endPoint())  
738 Sketch_2.setCoincident(SketchArc_10.endPoint(),  
    SketchArc_8.endPoint())  
739 Sketch_2.setCoincident(SketchArc_11.startPoint(),  
    SketchLine_31.startPoint())  
740 Sketch_2.setCoincident(SketchArc_11.endPoint(),  
    SketchLine_31.endPoint())  
741 Sketch_2.setLength(SketchLine_32.result(), "width")  
742 Sketch_2.setCoincident(SketchArc_8.center(),  
    SketchLine_30.endPoint())  
743 Sketch_2.setCoincident(SketchArc_10.center(),  
    SketchArc_7.results()[1])  
744 Sketch_2.setTangent(SketchArc_11.results()[1],  
    SketchCircle_4.results()[1])  
745 Sketch_2.setTangent(SketchArc_10.results()[1],  
    SketchCircle_5.results()[1])
```

Сохраните изменения. Проверьте работу кода в Salome.

Выполните проверочный расчет. Входные параметры: $Q = 0.007 \text{ м}^3/\text{с}$, $H = 40 \text{ м}$, $n = 2900 \text{ об/мин}$. Результат должен быть, как на рисунке 45.

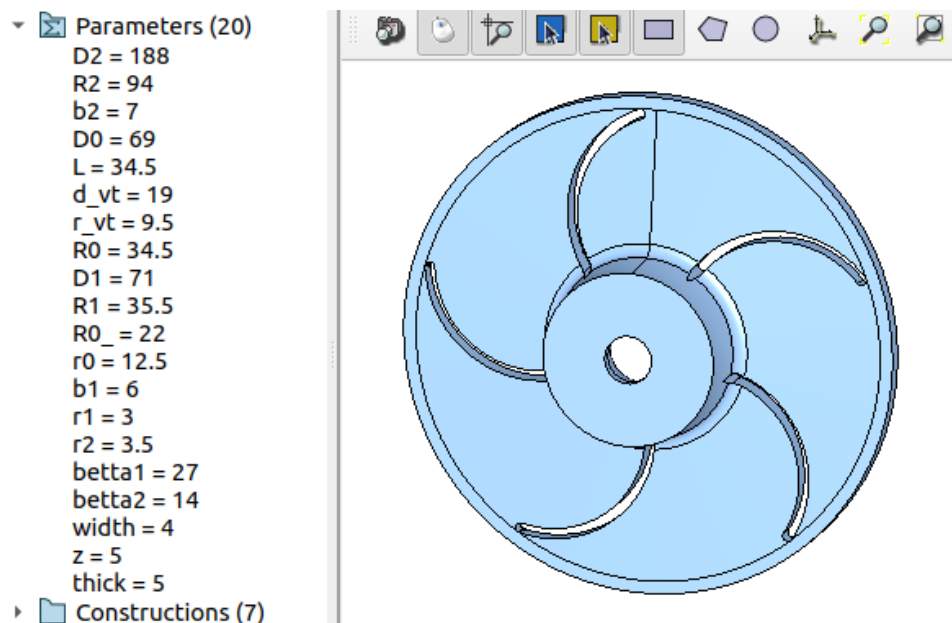


Рисунок 45 – Результат построения рабочего колеса

5 РАСЧЕТ СПИРАЛЬНОГО ОТВОДА

Для расчета используются коэффициенты K_c и K_H , зависящие от коэффициента быстроходности n_s . Определяются по графику на рисунке 46.

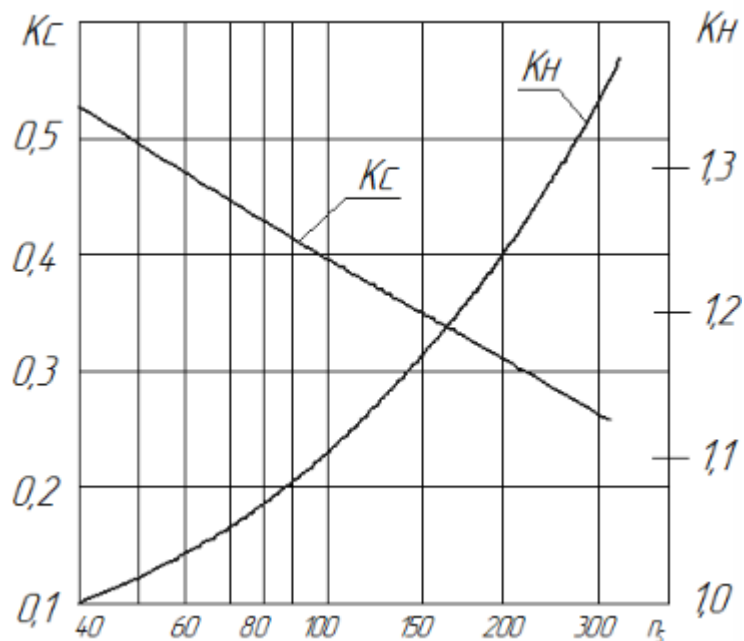


Рисунок 46 – График зависимости коэффициентов K_c и K_H , от n_s

Для тихоходных центробежных насосов коэффициент быстроходности лежит в пределах 50 – 80. Для трех расчетов, которые выполняются в ходе данного урока, коэффициент быстроходности находится в интервале от 52,3 до 59,4, поэтому можем принять коэффициенты K_c и K_H равные 0,47 и 1,04 соответственно.

Порядок расчета основных размеров отводного канала корпуса:

1. Определение радиуса внутренней окружности корпуса:

$$R_3 = K_H \cdot R_2, \text{ м} \quad (41)$$

2. Определение средней скорости протекания жидкости в сечении:

$$V_c = K_c \sqrt{2 \cdot g \cdot H}, \text{ м/с} \quad (42)$$

3. Определение расхода через каждое сечение:

$$q_i = \frac{Q \cdot \varphi_i}{360}, \text{ м}^3/\text{с} \quad (43)$$

где φ_i – угол положения сечения от начала улитки в градусах

4. Определение площади каждого сечения:

$$f_i = \frac{q_i}{V_c}, \text{ м}^2 \quad (44)$$

5. Определение радиуса каждого сечения:

$$r_i = \sqrt{\frac{f_i}{\pi}}, \text{ м} \quad (45)$$

6. Определение диаметра нагнетательного патрубка (на выходе из диффузора):

$$d_n = \sqrt{\frac{4 \cdot Q}{\pi \cdot V_n}}, \text{ м} \quad (46)$$

где V_n – скорость в нагнетательном патрубке (4 – 6.5) м/с

7. Определение длины диффузора:

$$L_d = \frac{d_n - d_{12}}{2tg \frac{\theta}{2}}, \text{ м} \quad (47)$$

где d_{12} – диаметр последнего сечения,

θ – угол конусности диффузора ($8^\circ - 10^\circ$)

6 РЕАЛИЗАЦИЯ РАСЧЕТА СПИРАЛЬНОГО ОТВОДА НА PYTHON

Скопируйте файл part1.py в ту же директорию, назовите новый файл part1_1. Внесите в него дополнения. Задайте входным параметрам следующие значения: $Q = 0.007 \text{ м}^3/\text{с}$, $H = 40 \text{ м}$, $n = 2900 \text{ об/мин}$.

Создайте 3 массива для хранения значений расхода, площади и радиуса каждого сечения. Объявите глобальные переменные: угол конусности диффузора, радиус внутренней окружности корпуса, средняя скорость протекания жидкости в сечении, диаметр нагнетательного патрубка, длина диффузора, коэффициенты, отвечающие за завершение выполнения расчета рабочего колеса.

Листинг 40 – Объявление глобальных переменных

```
119 q = []
120 f = []
121 r = []
122 tetta = 9.0
123 R3 = 0.0
124 Vc = 0.0
125 dn = 0.0
126 Ld = 0.0
127 k1_done = False
128 k2_done = False
```

В функцию funcErr добавьте использование глобальных переменных, присвоение значений переменным k1_done и k2_done при завершении расчета рабочего колеса. Задайте условие вызова функции func3.

Листинг 41 – Изменение функции funcErr

```
207 def funcErr():
208     global k2, k1, k1_done, k2_done
209
210     err_k1 = k1_ * 100.0 / k1
211     print('err_k1', err_k1)
212
213     if (math.fabs(100.0 - err_k1) > 10.0 and k1 < 1.15):
214         print('k1 again')
215         k1 = k1 + 0.01
216         func1(k1)
217         func2(k2)
218     elif k1 > 1.15:
219         print('k1 more than 1.15')
```

Листинг 42 – Функция funcErr продолжение

```
220         else:
221             print('k1 done')
222             k1_done = True
223
224         err_k2 = 100.0 * k2_ / k2
225         print('err_k2', err_k2)
226
227         if (math.fabs(100.0 - err_k2) > 10.0 and k2 < 1.1):
228             print('k2 again')
229             k2 = k2 + 0.01
230             func2(k2)
231             funcErr()
232         elif k2 > 1.1:
233             print('k2 more than 1.1')
234         else:
235             print('k2 done')
236             k2_done = True
237
238         if (k1_done == True and k2_done == True):
239             func3()
240         else:
241             funcErr()
```

Создайте функцию func3. Объявите использование глобальных переменных, присвойте трем массивам пустые значения. Согласно формулам определите радиус внутренней окружности корпуса и среднюю скорость протекания жидкости в сечении. Установите начальное значение угла 30°. Объявите два цикла. В первом будут вычисляться значения расхода через каждое сечение, во втором – значения площадей и радиусов. Скорость в нагнетательном патрубке была выбрана 5 м/с. Согласно формуле вычисляется диаметр на выходе из диффузора. Угол конусности был выбран 9°. При вычислении длины диффузора необходимо выполнить перевод из градусов в радианы. Все значения округляются с помощью round.

Листинг 43 – Функция func3

```
207 def func3():
208     global R3, D2, Vc, g, H, q, f, r, Q, Ld, dn, tetta
209     q = []
210     f = []
211     r = []
212     Kc = 0.47
213     Kn = 1.04
214     R3 = round(Kn * D2 / 2.0, 3)
215     print('R3', R3)
```

Листинг 44 – Функция func3 продолжение

```
216     Vc = round(Kc * math.sqrt(2.0 * g * H), 3)
217     print('Vc', Vc)
218
219     angle = 30.0
220     while angle <= 360.0:
221         q.append(round(Q*angle/360.0,5))
222         angle += 30.0
223
224     for i in range(12):
225         f.append(round(q[i]/Vc,4))
226         r.append(round(math.sqrt(f[i]/math.pi),3))
227
228     print('q[]', q)
229     print('f[]', f)
230     print('r[]', r)
231
232     Vn = 5.0
233     dn = round(math.sqrt(4.0 * Q/math.pi/Vn),3)
234     print('dn', dn)
235
236     Ld = round((dn - r[11]*2.0)/2.0/
237               math.tan(math.radians(tetta)/2.0),3)
237     print('Ld', Ld)
```

Выполненный расчет в терминале показан в листинге 45. Чтобы не загружать весь код в Salome, добавленные функции можете скопировать в файл calculation.py и запустить только его.

Листинг 45 – Расчет спирального отвода

```
R3 0.098
Vc 13.16
q[] [0.00058, 0.00117, 0.00175, 0.00233, 0.00292, 0.0035, 0.00408, 0.00467, 0.00525, 0.00583, 0.00642, 0.007]
f[] [0.0, 0.0001, 0.0001, 0.0002, 0.0002, 0.0003, 0.0003, 0.0004, 0.0004, 0.0004, 0.0005, 0.0005]
r[] [0.0, 0.006, 0.006, 0.008, 0.008, 0.01, 0.01, 0.011, 0.011, 0.011, 0.013, 0.013]
dn 0.042
Ld 0.102
```

Как видите, радиус корпуса спирального отвода в первом сечении под углом 30° слишком мал, что при округлении равен 0. На данной плоскости будем использовать радиус второго сечения.

7 ПОСТРОЕНИЕ СПИРАЛЬНОГО ОТВОДА

Откройте part1_1.py, добавьте использование переменных для построения. Все длины должны быть переведены в мм. Радиусу первого сечения присвойте второй элемент массива.

Листинг 46 – Добавление переменных

```
314 model.addParameter(partSet, "R3", "R3*1000")
315 model.addParameter(partSet, "r1_", "r[1]*1000")
316 model.addParameter(partSet, "r2_", "r[1]*1000")
317 model.addParameter(partSet, "r3_", "r[2]*1000")
318 model.addParameter(partSet, "r4_", "r[3]*1000")
319 model.addParameter(partSet, "r5_", "r[4]*1000")
320 model.addParameter(partSet, "r6_", "r[5]*1000")
321 model.addParameter(partSet, "r7_", "r[6]*1000")
322 model.addParameter(partSet, "r8_", "r[7]*1000")
323 model.addParameter(partSet, "r9_", "r[8]*1000")
324 model.addParameter(partSet, "r10_", "r[9]*1000")
325 model.addParameter(partSet, "r11_", "r[10]*1000")
326 model.addParameter(partSet, "r12_", "r[11]*1000")
327 model.addParameter(partSet, "dn", "dn*1000")
328 model.addParameter(partSet, "Ld", "Ld*1000")
329 model.addParameter(partSet, "tetta", "tetta")
```

Запустите part1_1.py в Salome, проверьте, что данные были успешно добавлены в параметры (Рисунок 47).

R3	R3*1000	98	
r1_	r[1]*1000	6	
r2_	r[1]*1000	6	
r3_	r[2]*1000	6	
r4_	r[3]*1000	8	
r5_	r[4]*1000	8	
r6_	r[5]*1000	10	
r7_	r[6]*1000	10	
r8_	r[7]*1000	11	
r9_	r[8]*1000	11	
r10_	r[9]*1000	11	
r11_	r[10]*1000	13	
r12_	r[11]*1000	13	
dn	dn*1000	42	
Ld	Ld*1000	102	
tetta	tetta	9	

Рисунок 47 – Добавление параметров

Выполните Part => New Part. На плоскости XOZ в эскизе создайте окружность радиусом R3, добавьте величину заданной толщины thick (Рисунок 48).

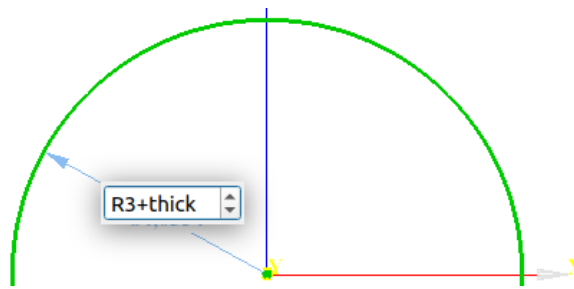


Рисунок 48 – Создание окружности

Создайте 4 осевые линии с шагом 30° от оси ОХ (Рисунок 49). На этих линиях будут располагаться вспомогательные окружности, необходимые для получения профиля.

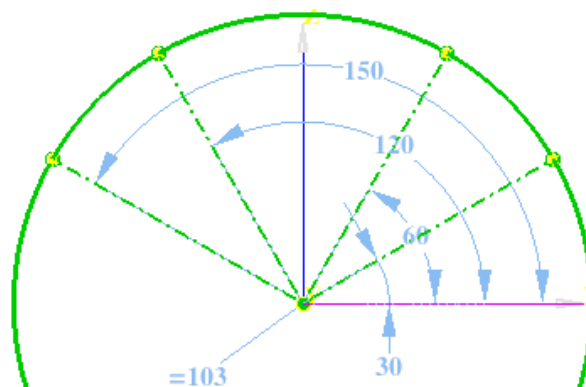


Рисунок 49 – Создание вспомогательных линий

Создайте 12 окружностей с центрами на осевых линиях (зависимость Coincident), радиусами $r1_$ – $r12_$, касательными к центральной окружности (Рисунок 50).

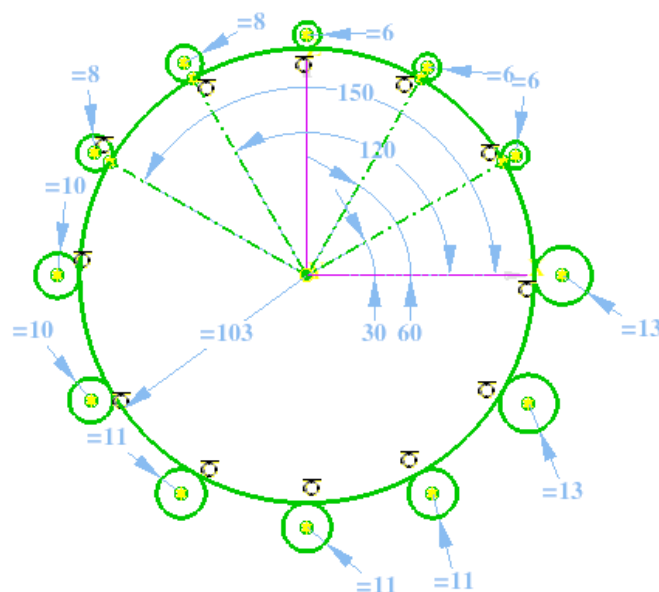


Рисунок 50 – Создание окружностей профиля

Из точки пересечения окружности радиуса r_{12} с центральной окружностью проведите вертикальную линию на высоту L_d (Рисунок 51).

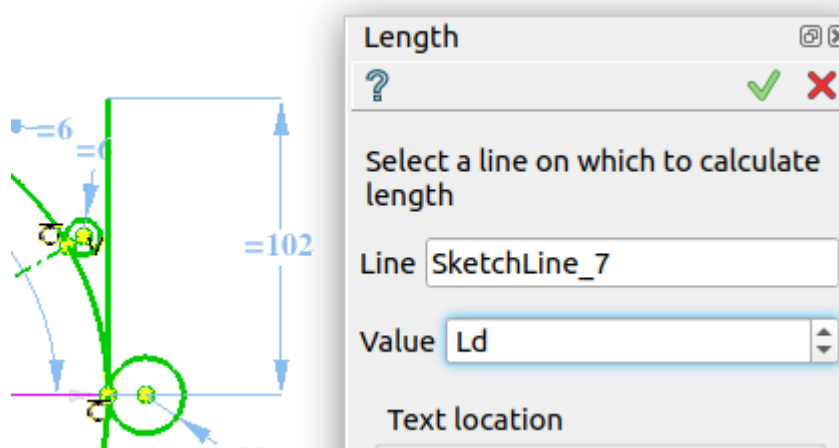


Рисунок 51 – Установка длины диффузора

Постройте выходную часть и установите диаметр нагнетательного патрубка d_n (Рисунок 52).

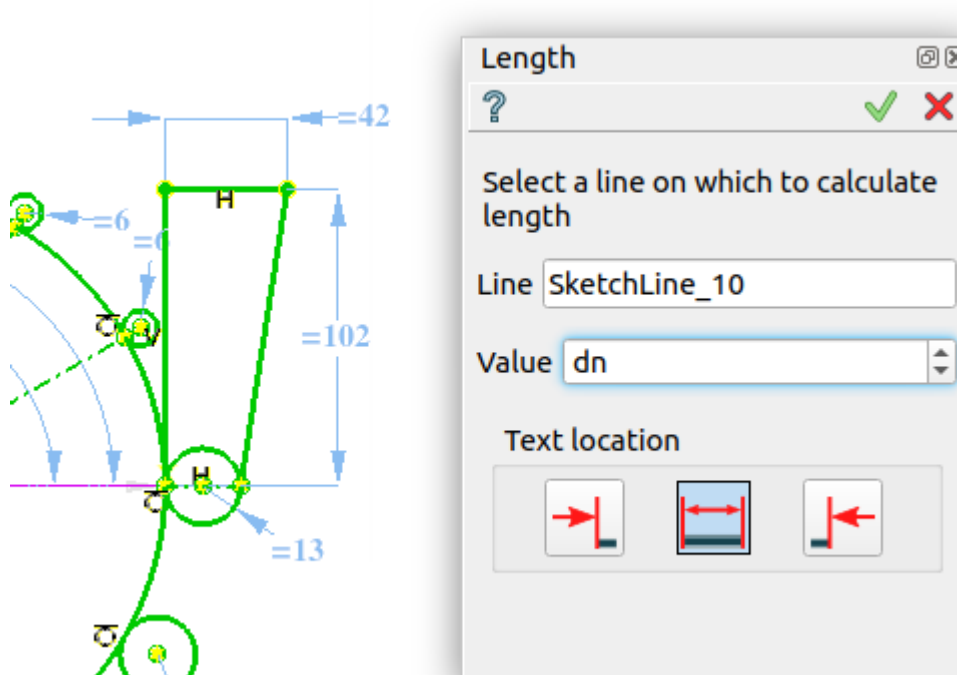


Рисунок 52 – Установка диаметра нагнетательного патрубка

Создайте новый эскиз на той же плоскости. Последовательно проводите дуги по трем точкам (Рисунок 53). Последнюю дугу стройте из центра, конец соедините с осью X. Также создайте линию к средней точке выходного диаметра. Создайте окружность радиусом $R_2 + \text{thick}$.

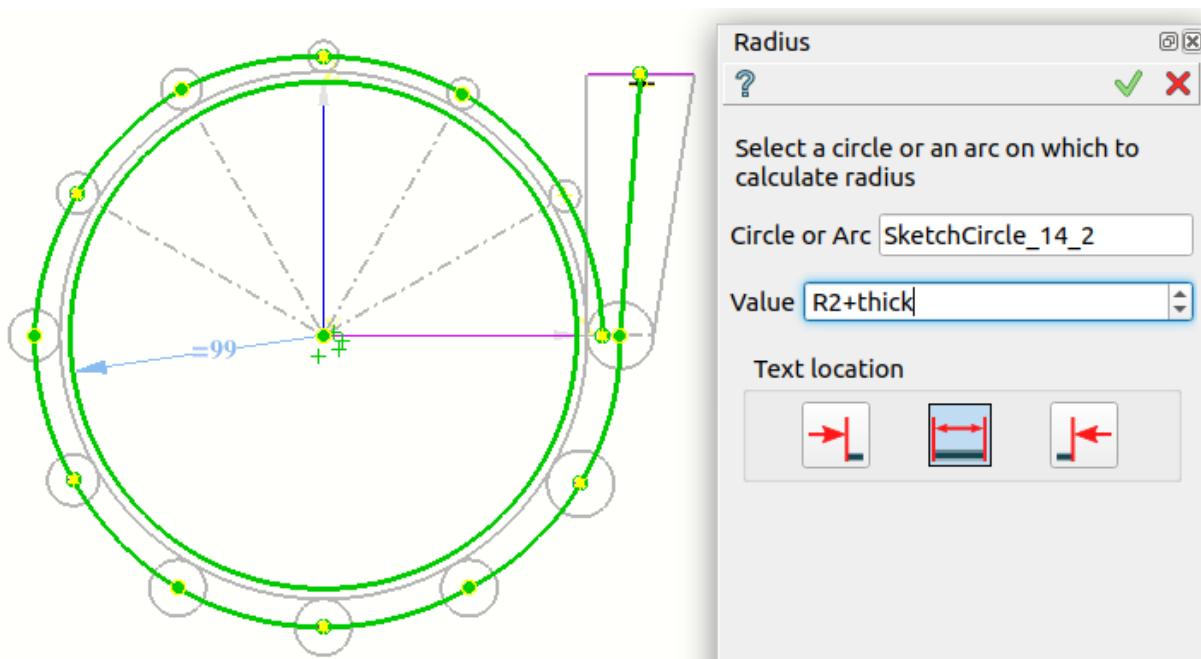


Рисунок 53 – Эскиз 2

Создайте 4 плоскости, удаленные от плоскости XOY на 30°, 60°, 120° и 150°. В меню Construction выберите Plane (Рисунок 54).

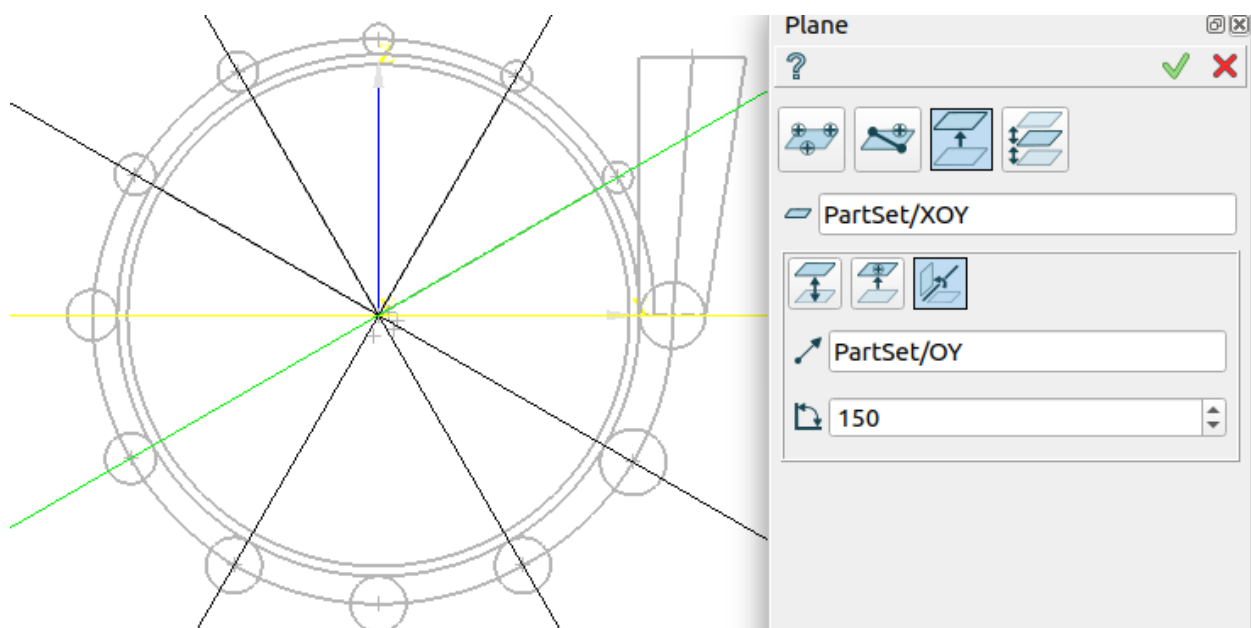


Рисунок 54 – Создание плоскостей

На плоскости XOY создайте эскиз, спроектируйте точку окружности эскиза 2 (инструмент projection) и точки пересечения (инструмент intersection), постройте вспомогательную окружность. Создайте прямоугольник, используйте зависимость средней точки для соединения одной стороны прямоугольника с центром окружности, также используйте зависимость

касательной. Сделайте линию прямоугольника вспомогательной, проведите дугу (Рисунок 55).

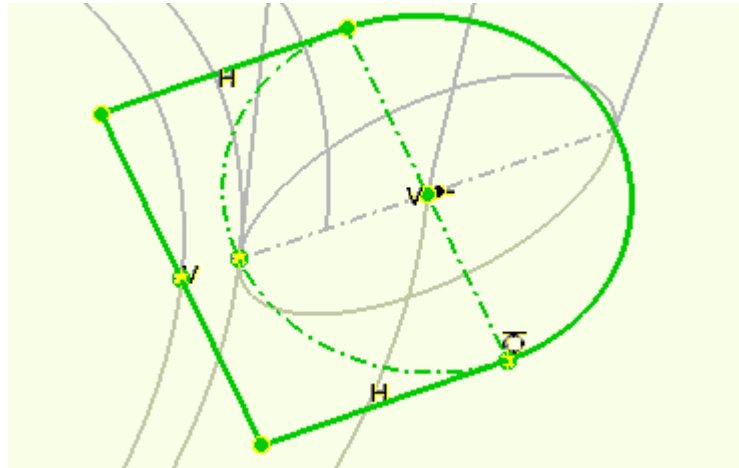


Рисунок 55 – Построение эскиза

Повторите действия для всех плоскостей. Для каждого эскиза в меню build выберите создание грани face (Рисунок 56).

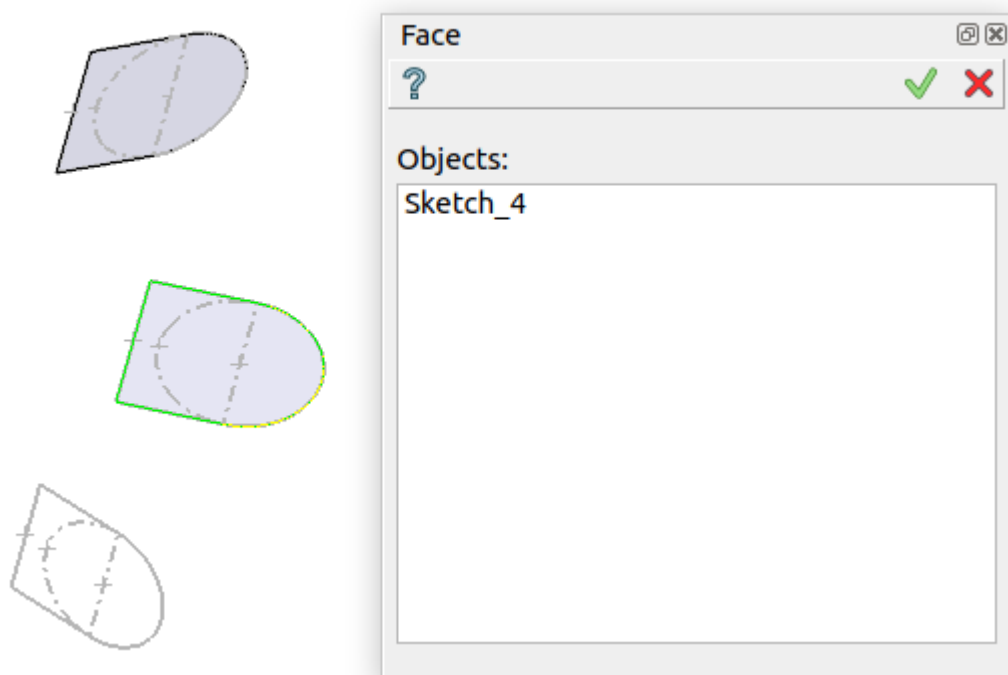


Рисунок 56 – Создание грани

Повторите действие для всех созданных эскизов. Должно получиться 12 граней (Рисунок 57).

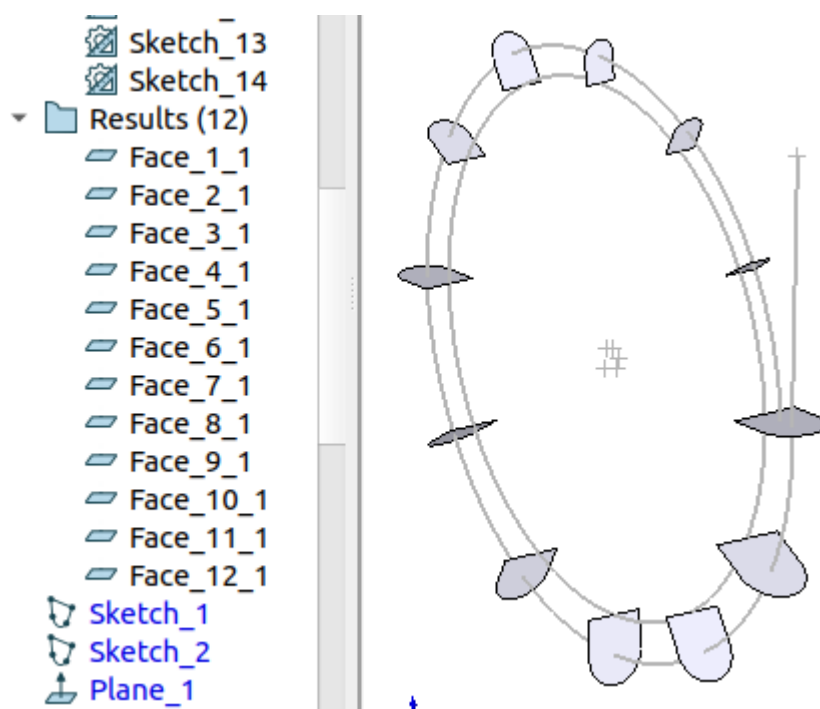


Рисунок 57 – Созданные грани

В меню Features выберите инструмент Pipe. В поле Base object укажите первые три грани, в качестве пути дугу из эскиза 2. Убедитесь, что выбран 3 вариант построения (Рисунок 58).

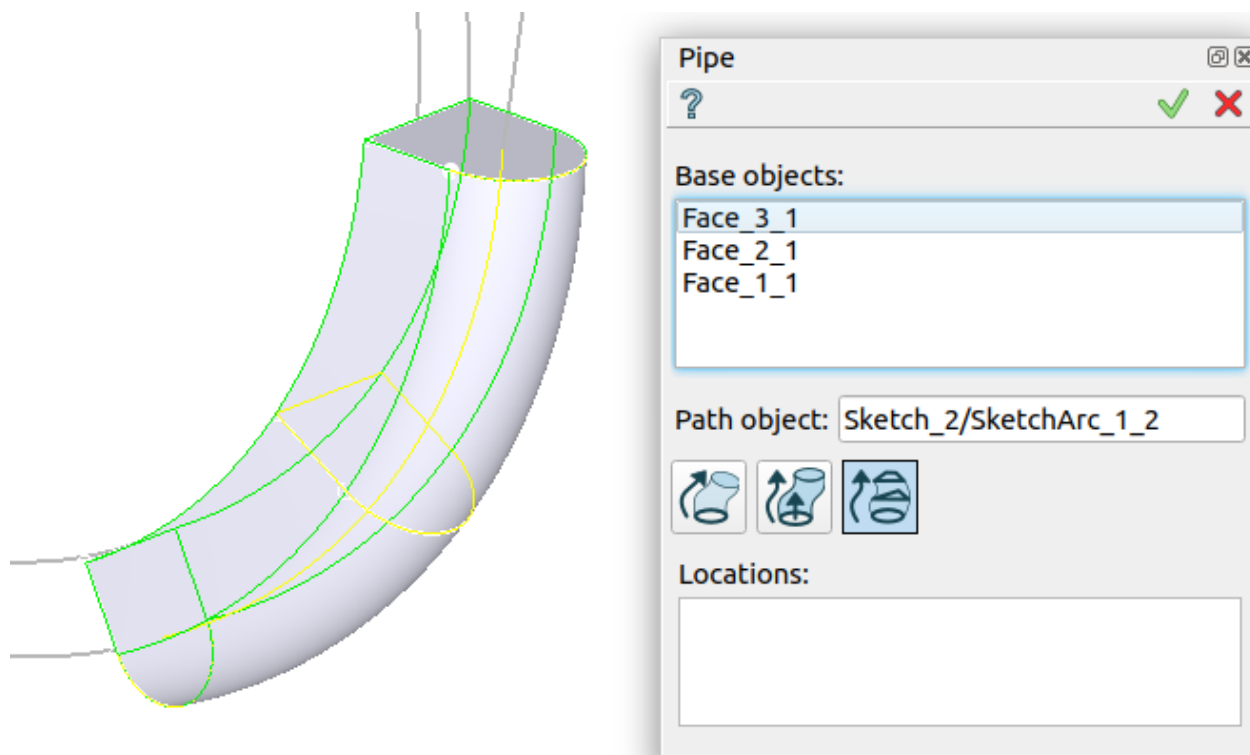


Рисунок 58 – Операция Pipe

Так как при построении Face 3 была использована, она больше не доступна, создайте грань снова (Рисунок 59).

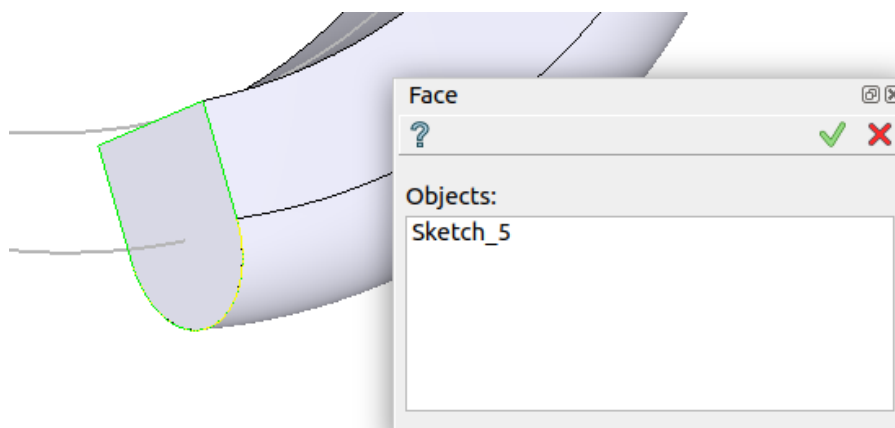


Рисунок 59 – Создание грани

Повторите операцию Pipe для следующих граней (Рисунок 60).

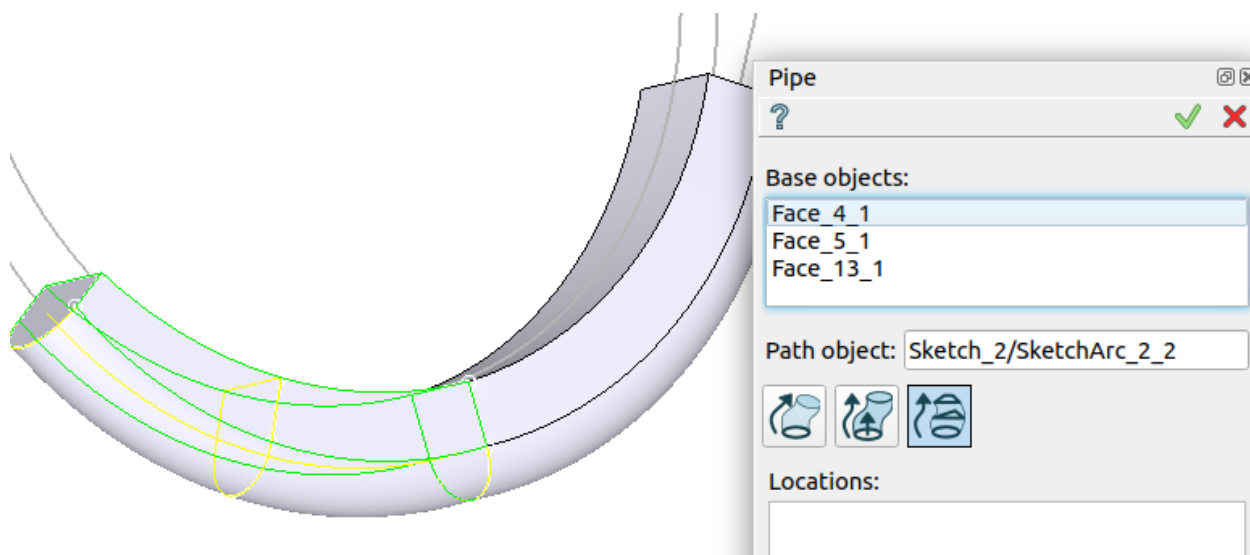


Рисунок 60 – Операция Pipe

Снова создайте грань, необходимую для следующей операции Pipe (Рисунок 61).

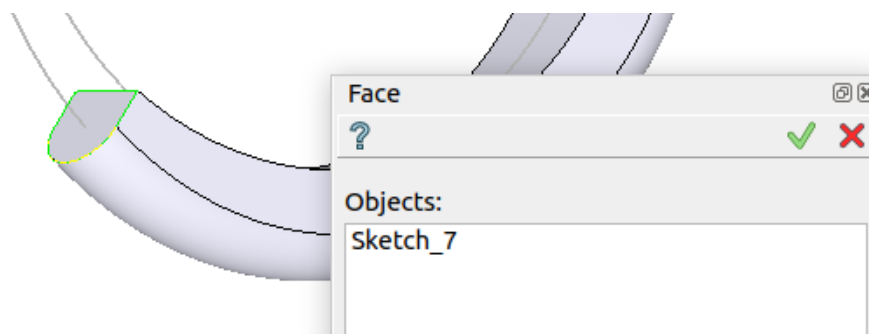


Рисунок 61 – Создание грани

Выполните Pipe для граней 7, 6 и 14 (Рисунок 62).

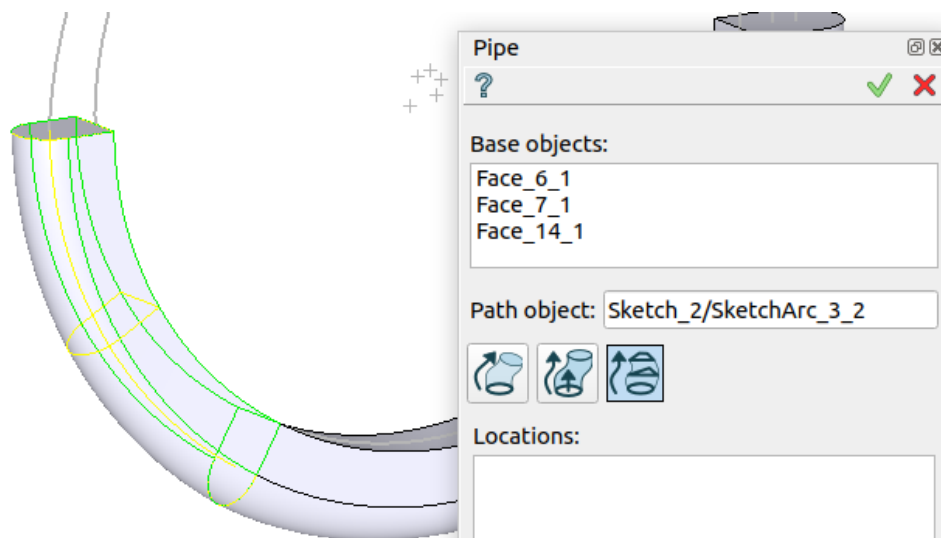


Рисунок 62 – Операция Pipe

Снова создайте грань из эскиза 9 (Рисунок 63).

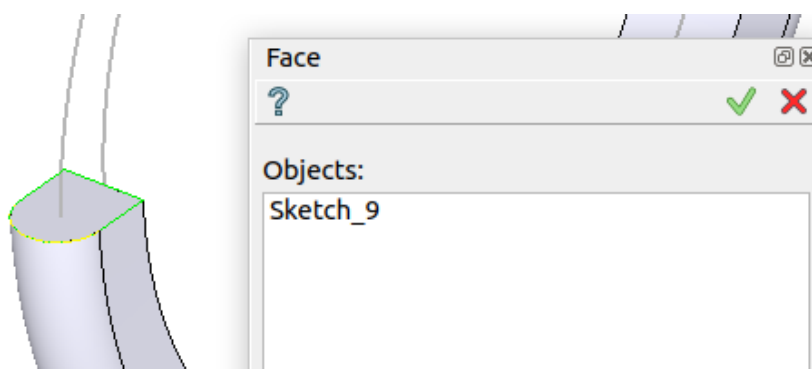


Рисунок 63 – Создание грани

Выполните операцию Pipe для граней 8, 9 и 15 (Рисунок 64).

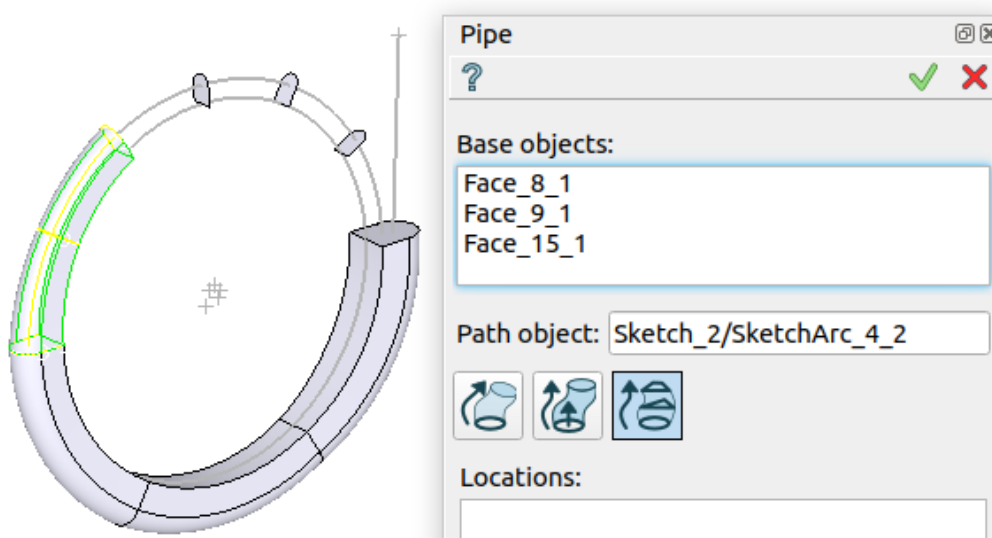


Рисунок 64 – Операция Pipe

Создайте грань из эскиза 11 (Рисунок 65).

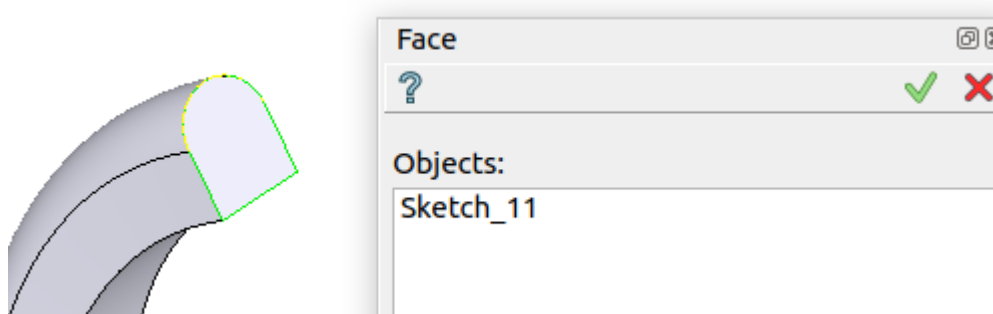


Рисунок 65 – Создание грани

Выполните операцию Pipe для 10, 11 и 16 граней (Рисунок 66).

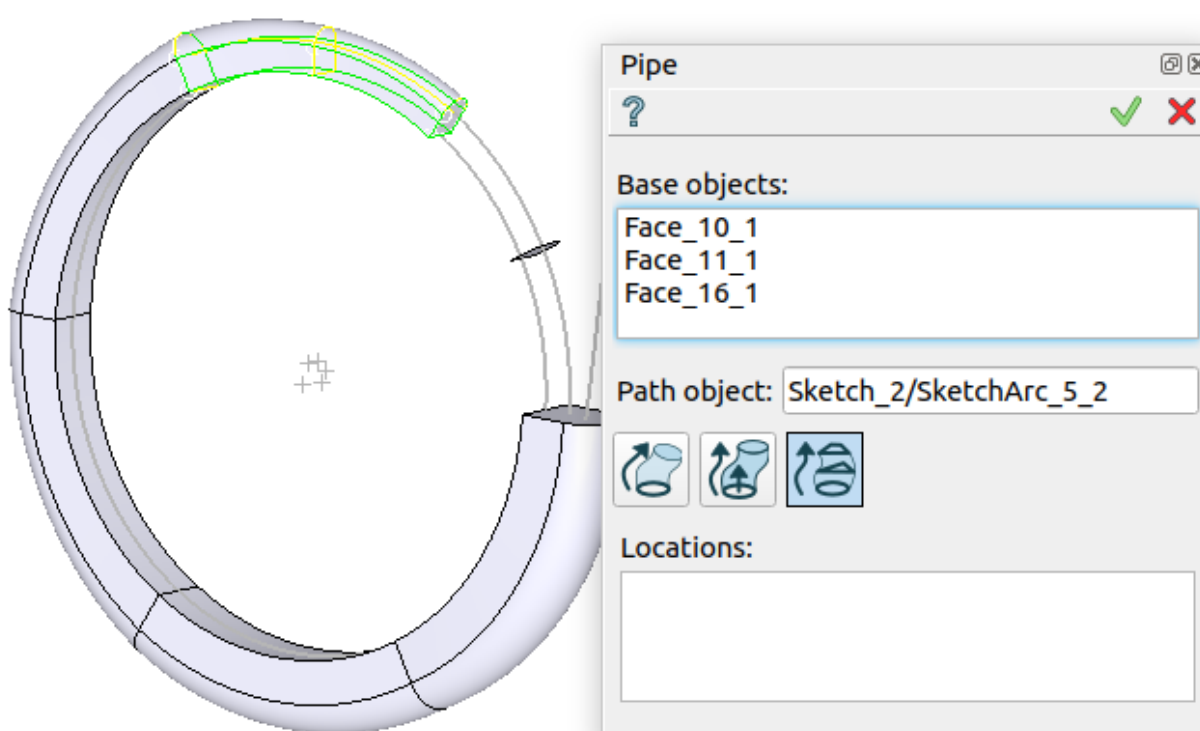


Рисунок 66 – Операция Pipe

Создайте грань из геометрии эскиза 13 (Рисунок 67).

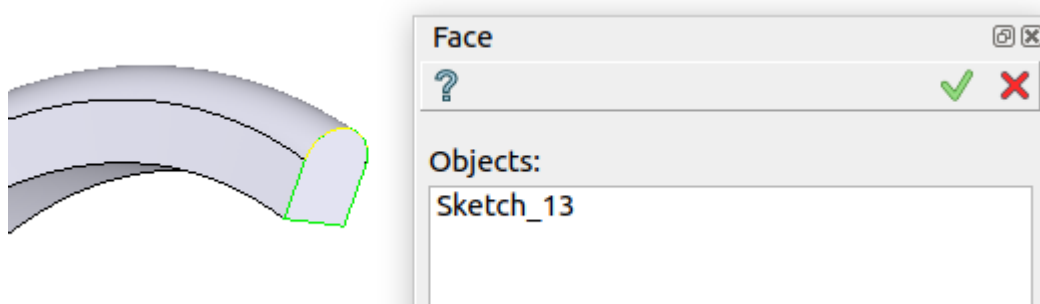


Рисунок 67 – Создание грани

Создайте еще один Pipe из граней 12 и 17 (Рисунок 68).

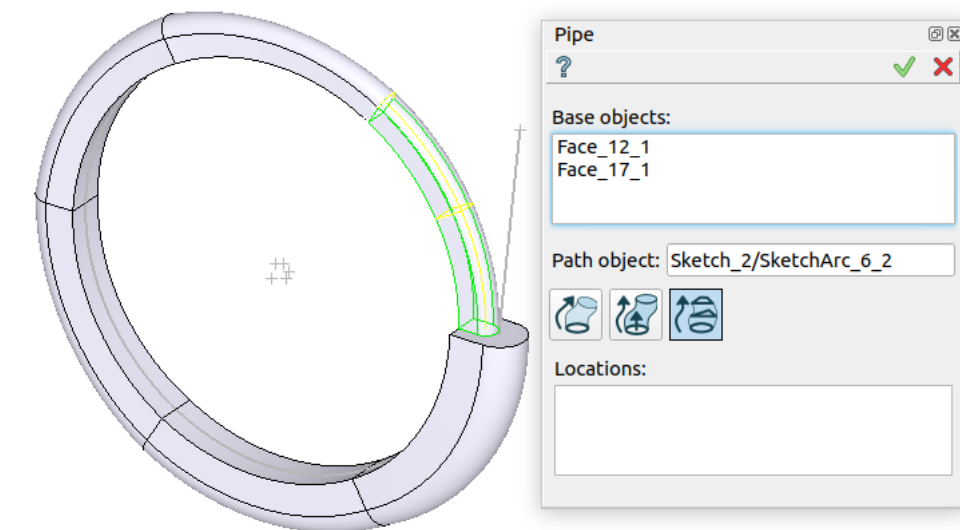


Рисунок 68 – Операция Pipe

Для построения диффузора создайте плоскость, отстоящую от плоскости XOY на величину, равную длине диффузора (Рисунок 69).

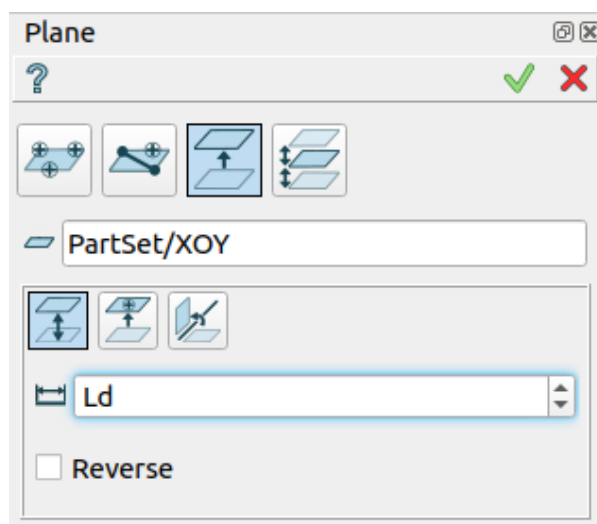


Рисунок 69 – Создание плоскости

На плоскости постройте окружность радиусом $dn/2$ (Рисунок 70).

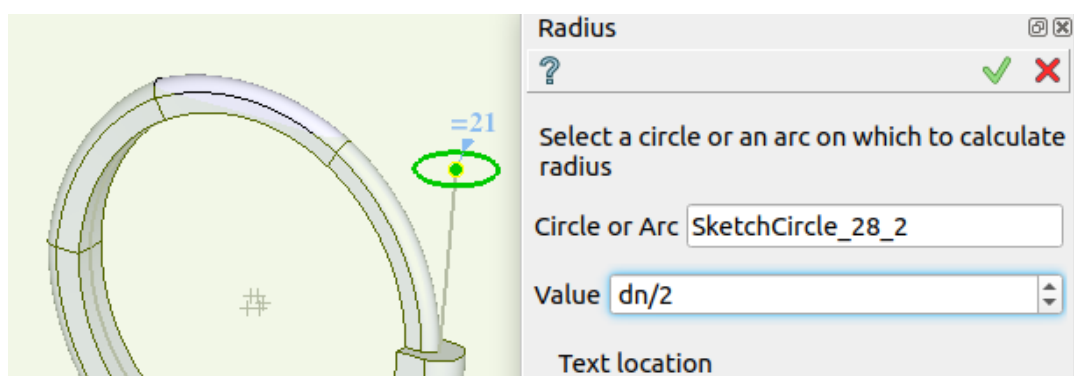


Рисунок 70 – Окружность выхода из диффузора

Из построенной окружности создайте грань (Рисунок 71).

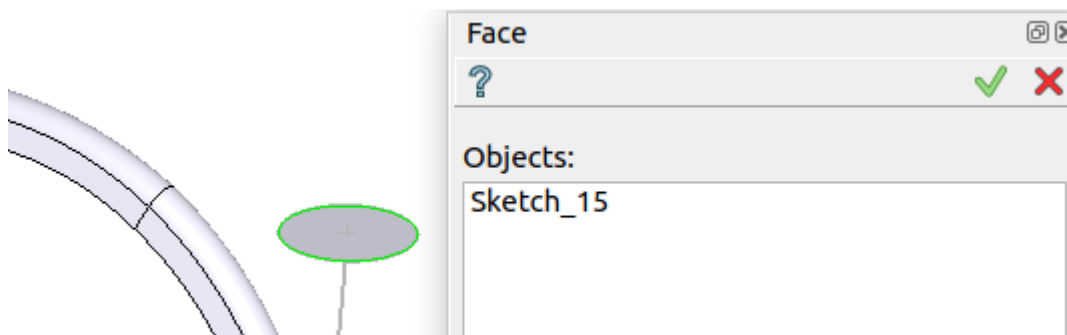


Рисунок 71 – Создание грани

Из геометрии эскиза 3 создайте грань (Рисунок 72).

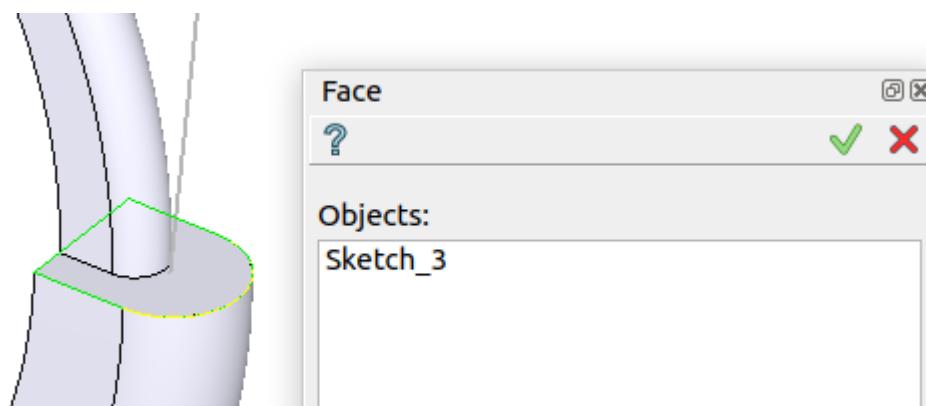


Рисунок 72 – Создание грани

С помощью операции Pipe соедините созданные грани, в качестве пути используйте линию из 2 эскиза (Рисунок 73).

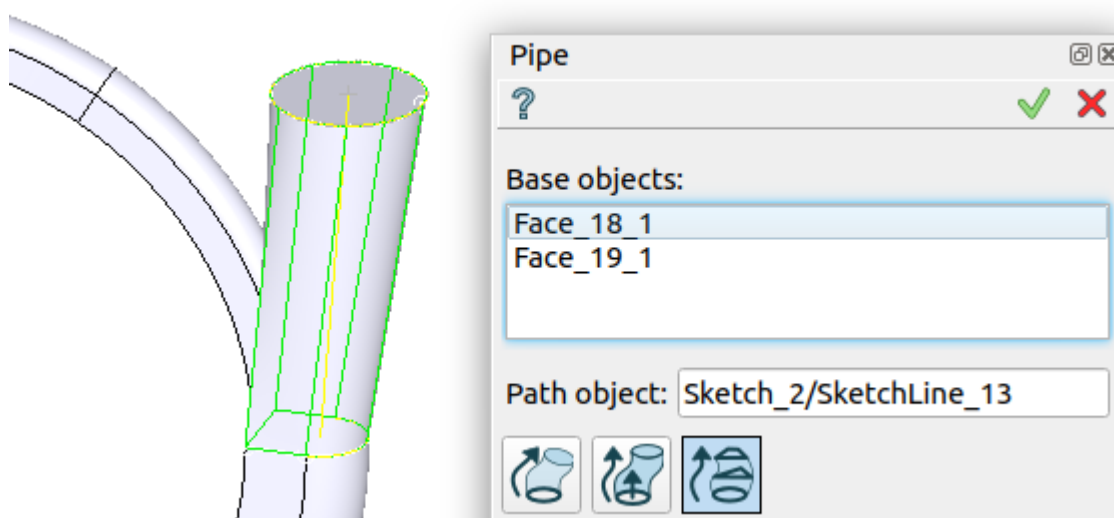


Рисунок 73 – Операция Pipe

Создайте грань из эскиза 15 (Рисунок 74).

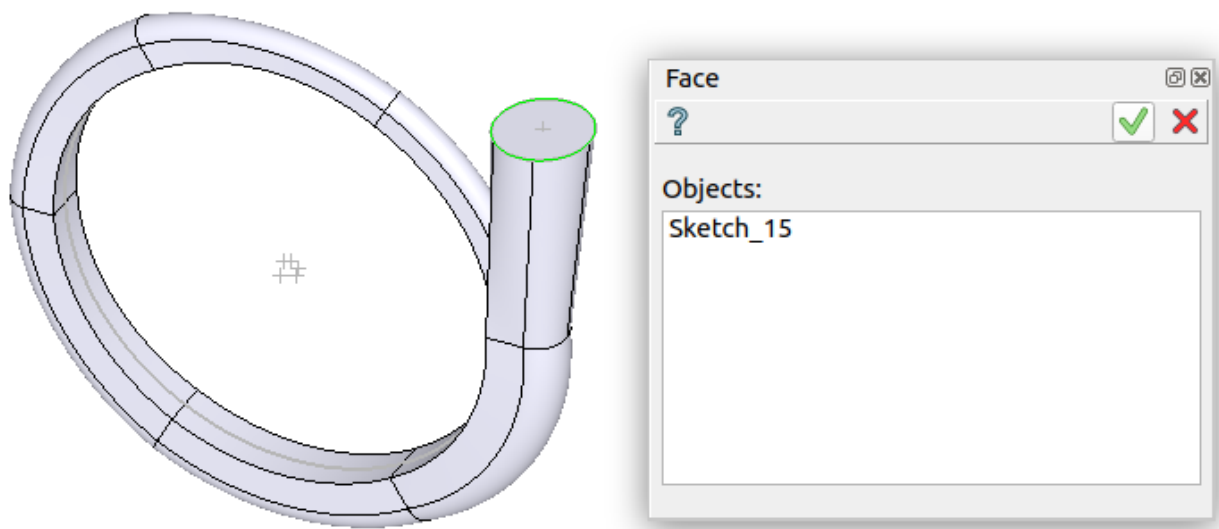


Рисунок 74 – Создание грани

Используйте созданную грань для операции Extrusion, в качестве расстояния укажите наружный диаметр рабочего колеса D_2 (Рисунок 75).

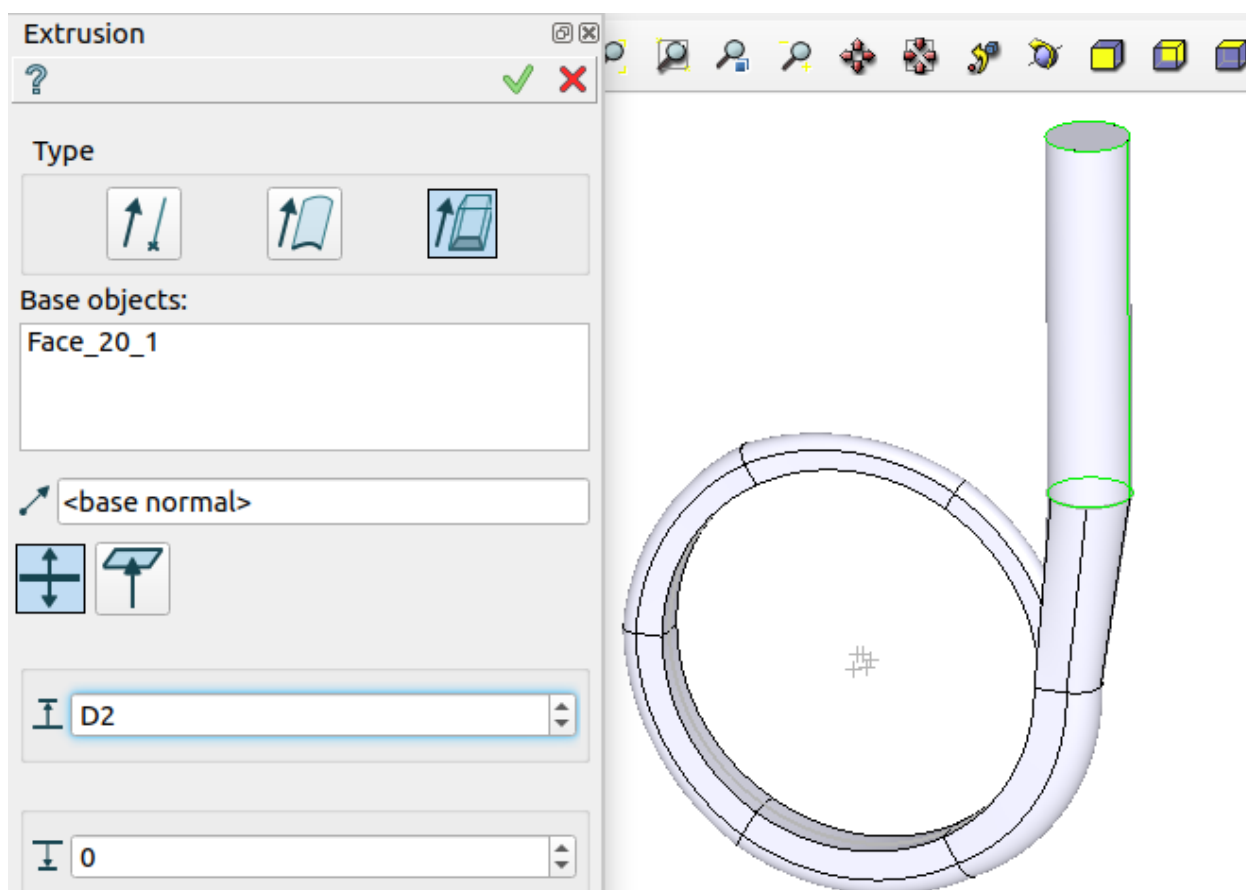


Рисунок 75 – Операция Extrusion

Объедините все из блока Result в Object Browser с помощью операции Fuse (Рисунок 76).

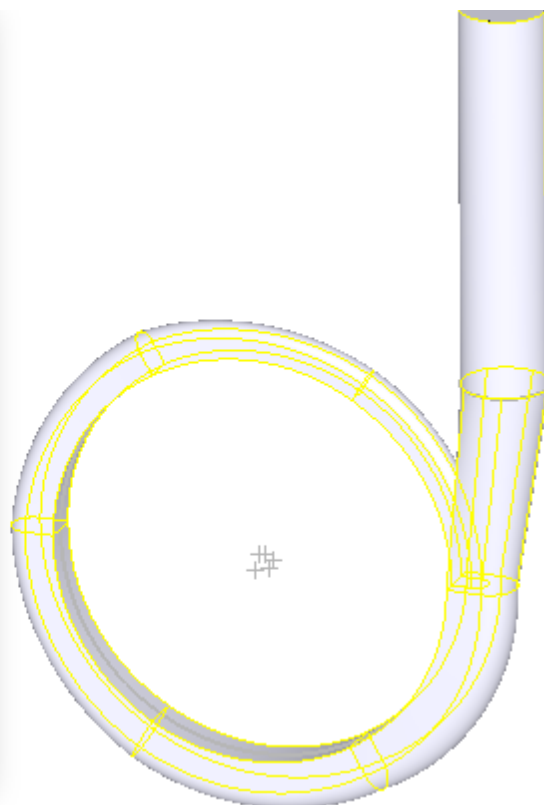
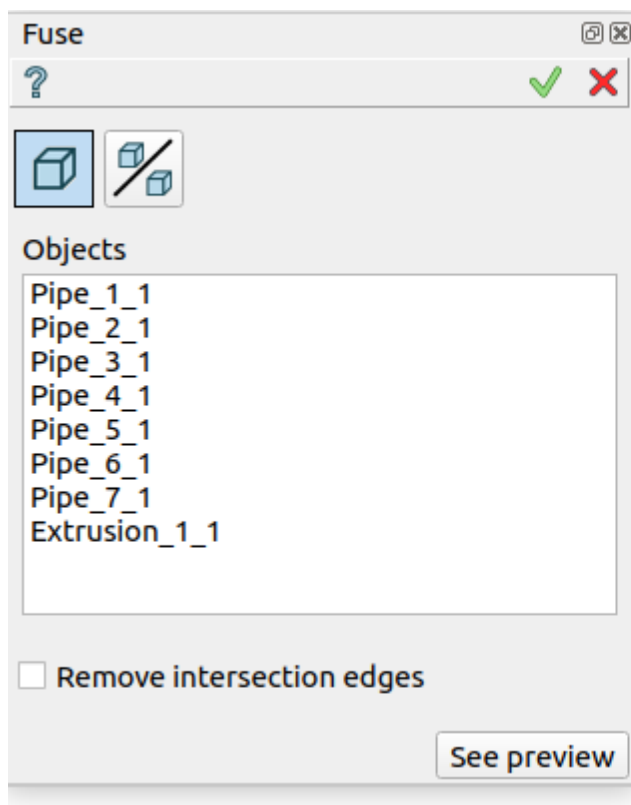


Рисунок 76 – Операция Fuse

Переименуйте результат операции в domain. Выполните File => DumpStudy. Файл назовите part2.py.

8 ПОДГОТОВКА ГЕОМЕТРИИ

8.1 Изменение расположения рабочего колеса

Рабочее колесо должно располагаться по центру спирального отвода. Последний построен симметрично относительно плоскости XOZ (Рисунок 77). Колесо необходимо переместить относительно оси OY.

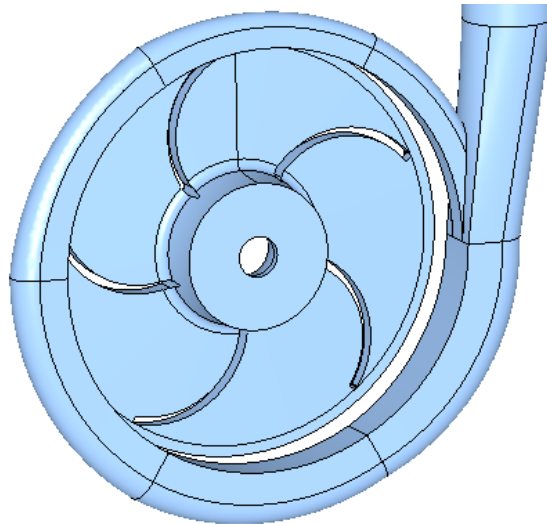


Рисунок 77 – Расположение рабочего колеса и спирального отвода

В меню Part выберите Translation. В качестве объекта укажите Part1 1, ось OY. Расстояние, на которое необходимо переместить рабочее колесо, рассчитывается следующим образом: из всей ширины L вычитаем половину ширины канала на выходе b_2 . Так как ось Y смотрит в противоположную сторону, то необходимо сменить знаки. Получим: $-L + b_2/2$ (Рисунок 78).

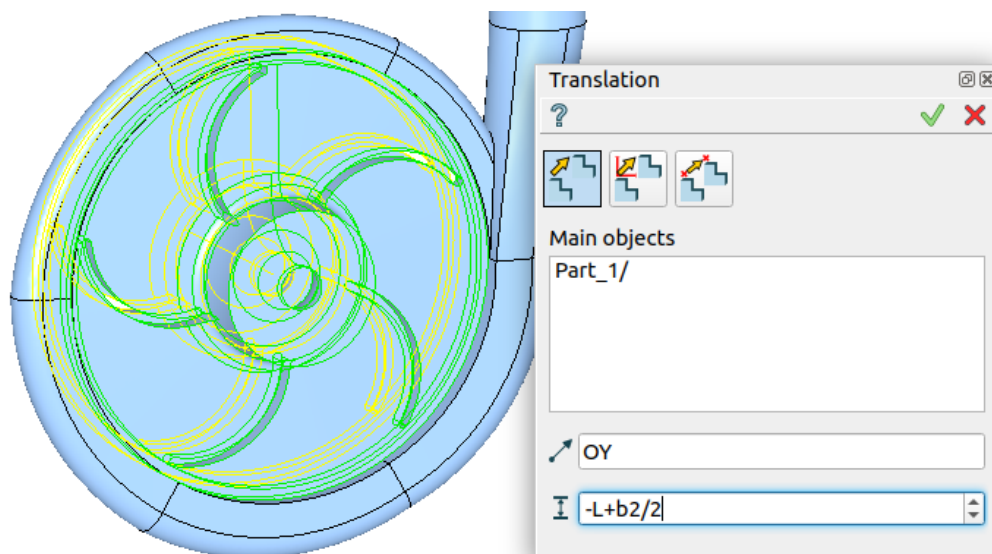


Рисунок 78 – Операция Translation

Убедитесь, что рабочее колесо расположено по центру (Рисунок 79).

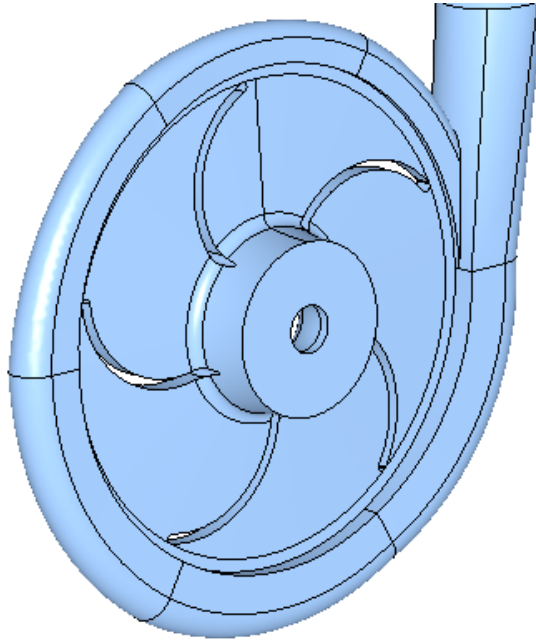


Рисунок 79 – Расположение рабочего колеса и спирального отвода

8.2 Создание входной геометрии

Необходимо создать геометрию, по которой вода будет поступать в рабочее колесо. В основном меню выберите Part => New Part. На плоскости XOZ создайте окружности радиусами R_0 и r_{vt} (Рисунок 80).

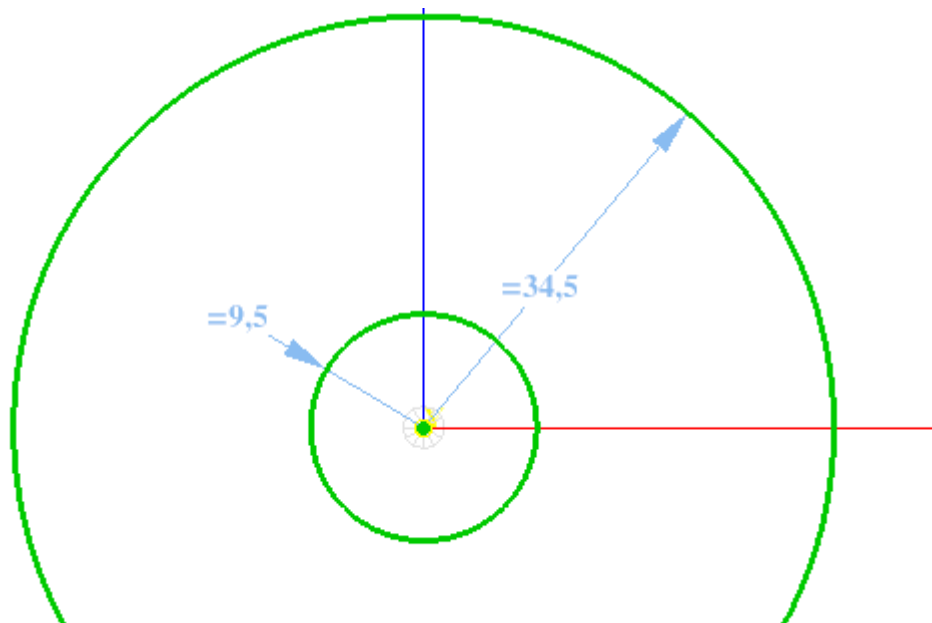


Рисунок 80 – Создание окружностей

Выполните операцию Extrusion. Расстояние не рассчитывается, его можно принять равным радиусу R_2 (Рисунок 81).

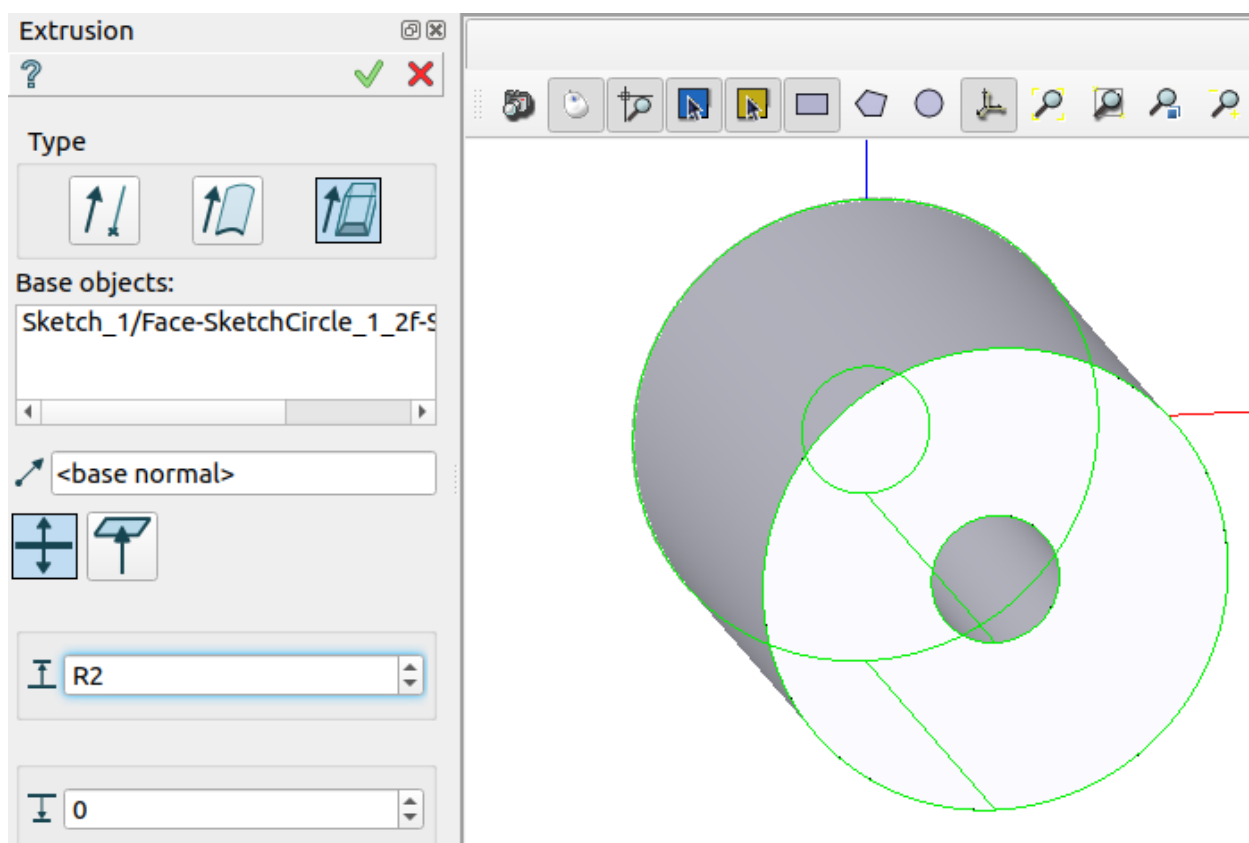


Рисунок 81 – Операция Extrusion

Выйдите из редактирования Part 3. Построенную область необходимо переместить относительно оси OY на то же расстояние (Рисунок 82).

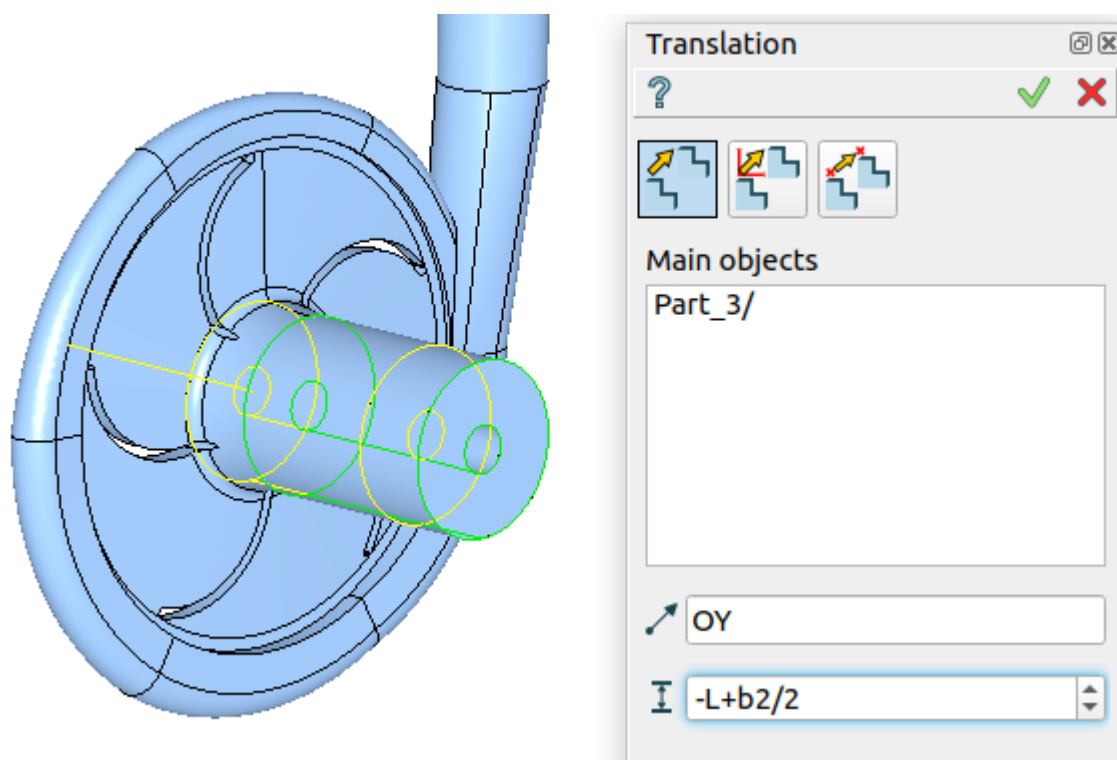


Рисунок 82 – Операция Translation

Полученный результат построенной геометрии показан на рисунке 83.

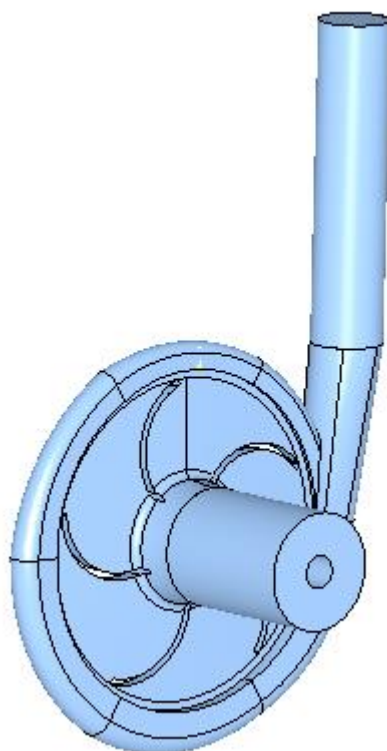


Рисунок 83 – Построенная геометрия

Сохраните изменения, назовите файл `all_parts.hdf`. Выполните команду `Dump Study` для формирования скрипта, назовите его `all_parts.py`.

9 АВТОМАТИЗАЦИЯ ПОСТРОЕНИЯ НА PYTHON

Перенесите расчет и автоматизированное построение Part1 в файл all_parts.py. Загрузите скрипт в Salome. Убедитесь, что ошибок нет.

Для обеспечения параметрического изменения геометрии спирального отвода необходимо внести изменения в код построения Part2.

12 окружностей должны быть построены за пределами окружности радиуса $R3+thick$, иначе зависимость касательной установится с внутренней стороны. Предварительно создайте переменные $R3$, $thick$ и $r_$. Первую переведите в мм, второй присвойте значение 5 из исходных данных, последняя будет равна радиусу наибольшей окружности. В качестве координат для каждой точки можно принять следующие значения.

Листинг 47 – Изменение координат окружностей

```
839 ### Create SketchCircle
840 SketchCircle_9 = Sketch_4.addCircle((2*R3+thick)*2/3, (2*R3+thick)/3, r_)
841
842 ### Create SketchCircle
843 SketchCircle_10 = Sketch_4.addCircle((2*R3+thick)/3, (2*R3+thick)*2/3, r_)
844
845 ### Create SketchCircle
846 SketchCircle_11 = Sketch_4.addCircle(0, 2*R3+thick, r_)
847
848 ### Create SketchCircle
849 SketchCircle_12 = Sketch_4.addCircle(-(2*R3+thick)/3, (2*R3+thick)*2/3, r_)
850
851 ### Create SketchCircle
852 SketchCircle_13 = Sketch_4.addCircle(-(2*R3+thick)*2/3, (2*R3+thick)/3, r_)
853
854 ### Create SketchCircle
855 SketchCircle_14 = Sketch_4.addCircle(-(2*R3+thick), 0, r_)
856
857 ### Create SketchCircle
858 SketchCircle_15 = Sketch_4.addCircle(-(2*R3+thick)*2/3, -(2*R3+thick)/3, r_)
859
860 ### Create SketchCircle
861 SketchCircle_16 = Sketch_4.addCircle(-(2*R3+thick)/3, -(2*R3+thick)*2/3, r_)
862
863 ### Create SketchCircle
864 SketchCircle_17 = Sketch_4.addCircle(0, -(2*R3+thick), r_)
865
866 ### Create SketchCircle
867 SketchCircle_18 = Sketch_4.addCircle((2*R3+thick)/3, -(2*R3+thick)*2/3, r_)
868
869 ### Create SketchCircle
870 SketchCircle_19 = Sketch_4.addCircle((2*R3+thick)*2/3, -(2*R3+thick)/3, r_)
871
872 ### Create SketchCircle
873 SketchCircle_20 = Sketch_4.addCircle(2*R3+thick, 0, r_)
```

Переменные вынесите до начала создания Part 1.

Листинг 48 – Объявление переменных

```
335 R3 = R3*1000
336 thick = 5
337 r_ = r[11]*1000
338 Ld = Ld*1000
339 dn = dn*1000
```

Измените координаты построения линий диффузора. Добавляйте координатам некоторое смещение, в противном случае зависимость Coincident может не сработать.

Листинг 49 – Построение профиля диффузора

```
918 ### Create SketchLine
919 SketchLine_39 = Sketch_4.addLine(R3+thick+1, 1, R3+thick+1, Ld+1)
920 Sketch_4.setCoincident(SketchPoint_19.coordinates(),
    SketchLine_39.startPoint())
921 Sketch_4.setVertical(SketchLine_39.result())
922 Sketch_4.setLength(SketchLine_39.result(), "Ld")
923
924 ### Create SketchLine
925 SketchLine_40 = Sketch_4.addLine(R3+thick+r_/2, 1, R3+thick+r_*1.5, 1)
926 SketchLine_40.setAuxiliary(True)
927 Sketch_4.setCoincident(SketchPoint_19.coordinates(),
    SketchLine_40.startPoint())
928 Sketch_4.setCoincident(SketchLine_40.endPoint(),
    SketchCircle_20.results()[1])
929 Sketch_4.setHorizontal(SketchLine_40.result())
930
931 ### Create SketchLine
932 SketchLine_41 = Sketch_4.addLine(R3+thick+r_*2+2, 2,
    R3+thick+r_*2+dn+2, Ld+2)
933 Sketch_4.setCoincident(SketchLine_40.endPoint(),
    SketchLine_41.startPoint())
934
935 ### Create SketchLine
936 SketchLine_42 = Sketch_4.addLine(R3+thick+r_*2+dn+3, Ld+3, R3+thick+3,
    Ld+3)
937 Sketch_4.setCoincident(SketchLine_41.endPoint(),
    SketchLine_42.startPoint())
938 Sketch_4.setCoincident(SketchLine_39.endPoint(),
    SketchLine_42.endPoint())
939 Sketch_4.setHorizontal(SketchLine_42.result())
940 Sketch_4.setLength(SketchLine_42.result(), "dn")
```

При построении профилей на 12 плоскостях необходимо обеспечить расположение прямоугольников внутри окружности радиуса $R3 + thick$. Для корректной установки зависимости касательной важно построить сторону прямоугольника за пределами спроецированной окружности. Обратите внимание на порядок координат каждого прямоугольника в вашем коде. Не

изменяйте его, иначе в зависимостях придется менять нумерацию линий, их начало и окончание. Операция Pipe может завершиться с ошибкой. В качестве базовой ширины можно принять длину радиуса $R3$, деленную на 10. Возможный вариант построения прямоугольника показан в листинге 50.

Листинг 50 – Построение прямоугольника

```
1106 ### Create SketchLine
1107 SketchLine_46 = Sketch_6.addLine( $R3/10$ ,  $R3/10$ ,  $-R3/10$ ,  $R3/10$ )
1108
1109 ### Create SketchLine
1110 SketchLine_47 = Sketch_6.addLine( $-R3/10$ ,  $R3/10$ ,  $-R3/10$ ,  $-R3/10$ )
1111
1112 ### Create SketchLine
1113 SketchLine_48 = Sketch_6.addLine( $-R3/10$ ,  $-R3/10$ ,  $R3/10$ ,  $-R3/10$ )
1114
1115 ### Create SketchLine
1116 SketchLine_49 = Sketch_6.addLine( $R3/10$ ,  $-R3/10$ ,  $R3/10$ ,  $R3/10$ )
```

По аналогии постройте прямоугольники на остальных плоскостях. Если при загрузке в Salome будет появляться ошибка в построении эскиза, проверьте зависимости соединения дуг с линиями. Возможно, неверный порядок начала и конца линии.

Для корректного построения дуги ее координаты нужно расположить за пределами построенной окружности, иначе может получиться как на рисунке 84.



Рисунок 84 – Неверное расположение дуги окружности

Можно использовать радиус $R3$ для установки точек. Если дуга строится в другом направлении, попробуйте в коде изменить флаг (True, False).

Листинг 51 – Построение дуги окружности

```
1935 ### Create SketchArc
1936 SketchArc_45 = Sketch_17.addArc(R3*2, 0, R3*2, 5.999999999999979, R3*2,
    -5.999999999999989, True)
1937 SketchArc_45.setName("SketchArc_18")
1938 SketchArc_45.result().setName("SketchArc_18")
1939 SketchArc_45.results()[1].setName("SketchArc_18_2")
1940 Sketch_17.setCoincident(SketchAPI_Point(SketchPoint_87).coordinates(),
    SketchArc_45.center())
1941 Sketch_17.setCoincident(SketchLine_90.startPoint(), SketchArc_45.startPoint())
1942 Sketch_17.setCoincident(SketchLine_93.result(), SketchArc_45.endPoint())
1943 model.do()
```

Если при построении Pipe получаете ошибку, как на рисунке 85, попробуйте изменить порядок объявления граней. Начните со средней.

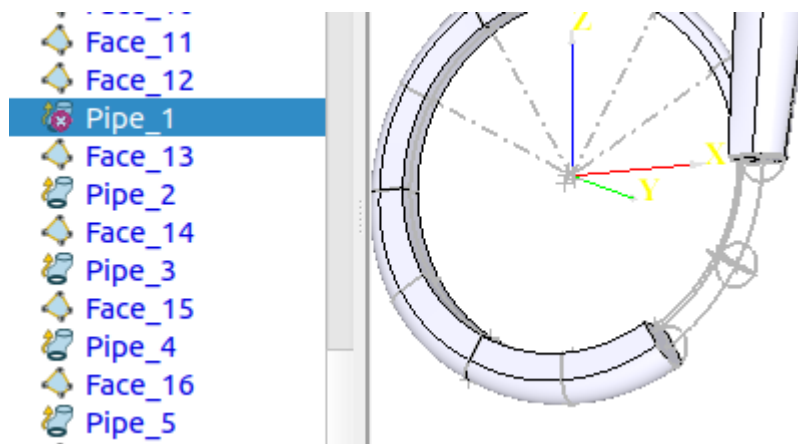


Рисунок 85 – Ошибка при построении Pipe

Листинг 52 – Построение первого Pipe

```
1981 ### Create Pipe
1982 Pipe_1_objects_1 = [model.selection("FACE", "Face_2_1"),
1983                     model.selection("FACE", "Face_3_1"),
1984                     model.selection("FACE", "Face_1_1")]
1985 Pipe_1 = model.addPipe(Part_2_doc, Pipe_1_objects_1,
    model.selection("EDGE", "Sketch_2/SketchArc_1_2"), [])
```

Проверьте построение в Salome для всех трех вариантов исходных данных. На этом параметризация заканчивается, далее в работе будет использоваться только один набор входных параметров.

10 ВНЕСЕНИЕ ДОПОЛНЕНИЙ В ПОСТРОЕНИЕ СПИРАЛЬНОГО ОТВОДА

При построении использовались следующие значения: $Q = 0.008 \text{ м}^3/\text{с}$, $H = 9.7 \text{ м}$, $n = 1000 \text{ об/мин}$.

На данный момент между спиральным отводом и рабочим колесом с одной стороны присутствует пустое пространство (Рисунок 86), а с другой стороны геометрия накладывается друг на друга.

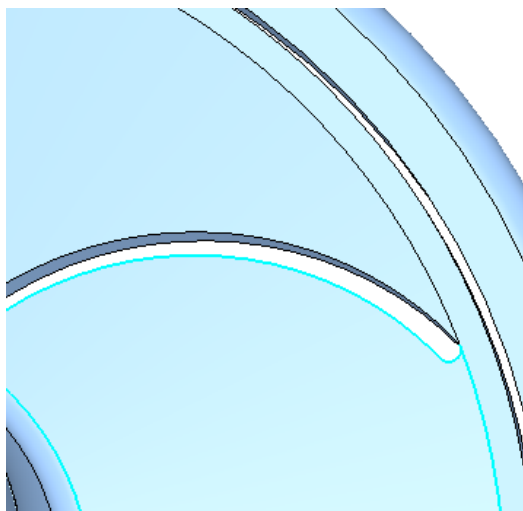


Рисунок 86 – Пространство между колесом и отводом

Зайдите в редактирование Part2. На плоскости XOZ создайте эскиз. Спроектируйте дуги эскиза 2. Соедините концы линией (Рисунок 87).

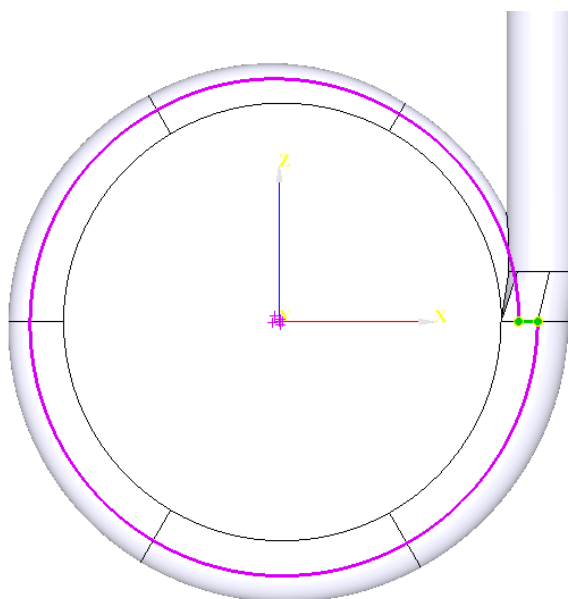


Рисунок 87 – Эскиз 16

Операцией Extrusion выдавите полученную область на расстояние $b2/2$ в обе стороны (Рисунок 88).

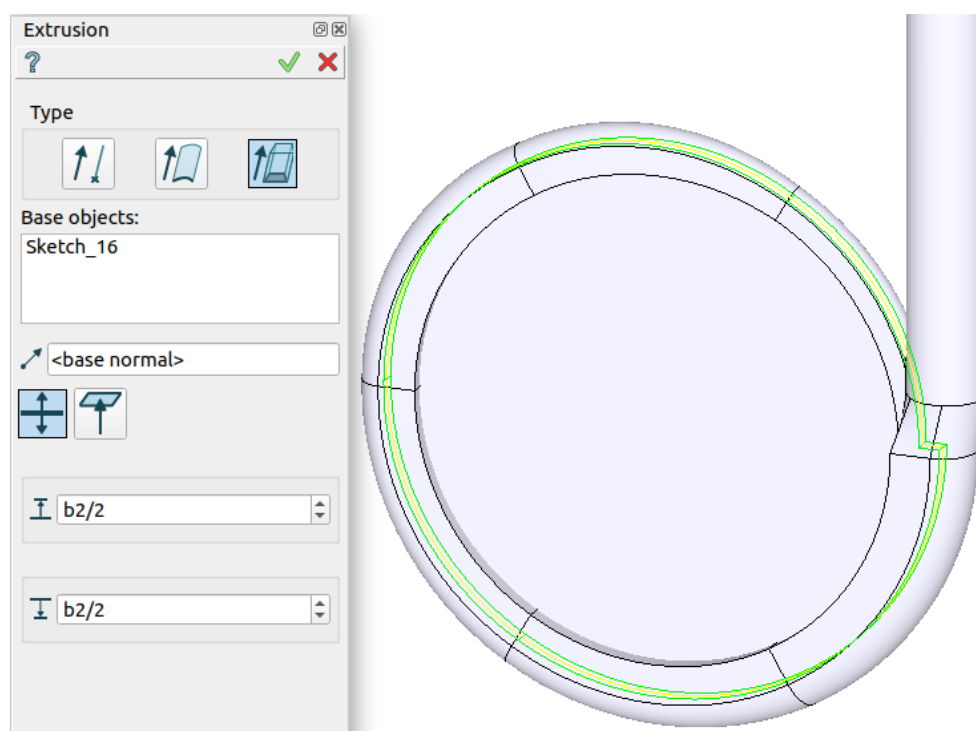


Рисунок 88 – Операция Extrusion

Создайте еще один эскиз на плоскости XOZ, постройте на нем окружность радиусом R3 (Рисунок 89).

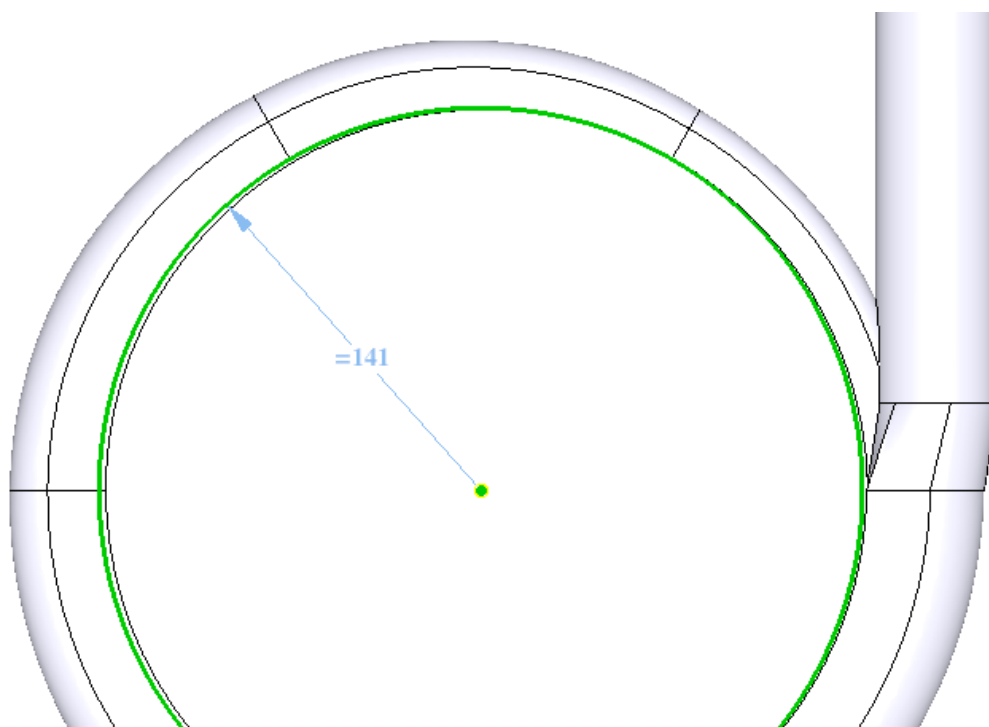


Рисунок 89 – Построение окружности

Выдавите вычитанием полученную область на все расстояние в обе стороны (Рисунок 90).

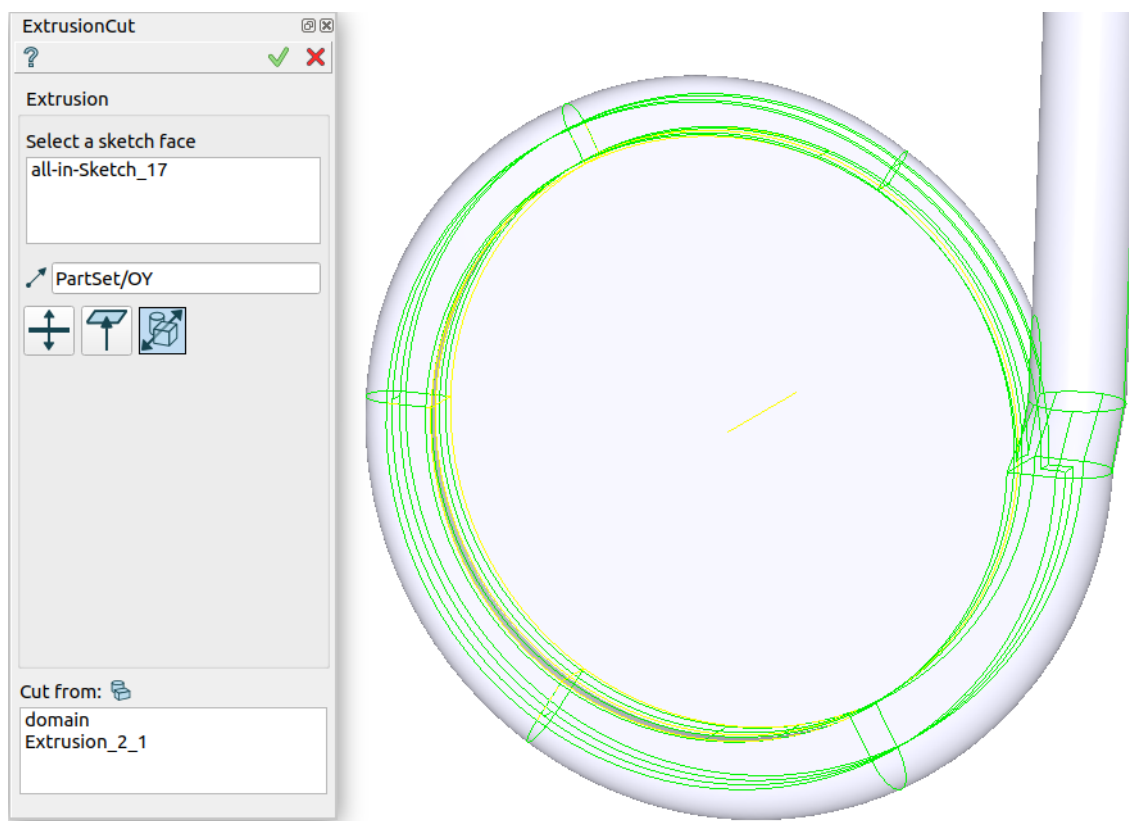


Рисунок 90 – Операция ExtrusionCut

Создайте новый эскиз, постройте на нем две окружности радиусами R3 и R2 + thick (Рисунок 91).

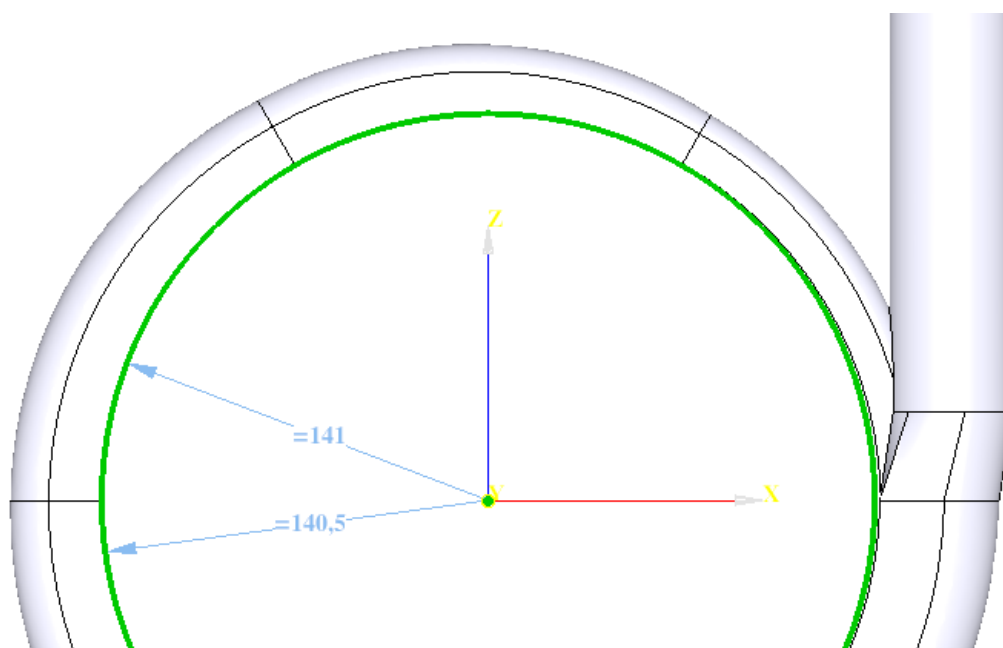


Рисунок 91 – Построение окружностей

Выдавите полученную область в обе стороны на расстояние $b2/2$ (Рисунок 92).

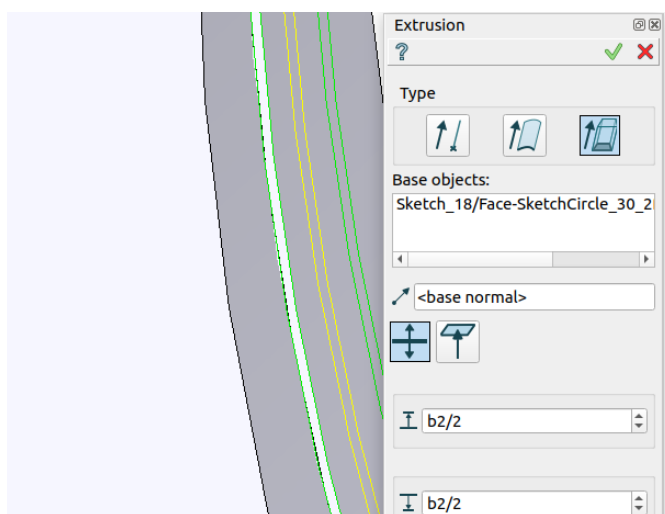


Рисунок 92 – Операция Extrusion

Выполните операцию Fuse (Рисунок 93).

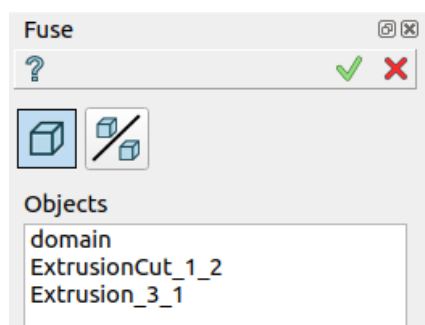


Рисунок 93 – Операция Fuse

Увеличьте высоту выходной трубы, чтобы не возникло проблем при расчете. Для данных входных параметров можете использовать значение 500 (Рисунок 94).

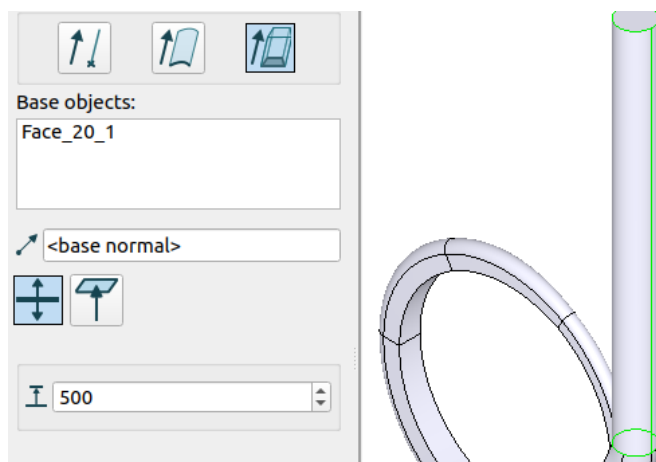


Рисунок 94 – Операция Extrusion

Убедитесь, что поверхность рабочего колеса касается поверхности спирального отвода. Выполните команду Dump Study во временный файл. Добавьте в скрипт последние выполненные действия.

11 СОЗДАНИЕ ГРАНИЦ

Перейдите в Part3, в разделе Features выберите Group. Создайте первую группу по поверхности для грани входа, назовите inlet (Рисунок 95).

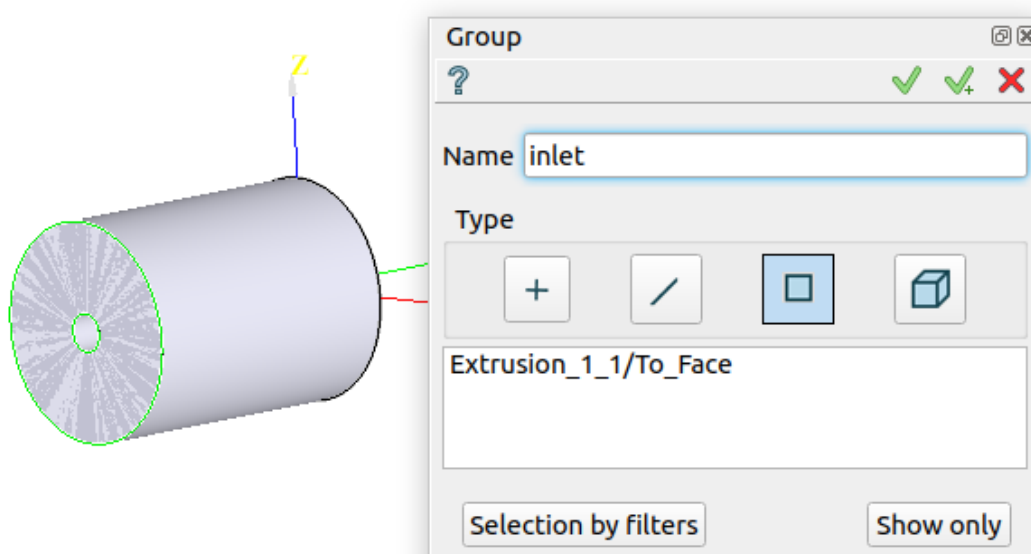


Рисунок 95 – Создание группы inlet

Создайте вторую группу, выберите грань, которая будет касаться грани Part1. Такие грани в OpenFOAM обозначают AMI, назовите ее AMI_inlet (Рисунок 96).

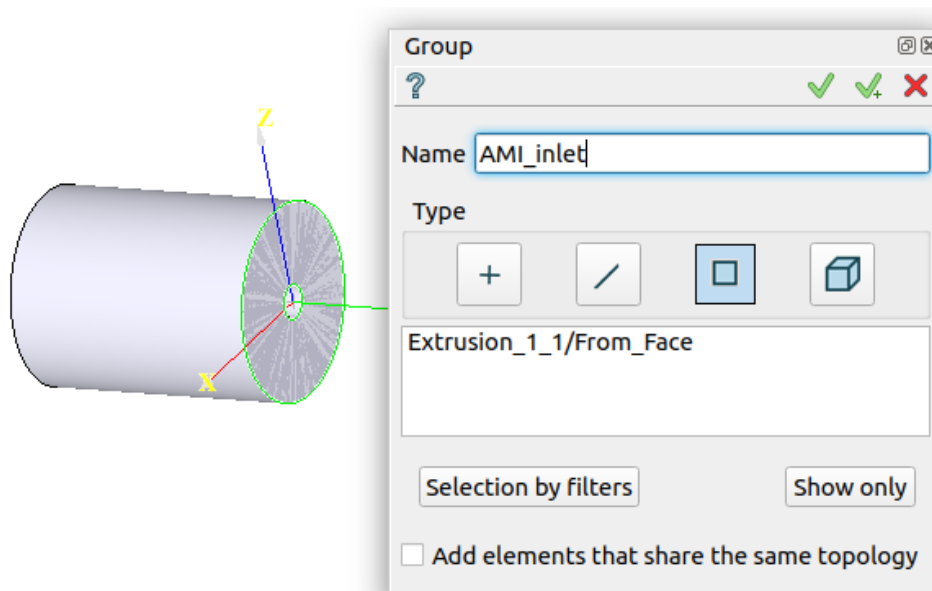


Рисунок 96 – Создание группы AMI_inlet

Выберите оставшиеся две грани для создания третьей группы, назовите wall_inlet (Рисунок 97).

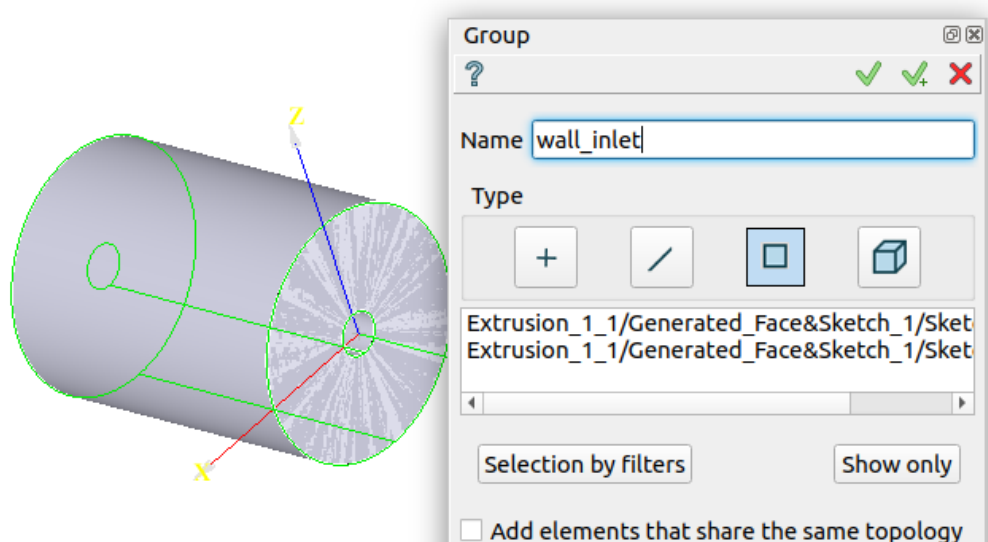


Рисунок 97 – Создание группы wall_inlet

Перейдите в Part1, создайте группу по поверхности, граничащую с Part3 (Рисунок 98).

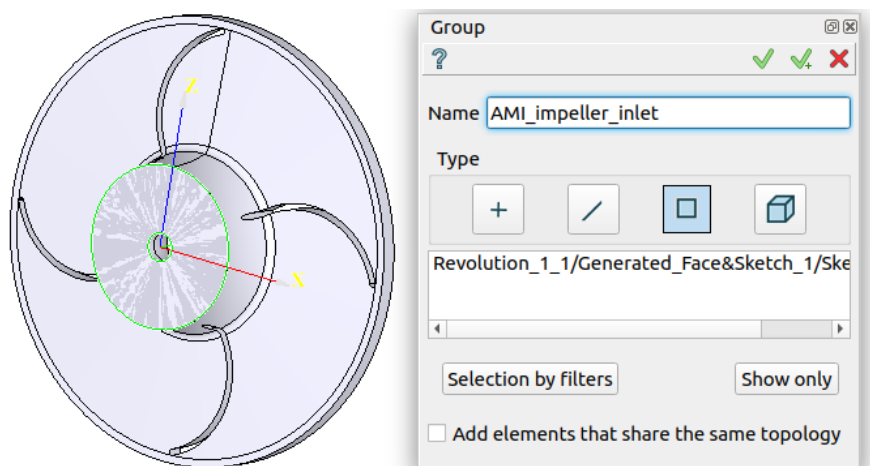


Рисунок 98 – Создание группы AMI_impeller_inlet

Затем создайте группу по поверхности, граничащую с Part2 (Рисунок 99).

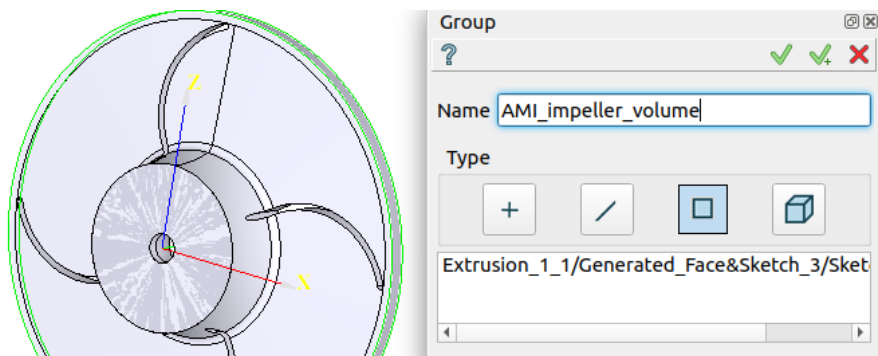


Рисунок 99 – Создание группы AMI_impeller_volume

Создайте промежуточную группу, которая содержит в себе все грани (Рисунок 100).

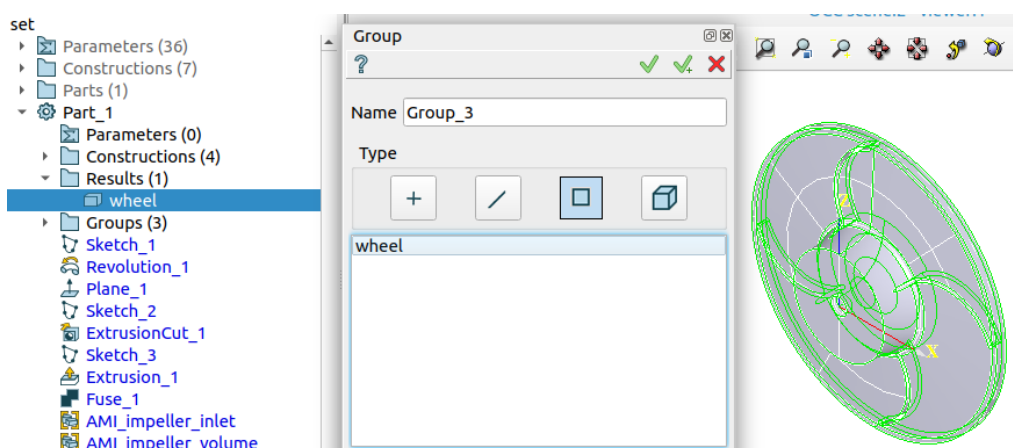


Рисунок 100 – Создание группы 3

Выберите Group Substraction для формирования последней группы. В качестве главной группы выберите Group_3 и удалите из нее грани двух других групп (Рисунок 101).

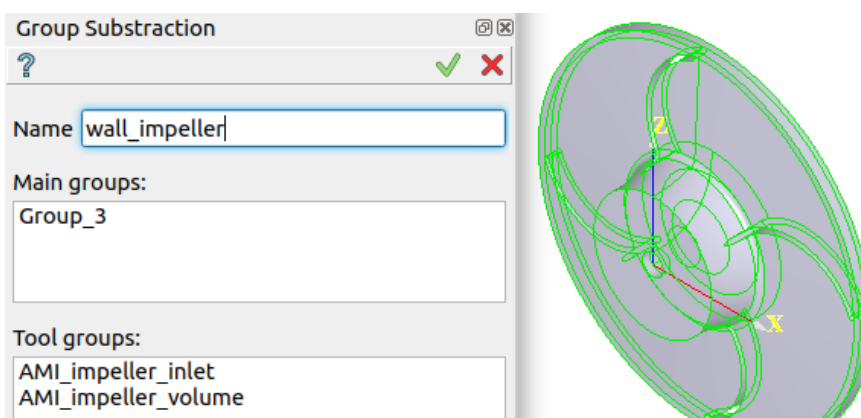


Рисунок 101 – Создание группы wall_impeller

Перейдите в Part2, выберите грань для создания AMI поверхности (Рисунок 102).

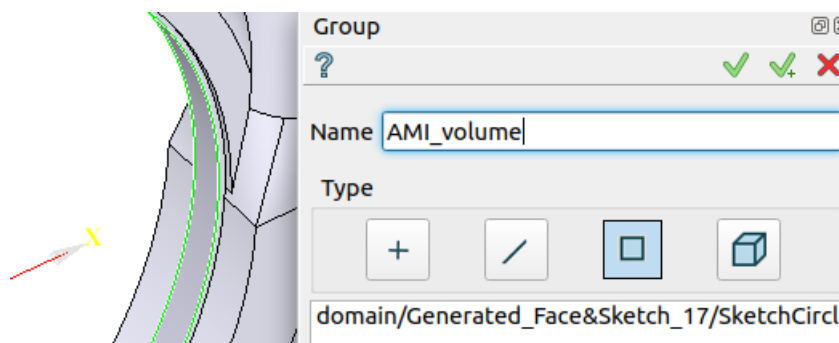


Рисунок 102 – Создание группы AMI_volume

Создайте следующую группу, выбрав грань на выходе (Рисунок 103).

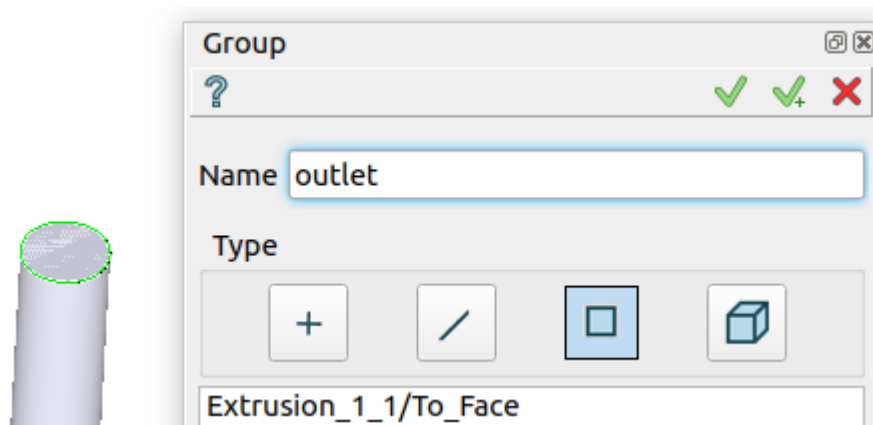


Рисунок 103 – Создание группы outlet

Создайте следующую группу, которая содержит в себе все грани (Рисунок 104).

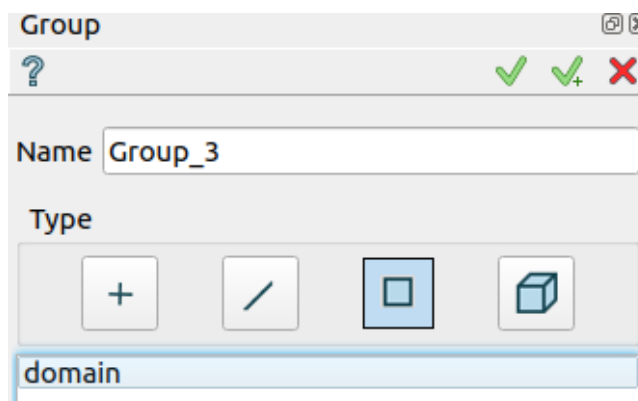


Рисунок 104 – Создание группы 3

Для создания последней группы используйте GroupSubstraction (Рисунок 105).

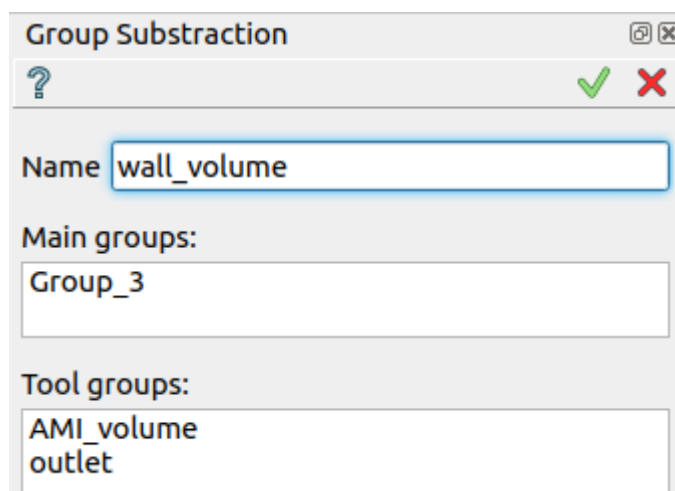


Рисунок 105 – Создание группы wall_volume

Выполните Dump Study, перенесите код создания групп в основной файл.

Листинг 53 – Создание групп рабочего колеса

```
786 ### Create Group
787 Group_1 = model.addGroup(Part_1_doc, "Faces",
    [model.selection("FACE", "Revolution_1_1/Generated_Face&Sketch_1/-
    SketchLine_5")])
788 Group_1.setName("AMI_impeller_inlet")
789 Group_1.result().setName("AMI_impeller_inlet")
790
791 ### Create Group
792 Group_2 = model.addGroup(Part_1_doc, "Faces",
    [model.selection("FACE", "Extrusion_1_1/Generated_Face&Sketch_3/-
    SketchCircle_6_2")])
793 Group_2.setName("AMI_impeller_volume")
794 Group_2.result().setName("AMI_impeller_volume")
795
796 ### Create Group
797 Group_3 = model.addGroup(Part_1_doc, "Faces",
    [model.selection("SOLID", "wheel")])
798
799 ### Create GroupSubstraction
800 GroupSubstraction_1 = model.addGroupSubstraction(Part_1_doc,
    [model.selection("COMPOUND", "Group_3")],
    [model.selection("COMPOUND", "AMI_impeller_inlet"),
    model.selection("COMPOUND", "AMI_impeller_volume")])
801 GroupSubstraction_1.setName("wall_impeller")
802 GroupSubstraction_1.result().setName("wall_impeller")
```

Листинг 54 – Создание групп спирального отвода

```
2187 ### Create Group
2188 Group_4 = model.addGroup(Part_2_doc, "Faces",
    [model.selection("FACE", "domain/Generated_Face&Sketch_17/-
    SketchCircle_29_2")])
2189 Group_4.setName("AMI_volume")
2190 Group_4.result().setName("AMI_volume")
2191
2192 ### Create Group
2193 Group_5 = model.addGroup(Part_2_doc, "Faces",
    [model.selection("FACE", "Extrusion_1_1/To_Face")])
2194 Group_5.setName("outlet")
2195 Group_5.result().setName("outlet")
2196
2197 ### Create Group
2198 Group_6 = model.addGroup(Part_2_doc, "Faces",
    [model.selection("SOLID", "domain")])
2199
2200 ### Create GroupSubstraction
2201 GroupSubstraction_2 = model.addGroupSubstraction(Part_2_doc,
    [model.selection("COMPOUND", "Group_3")],
    [model.selection("COMPOUND", "AMI_volume"),
    model.selection("COMPOUND", "outlet")])
2202 GroupSubstraction_2.setName("wall_volume")
2203 GroupSubstraction_2.result().setName("wall_volume")
```

Листинг 55 – Создание групп входной части

```
2239 ### Create Group
2240 Group_7 = model.addGroup(Part_3_doc, "Faces",
    [model.selection("FACE", "Extrusion_1_1/To_Face")])
2241 Group_7.setName("inlet")
2242 Group_7.result().setName("inlet")
2243
2244 ### Create Group
2245 Group_8 = model.addGroup(Part_3_doc, "Faces",
    [model.selection("FACE", "Extrusion_1_1/From_Face")])
2246 Group_8.setName("AMI_inlet")
2247 Group_8.result().setName("AMI_inlet")
2248
2249 ### Create Group
2250 Group_9 = model.addGroup(Part_3_doc, "Faces",
    [model.selection("FACE", "Extrusion_1_1/Generated_Face&Sketch_1/-
    SketchCircle_2_2"), model.selection("FACE", "Extrusion_1_1/-
    Generated_Face&Sketch_1/SketchCircle_1_2")])
2251 Group_9.setName("wall_inlet")
2252 Group_9.result().setName("wall_inlet")
```

12 ПОСТРОЕНИЕ СЕТКИ

В меню Features выберите Export to GEOM. Перейдите в модуль Mesh. Выберите Translation_1 из раздела Geometry для создания первой сетки рабочего колеса (Рисунок 106).

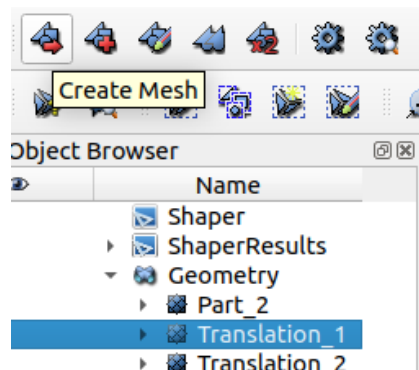


Рисунок 106 – Создание сетки для рабочего колеса

Для 3D алгоритма установите NETGEN 3D, в качестве гипотезы используйте Max. Element Volume (Рисунок 107).

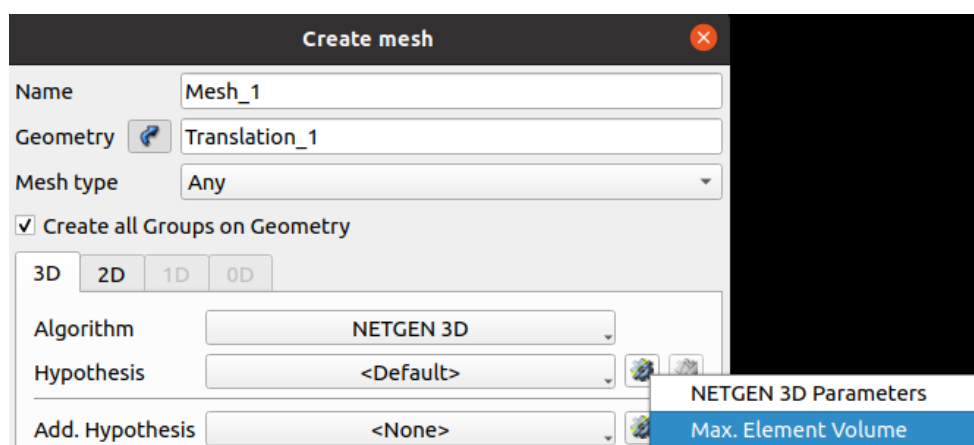


Рисунок 107 – Алгоритм 3D для Mesh 1

Установите параметру Max. Volume значение 17 (Рисунок 108).

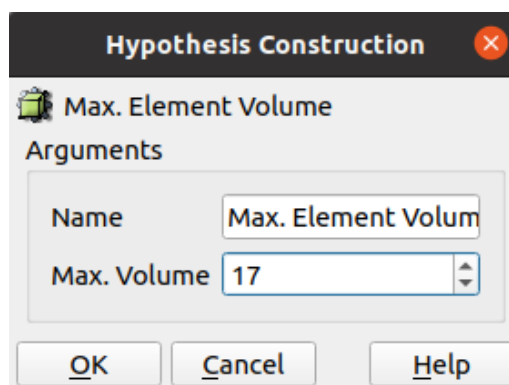


Рисунок 108 – Изменение Max. Volume

Для 2D используйте алгоритм Triangle: Mefisto, для гипотезы выберите Max. Element Area, равное 9 (Рисунок 109).

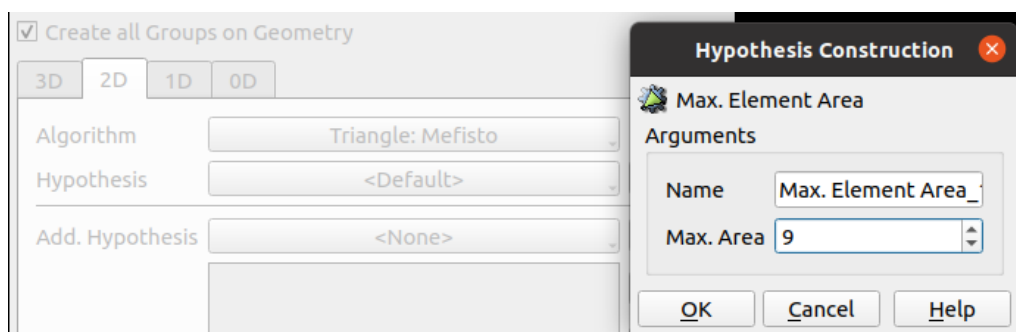


Рисунок 109 – Выбор алгоритма и гипотезы для 2D

Для 1D установите Wire Discretization и Local Length, равное 6, алгоритму и гипотезе соответственно (Рисунок 110).

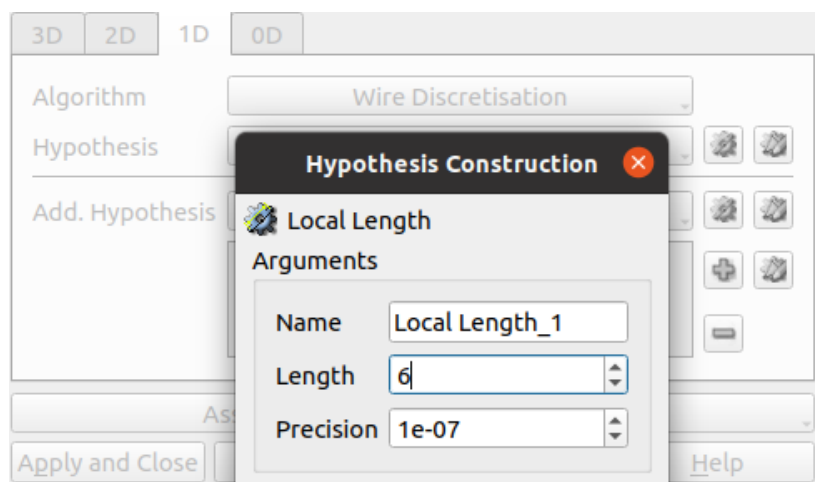


Рисунок 110 – Выбор алгоритма и гипотезы для 1D

Для 0D в качестве алгоритма используйте Segment around Vertex, Length оставьте по умолчанию 1 (Рисунок 111).

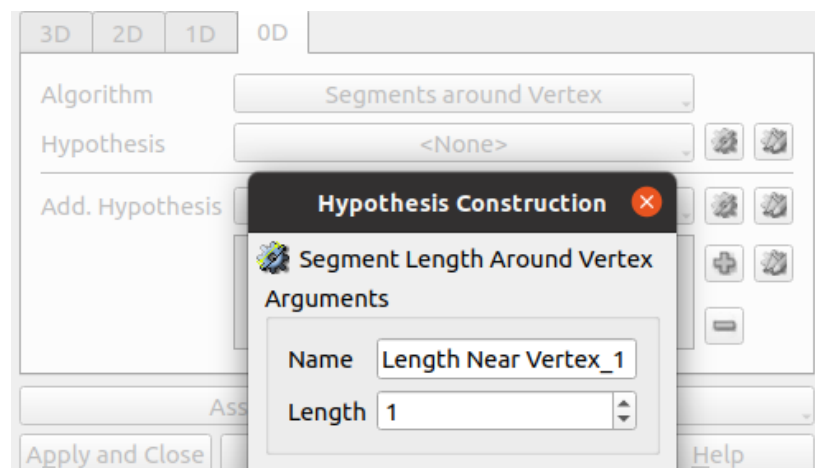


Рисунок 111 – Выбор алгоритма и гипотезы для 0D

Выполните расчет сетки, в контекстном меню выберите Compute. Сетка должна построиться без ошибок (Рисунок 112).

Mesh computation succeed				
Compute mesh				
Name				
Mesh_1				
Mesh Infos				
	Total	Linear	Quadratic	Bi-Quadratic
Nodes :	17536			
0D Elements :	0			
Balls :	0			
Edges :	3043	3043	0	
Faces :	23236	23236	0	0
Triangles :	23236	23236	0	0
Quadrangles :	0	0	0	0
Polygons :	0	0	0	
Volumes :	66675	66675	0	0
Tetrahedrons :	66675	66675	0	
Hexahedrons :	0	0	0	0
Pyramids :	0	0	0	
Prisms :	0	0	0	0
Hexagonal prisms :	0			
Polyhedrons :	0			

Рисунок 112 – Результат построения сетки

Сетка должна получиться такой, как на рисунке 113.

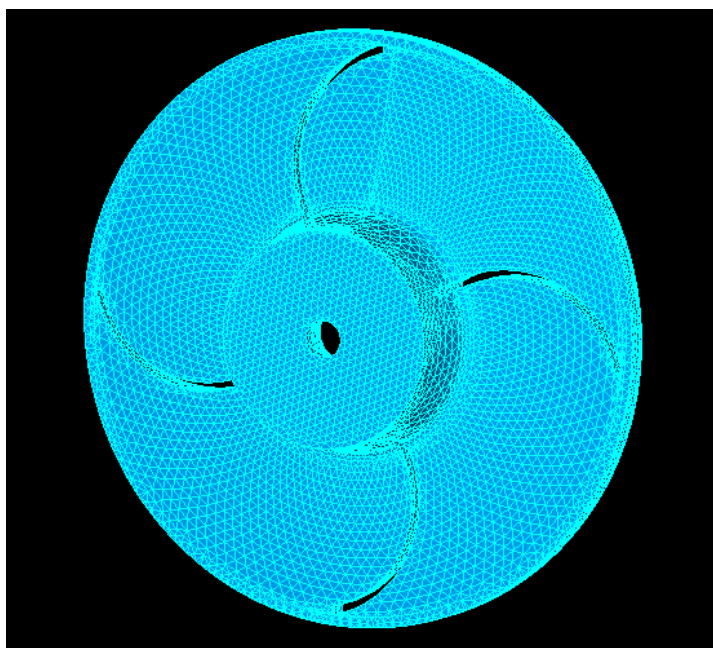


Рисунок 113 – Сетка рабочего колеса

Выполните построение сетки для входной части. Установите алгоритм NETGEN 1D-2D-3D, в качестве гипотезы выберите NETGEN 3D Parameters (Рисунок 114).

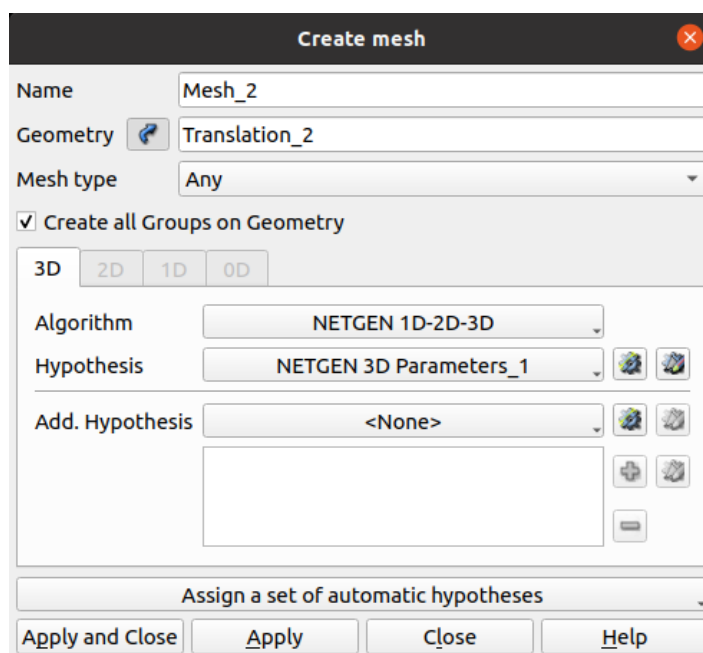


Рисунок 114 – Установка алгоритма

Установите максимальный и минимальный размеры (Рисунок 115).

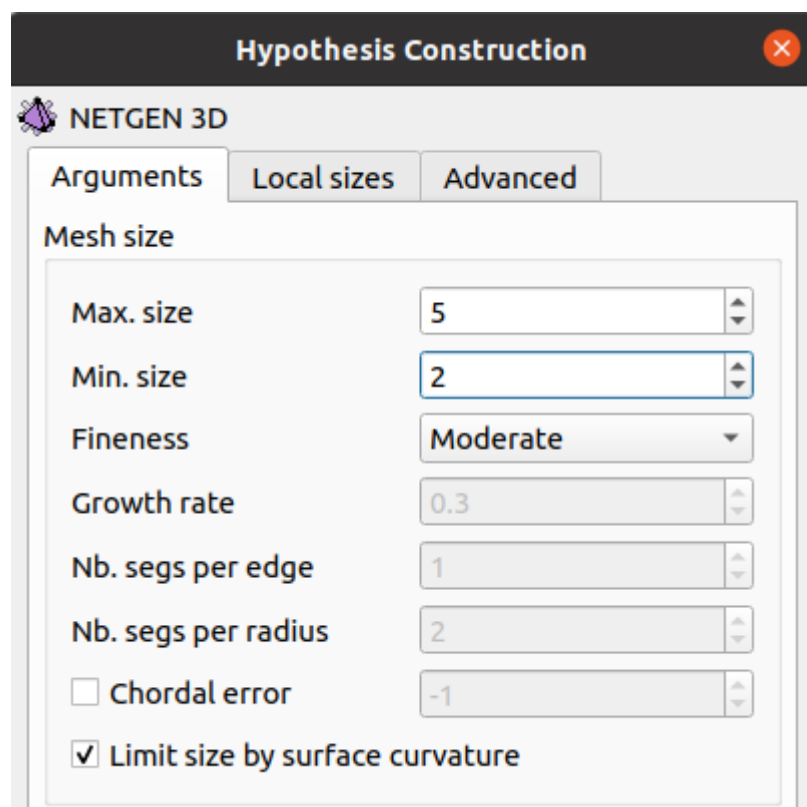


Рисунок 115 – Изменение максимального и минимального размеров

Перейдите на вкладку Local sizes, в дереве объектов выберите группу wall_inlet, нажмите на кнопку On face, установите значение 5 (Рисунок 116).

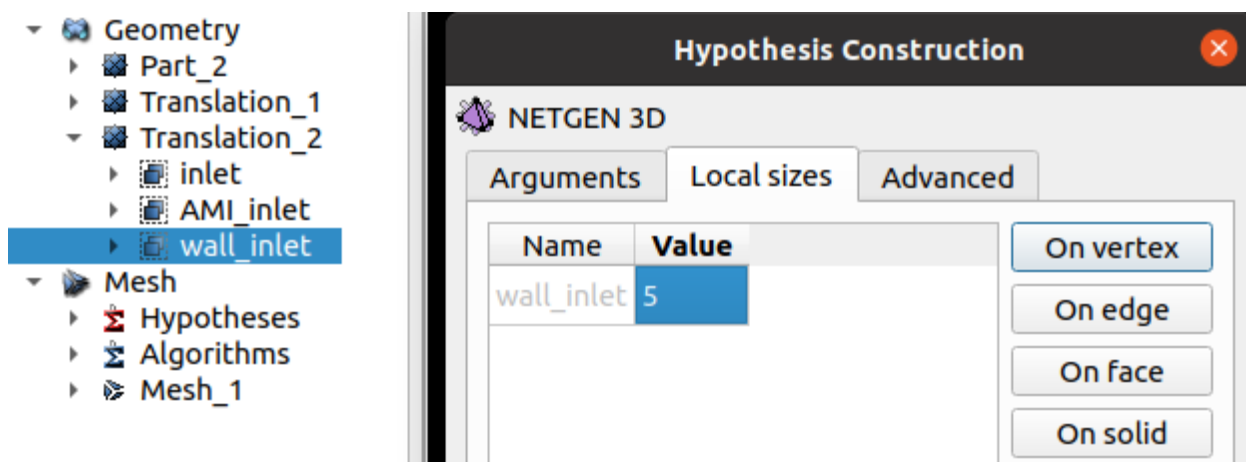


Рисунок 116 – Установка разрешения сетки на грани

В контекстном меню выберите Compute, расчет должен пройти без ошибок (Рисунок 117).

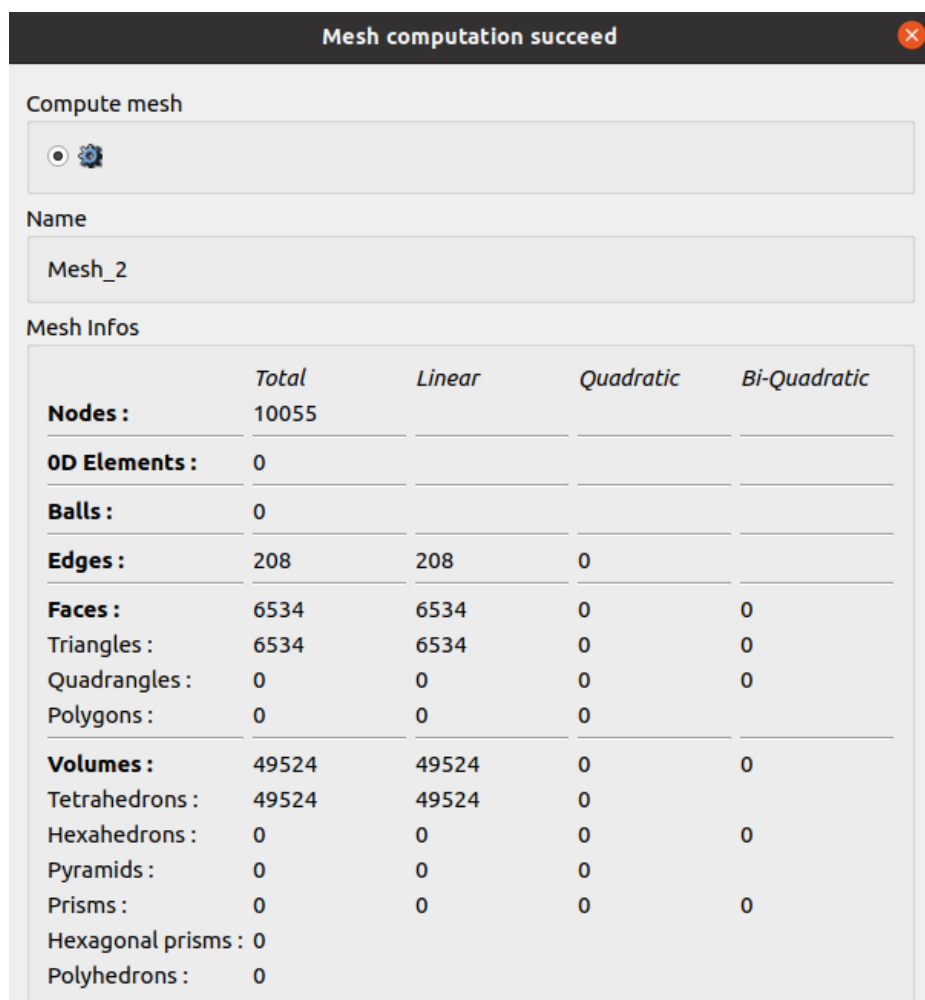


Рисунок 117 – Результат построения сетки

Результат построения показан на рисунке 118.

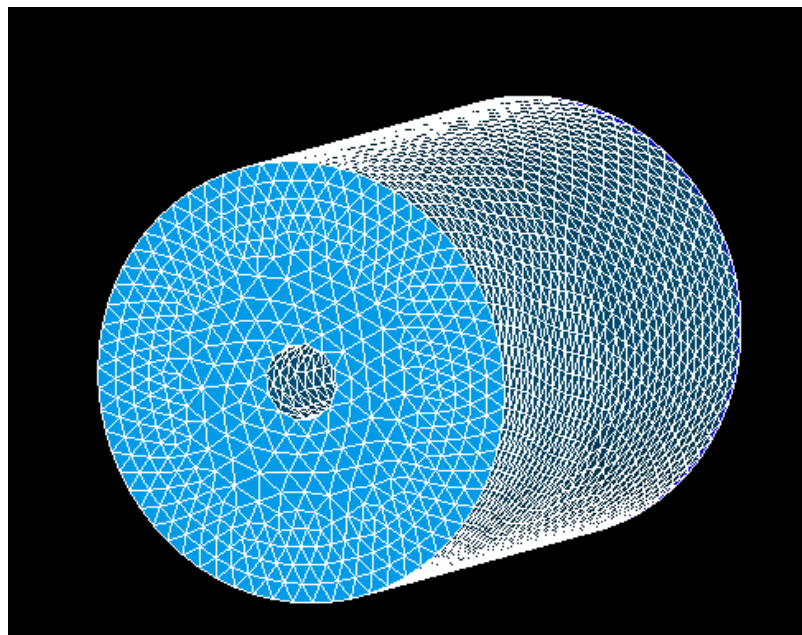
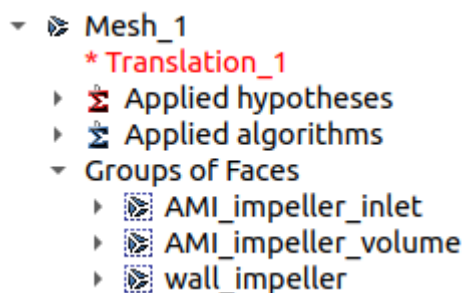


Рисунок 118 – Сетка входной части

Сетка для спирального отвода будет построена не в Salome, а в OpenFOAM. Рассмотрим для этого еще одну утилиту snappyHexMesh. Создайте в рабочей директории OpenFOAM каталог rimp. Все сетки строятся отдельно, затем объединяются. В качестве решателя будет использоваться pimpleFoam, поэтому переходите в раздел tutorials и копируйте в директорию rimp учебный пример impeller, расположенный по пути /opt/openfoam9/tutorials/incompressible/pimpleFoam/RAS. Сделайте еще две копии. Каталоги назовите: inlet, impeller, volume. В первые два экспортируйте созданные сетки в формате UNV, название не столь важно, можете использовать Mesh_1. Предварительно удалите лишнюю группу рабочего колеса.

Листинг 56 – Группы по поверхности рабочего колеса



Перейдите в каталог volume/constant. Переименуйте директорию geometry в triSurface. Удалите содержимое директории. В нее необходимо экспортировать группы спирального отвода из Salome в формате STL. Перейдите в модуль Geometry, в меню File выберите Export и формат STL (Рисунок 119). Выполните для трех групп, Group_3 не нужна.

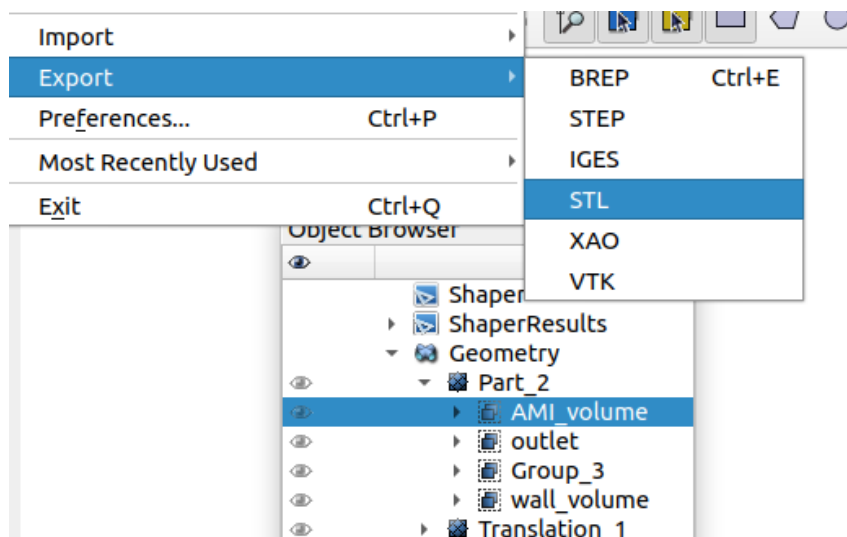


Рисунок 119 – Экспорт групп

Чтобы использовать утилиту snappyHexMesh, нужно изначально создать базовую сетку, определяющую расчетную область. Сделать ее можно с помощью утилиты blockMesh. Перейдите в каталог system, откройте файл blockMeshDict. Так как построение спирального отвода выполнено в мм, значение convertToMeters установите 1000. Базовая область должна превышать размеры геометрии, используйте 0.6 м в каждом направлении.

Листинг 57 – Файл blockMeshDict начало

```

17 convertToMeters 1000;
18
19
20 vertices
21 (
22     ( -0.6 -0.6 -0.6)
23     (  0.6 -0.6 -0.6)
24     (  0.6  0.6 -0.6)
25     ( -0.6  0.6 -0.6)
26     ( -0.6 -0.6  0.6)
27     (  0.6 -0.6  0.6)
28     (  0.6  0.6  0.6)
29     ( -0.6  0.6  0.6)
30 );

```

В списке blocks стандартный порядок блоков, базовая сетка состоит из 90 ячеек в каждом направлении. Список boundary содержит построенный куб, тип по умолчанию patch.

Листинг 58 – Файл blockMeshDict продолжение

```
32 blocks
33 (
34     hex (0 1 2 3 4 5 6 7) (90 90 90) simpleGrading (1 1 1)
35 );
36
37 edges
38 (
39 );
40
41 boundary
42 (
43     allBoundary
44     {
45         type patch;
46         faces
47         (
48             (3 7 6 2)
49             (0 4 7 3)
50             (2 6 5 1)
51             (1 5 4 0)
52             (0 3 2 1)
53             (4 5 6 7)
54         );
55     }
56 );
```

Сохраните изменения. Перейдите снова в раздел учебных примеров и выполните поиск файла snappyHexMesh. Скопируйте любой из них, далее внесем в него изменения. Подробное описание каждого параметра можете найти в приложенном к данной лабораторной работе файле, здесь будет дано лишь краткое описание. В начале файла включите построение «зубчатой» сетки (castellatedMesh) и привязку к поверхностям (snap). Дополнительные слои на поверхностях строить не будем (addLayers).

Листинг 59 – Файл snappyHexMeshDict начало

```
8 FoamFile
9 {
10     format      ascii;
11     class        dictionary;
12     object       snappyHexMeshDict;
13 }
14
15
16 castellatedMesh true;
17 snap           true;
18 addLayers      false;
```

В разделе `geometry` укажите всю геометрию, которая будет использоваться при построении сетки. Используйте файлы, расположенные в директории `triSurface`.

Листинг 60 – Объявление геометрии

```
21 geometry
22 {
23     AMI_volume
24     {
25         type triSurfaceMesh;
26         file "AMI_volume.stl";
27     }
28     outlet
29     {
30         type triSurfaceMesh;
31         file "outlet.stl";
32     }
33     wall_volume
34     {
35         type triSurfaceMesh;
36         file "wall_volume.stl";
37     }
38 };
```

В разделе `castellatedMeshControls` укажите максимальное число элементов для одного процессора `maxLocalCells` (данный параметр используется при параллельном построении), максимальное количество элементов `maxGlobalCells` (параметр является примерным), число улучшаемых ячеек `minRefinementCells`, количество слоев между разными уровнями `nCellsBetweenLevels`. Список опций `features` оставьте пустым.

Листинг 61 – Раздел `castellatedMeshControls` начало

```
41 castellatedMeshControls
42 {
43     maxLocalCells 1000000;
44
45     maxGlobalCells 20000000;
46
47     minRefinementCells 100;
48
49     nCellsBetweenLevels 1;
50
51     features
52     (
53     );
```

В подразделе refinementSurface определяются параметры разбиения поверхностей из раздела geometry. Для поверхностей указываются минимальный и максимальный уровни улучшения. Максимальный применяется тогда, когда ячейка образует угол, который больше, чем resolveFeatureAngle.

Листинг 62 – Подраздел refinementSurface

```
55     refinementSurfaces
56     {
57         outlet
58         {
59             mode inside;
60             level (2 4);
61             patchInfo { type patch; }
62         }
63         AMI_volume
64         {
65             mode inside;
66             level (2 4);
67             patchInfo { type patch; }
68         }
69         wall_volume
70         {
71             mode inside;
72             level (2 4);
73             patchInfo { type wall; }
74         }
75     }
```

В подразделе refinementRegions могут дополнительно быть заданы области для улучшения. Параметр locationInMesh отвечает за создание сетки внутри ограничивающих поверхностей, либо снаружи. Точка должна лежать внутри спирального отвода.

Листинг 63 – Раздел castelatedMeshControls окончание

```
77     refinementRegions
78     {
79     }
80
81     resolveFeatureAngle 60;
82
83     locationInMesh (150 0 0);
84
85     allowFreeStandingZoneFaces true;
86 }
```

Для создания гладкой поверхности используют раздел snapControls. Установите значение следующим параметрам: количеству итераций сглаживания nSmoothPatch, отношению расстояний для точек tolerance,

количеству итераций релаксации перемещений сетки nSolveIter, максимальному количеству итераций релаксации привязки nRelaxIter.

Листинг 64 – Раздел snapControls

```
89 snapControls
90 {
91     nSmoothPatch 3;
92     tolerance 2.0;
93     nSolveIter 30;
94     nRelaxIter 5;
95 }
```

Раздел addLayersControl не будет принимать участие в построении сетки, однако он должен присутствовать в файле. С построением сетки можете поэкспериментировать.

Листинг 65 – Раздел addLayersControl

```
101 addLayersControls
102 {
103     relativeSizes true;
104     layers
105     {
106     }
107     expansionRatio 1.0;
108     finalLayerThickness 0.3;
109     minThickness 0.05;
110     nGrow 0;
111     featureAngle 60;
112     nRelaxIter 3;
113     nSmoothSurfaceNormals 1;
114     nSmoothNormals 3;
115     nSmoothThickness 10;
116     maxFaceThicknessRatio 0.5;
117     maxThicknessToMedialRatio 0.3;
118     minMedianAxisAngle 0;
119     nBufferCellsNoExtrude 0;
120     nLayerIter 50;
121     nRelaxedIter 20;
122 }
```

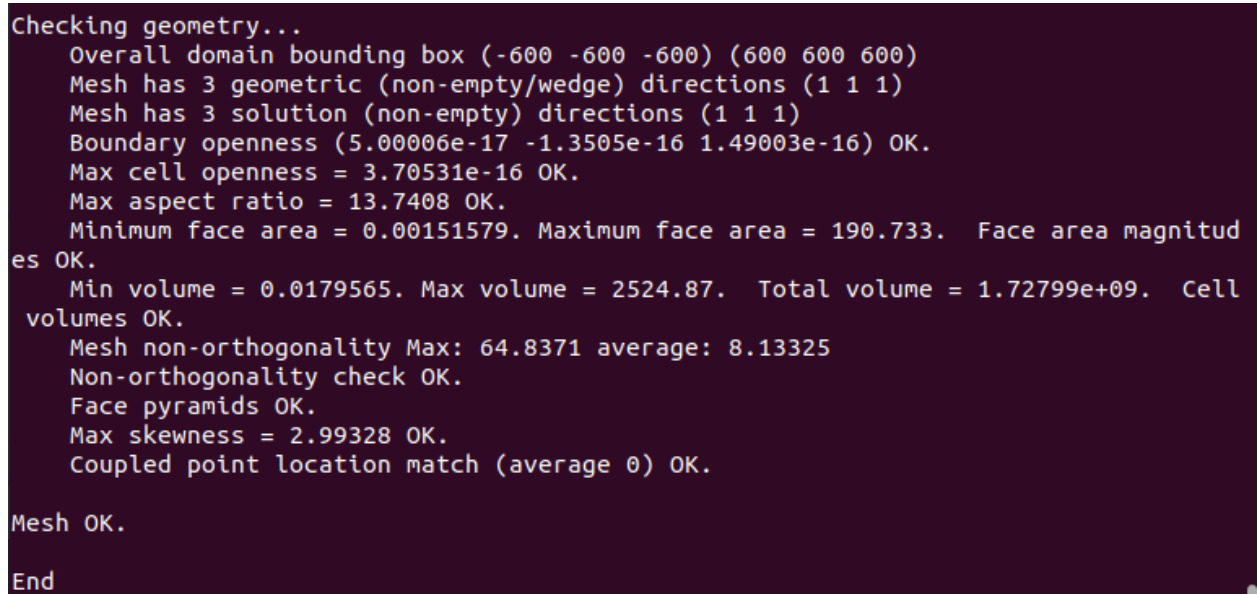
Раздел MeshQualityControls предназначен для контроля качества сетки. Подключите использование другого файла. Также можно и самостоятельно задавать все параметры.

Листинг 66 – Файл snappyHexMeshDict окончание

```
141 meshQualityControls
142 {
143     #includeEtc "caseDicts/mesh/generation/meshQualityDict"
144 }
145
146 mergeTolerance 1e-6;
```

Сохраните изменения. Откройте из каталога volume терминал, выполните последовательно команды blockMesh и snappyHexMesh -overwrite. Флаг overwrite нужен для перезаписи созданной сетки в основной каталог constant, иначе сетка будет создаваться во временных файлах.

Когда сетка будет построена, вызовите утилиту checkMesh для проверки. Убедитесь, что результат выполнения – Mesh OK (Рисунок 120).



```
Checking geometry...
Overall domain bounding box (-600 -600 -600) (600 600 600)
Mesh has 3 geometric (non-empty/wedge) directions (1 1 1)
Mesh has 3 solution (non-empty) directions (1 1 1)
Boundary openness (5.00006e-17 -1.3505e-16 1.49003e-16) OK.
Max cell openness = 3.70531e-16 OK.
Max aspect ratio = 13.7408 OK.
Minimum face area = 0.00151579. Maximum face area = 190.733. Face area magnitud
es OK.
Min volume = 0.0179565. Max volume = 2524.87. Total volume = 1.72799e+09. Cell
volumes OK.
Mesh non-orthogonality Max: 64.8371 average: 8.13325
Non-orthogonality check OK.
Face pyramids OK.
Max skewness = 2.99328 OK.
Coupled point location match (average 0) OK.

Mesh OK.

End
```

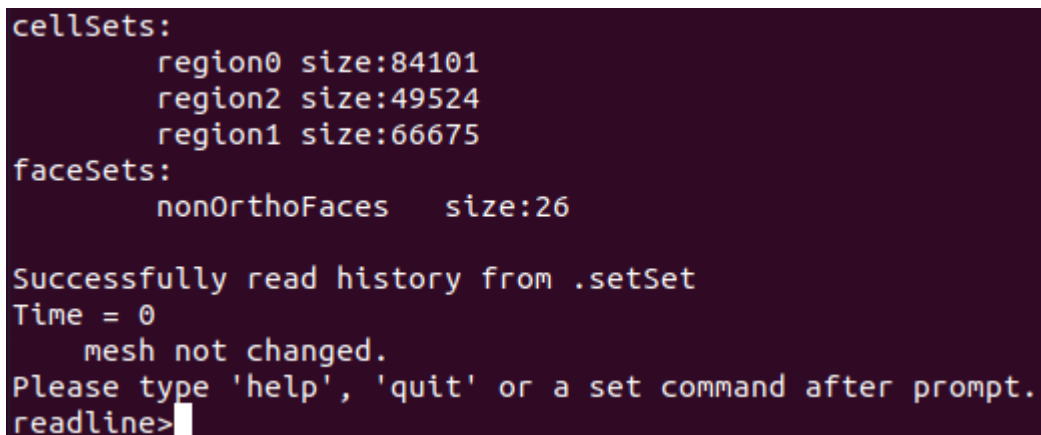
Рисунок 120 – Результат проверки сетки

Обратите внимание, что можно запускать и более глубокую проверку: checkMesh -allTopology -allGeometry. По результату выполнения не будет пройден один тест, но он не так критичен. Над сеткой можно поработать и создать такую, которая пройдет все глубокие тесты. Для данной работы достаточно, чтобы сетки проходили базовую проверку.

Перейдите в терминале в каталог inlet: `cd ../inlet`. Выполните построение сетки: `ideasUnvToFoam Mesh_1.unv`. Запустите проверку сетки: `checkMesh`. Повторите для каталога impeller.

Перейдите в корневой каталог pump. Соедините две сетки inlet и impeller: `mergeMeshes impeller/ inlet/ -overwrite`. Объединенная сетка будет находиться в каталоге impeller. Перейдите в него и снова выполните проверку сетки. Перейдите в каталог pump. Объедините сетки volume и impeller: `mergeMeshes volume/ impeller/ -overwrite`. Перейдите в каталог volume, выполните проверку сетки. Переведите единицы измерения в метры: `transformPoints "scale = (0.001 0.001 0.001)"`.

Далее укажем ячейки, которые образуют рабочее колесо, для использования в OpenFOAM при вращении. Введите в терминале `setSet`. Всего три региона: вход, рабочее колесо и спиральный отвод (Рисунок 121). Рабочее колесо соответствует region1. Введите `help`, чтобы получить команду.

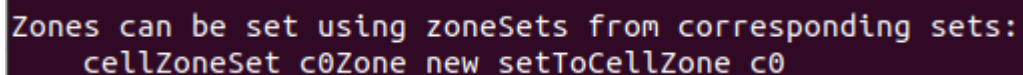


```
cellSets:
    region0 size:84101
    region2 size:49524
    region1 size:66675
faceSets:
    nonOrthoFaces size:26

Successfully read history from .setSet
Time = 0
    mesh not changed.
Please type 'help', 'quit' or a set command after prompt.
readline>
```

Рисунок 121 – Установка региона

Вывод должен содержать команду для установки области ротора (Рисунок 122).



```
Zones can be set using zoneSets from corresponding sets:
cellZoneSet c0Zone new setToCellZone c0
```

Рисунок 122 – Команда для установки зоны ротора

Введите в терминале команду. В каталоге pump/VTK/rotor должен появиться файл rotor_0.vtk (Рисунок 123).

```
readline>cellZoneSet rotor new setToCellZone region1
Set:rotor Size:0 Action:new
Adding all cells from cellSet region1 ...
Writing rotor (size 66534) to "constant/polyMesh/sets/rotor" and to vtk
file "VTK/rotor/rotor_0.vtk"
```

Рисунок 123 – Установка зоны ротора

Введите команду `quit` для выхода. Затем введите `touch foam.foam` для создания файла для ParaView. Следующей командой откройте ParaView: `paraview`. Проверьте сетку (Рисунок 124) и корректность установки ротора.

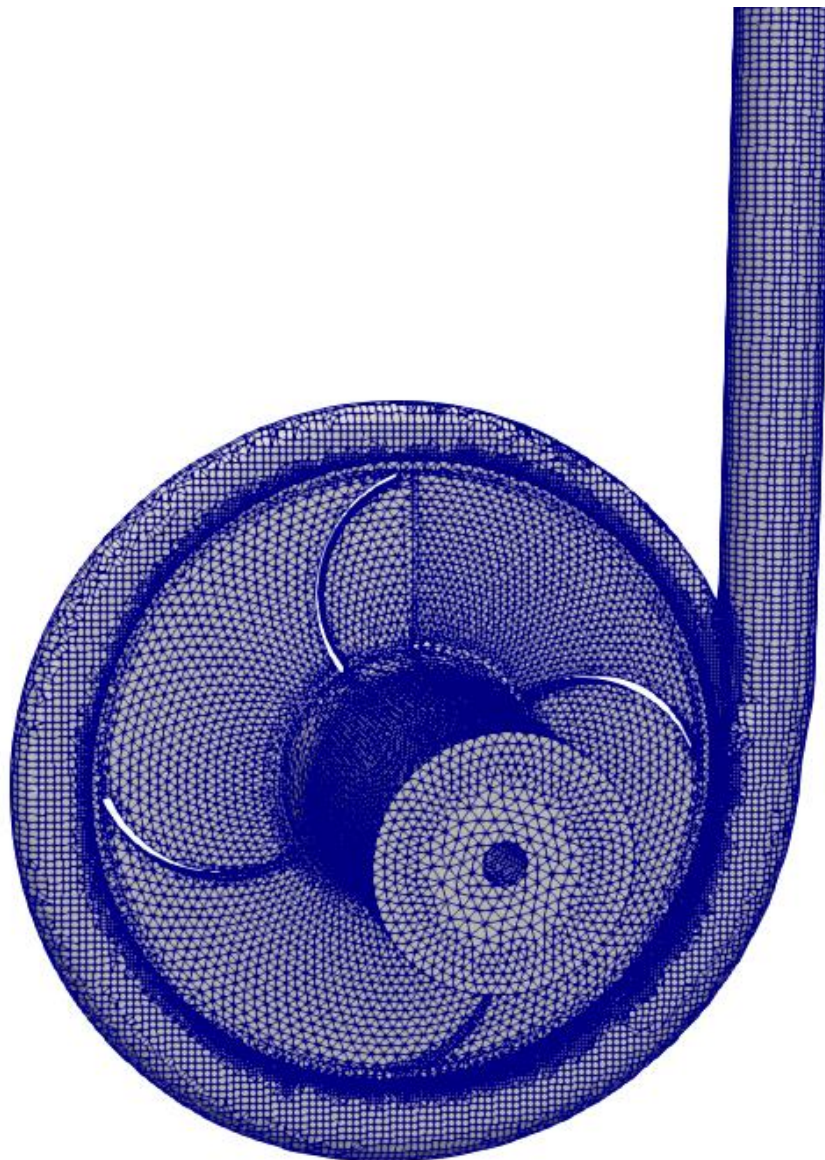


Рисунок 124 – Построенная сетка

13 НАСТРОЙКА OPENFOAM

Откройте каталог 0 для настройки начальных и граничных условий. Начнем со скорости. Для входного сечения установите тип `flowRateInletVelocity`. Для данного типа скорость указывается в м³/с. Используем данные согласно условию: $Q = 0.008$ м³/с. Объемный расход представлен в виде таблицы для начального момента времени, затем промежуточного и конечного. Первому установите минимальную скорость, остальным значение Q . Время окончания расчета будет установлено равным 5 с. Для выходного сечения используйте смешанный тип `inletOutlet`, который ведет себя как `zeroGradient`, но переключается на `fixedValue`, если вектор скорости вблизи границы направлен внутрь области (обратное течение).

Листинг 67 – Файл U

```
17 dimensions      [0 1 -1 0 0 0 0];
18
19 internalField    uniform (0 0 0);
20
21 boundaryField
22 {
23     inlet
24     {
25
26         type       flowRateInletVelocity;
27         volumetricFlowRate table (
28                                     (0.0 1e-6)
29                                     (0.01 0.008)
30                                     (5.0 0.008)
31                                 );
32
33         value       uniform (0 0 0);
34         extrapolateProfile no;
35     }
36     outlet
37     {
38         type       inletOutlet;
39         value       $internalField;
40         inletValue  uniform (0 0 0);
41     }
42 }
```

Стенки будут адиабатическими, для `wall_impeller` установите тип `movingWallVelocity`, начальная скорость равна 0. Для всех поверхностей AMI используйте тип `cyclicAMI` для всех файлов.

Листинг 68 – Файл U продолжение

```
42     "(wall_inlet|wall_volume)"
43     {
44         type          noSlip;
45     }
46
47     wall_impeller
48     {
49         type          movingWallVelocity;
50         value         uniform (0 0 0);
51     }
52
53     "(AMI_volume|AMI_impeller_volume|AMI_inlet|AMI_impeller_inlet)"
54     {
55         type          cyclicAMI;
56     }
57 }
```

Сохраните изменения.

Откройте файл давления. Начальное значение поля давления установите равным 1 атм. Для размерности $\text{м}^2/\text{с}^2$ это 101.629889. На входе и стенках нулевой градиент, на выходе 1 атм.

Листинг 69 – Файл p

```
17 dimensions      [0 2 -2 0 0 0 0];
18
19 internalField    uniform 101.629889;
20
21 boundaryField
22 {
23
24     inlet
25     {
26         type          zeroGradient;
27     }
28
29
30     outlet
31     {
32         type          fixedValue;
33         value         $internalField;
34     }
35
36     "(wall_inlet|wall_volume|wall_impeller)"
37     {
38         type          zeroGradient;
39     }
40
41     "(AMI_volume|AMI_impeller_volume|AMI_inlet|AMI_impeller_inlet)"
42     {
43         type          cyclicAMI;
44     }
45 }
```

Следующие файлы отвечают за турбулентность. Будем использовать модель kOmegaSST. Данная модель объединяет в себе модель kEpsilon и kOmega. Первая модель хорошо рассчитывает внутренние течения, но не справляется при расчете потоков с высокими градиентами давлений и отрывом потока. Вторая предназначена для расчетов отрывных течений вблизи стенки. Так как модель kOmegaSST сочетает в себе лучшие стороны двух других, ее использование дает хорошие результаты, также она является не требовательной к вычислительной мощности.

Откройте файл nut и установите следующие значения.

Листинг 70 – Файл nut

```

21 boundaryField
22 {
23     inlet
24     {
25         type          calculated;
26         value          uniform 0;
27     }
28
29     outlet
30     {
31         type          calculated;
32         value          uniform 0;
33     }
34
35     "(wall_inlet|wall_volume|wall_impeller)"
36     {
37         type          nutkRoughWallFunction;
38         Ks             uniform 0.000016;
39         Cs             uniform 0.6;
40         value uniform 0;
41     }
42
43     "(AMI_volume|AMI_impeller_volume|AMI_inlet|AMI_impeller_inlet)"
44     {
45         type          cyclicAMI;
46     }
47 }

```

Теперь необходимо выполнить расчет турбулентных характеристик. В интернете можно найти онлайн калькуляторы. Можете использовать сайт <http://www.wolfdynamics.com/>. Перейдите в раздел SOFTWARE & WEB TOOLS, выберите Calculator for the estimation of turbulence properties values (initial and boundary conditions). Отметьте вычисление параметров турбулентности на основании интенсивности турбулентности, коэффициента

вязкости, скорости свободного потока и кинематической вязкости (Рисунок 125).

Choose turbulence variables to compute

- ☐ Compute ω from κ and ϵ ($\beta^* = C_\mu = 0.09$)
- ☐ Compute ϵ from κ and ω ($\beta = C_\mu = 0.09$)
- ☐ Compute turbulence variables (κ, ϵ, ω) from Tu, Tu_L and U_∞
- ☒ Compute turbulence variables (κ, ϵ, ω) from $Tu, \mu_t/\mu, U_\infty$ and kinematic viscosity ν

Рисунок 125 – Вычисление параметров турбулентности

Значение скорости берем из расчета, переменная C0 (Рисунок 126).

```
U1 6
C0 1
C_m1 1
```

Рисунок 126 – Определение скорости C0

Для случаев высокой турбулентности, например, высокоскоростной поток внутри сложных геометрий (теплообменники, вращающиеся машины), интенсивность турбулентности составляет от 5% до 20%. Будем использовать 5%. Коэффициент вязкости установите 10. Кинематическая вязкость для воды составляет 1.0^{-6} м²/с. Нажмите Compute. Значения k, ω, ϵ должны рассчитаться (Рисунок 127).

Properties - Input values

Freestream velocity	U_∞	1 m/s
Turbulence kinetic energy	κ	0.003750000000000000 m ² /s
Turbulence dissipation	ϵ	0.126562500000000005 m ² /s ³
Specific turbulence dissipation	ω	375.00000000000006 1/s
Turbulence intensity	Tu	5 %
Turbulence length scale	Tu_L	m
Kinematic viscosity	ν	1e-06 m ² /s *
Eddy viscosity ratio	μ_t/μ	10 []

Рисунок 127 – Расчет параметров турбулентности

Создайте файл omega. Используйте вычисленное значение.

Листинг 71 – Файл omega

```
8 FoamFile
9 {
10     format      ascii;
11     class        volScalarField;
12     location     "0";
13     object       omega;
14 }
15 // * * * * *
16
17 dimensions      [0 0 -1 0 0 0 0];
18
19 internalField    uniform 375.00000000000006;
20
21 boundaryField
22 {
23     inlet
24     {
25         type      fixedValue;
26         value      uniform 375.00000000000006;
27     }
28     outlet
29     {
30         type      inletOutlet;
31         inletValue uniform 375.00000000000006;
32     }
33     "(wall_inlet|wall_volume|wall_impeller)"
34     {
35         type      omegaWallFunction;
36         value      $internalField;
37     }
38     "(AMI_volume|AMI_impeller_volume|AMI_inlet|AMI_impeller_inlet)"
39     {
40         type      cyclicAMI;
41     }
42 }
```

По аналогии измените файлы k и epsilon. Последний файл не нужен для данного расчета, так как используем другую модель турбулентности, но можно выполнить расчет с kEpsilon моделью и сравнить результаты.

Листинг 72 – Файл k

```
19 internalField    uniform 0.003750000000000007;
20
21 boundaryField
22 {
23
24     inlet
25     {
26         type      fixedValue;
27         value      uniform 0.003750000000000007;
28     }
```

Листинг 73 – Файл k продолжение

```
30     outlet
31     {
32         type            inletOutlet;
33         inletValue      uniform 0.0037500000000000007;
34     }
35
36     "(wall_inlet|wall_volume|wall_impeller)"
37     {
38         type            kqRWallFunction;
39         value           uniform 0;
40     }
41
42     "(AMI_volume|AMI_impeller_volume|AMI_inlet|AMI_impeller_inlet)"
43     {
44         type            cyclicAMI;
45     }
46 }
```

Листинг 74 – Файл epsilon

```
19 internalField    uniform 0.126562500000000005;
20
21 boundaryField
22 {
23
24     inlet
25     {
26         type          fixedValue;
27         value         uniform 0.126562500000000005;
28     }
29
30     outlet
31     {
32         type          inletOutlet;
33         inletValue    uniform 0.126562500000000005;
34     }
35
36     "(wall_inlet|wall_volume|wall_impeller)"
37     {
38         type          epsilonWallFunction;
39         value         $internalField;
40     }
41
42     "(AMI_volume|AMI_impeller_volume|AMI_inlet|AMI_impeller_inlet)"
43     {
44         type          cyclicAMI;
45     }
46 }
```

Перейдите в каталог constant, откройте файл dynamicMeshDict, в нем описываются настройки ротора. Значение cellZone должно равняться rotor.

Ось вращения Y, угловая скорость задается по аналогии с файлом скорости U. В таблице записываются временные промежутки. Значение -104.7 взято из расчета.

Листинг 75 – Файл dynamicMeshDict

```
17 dynamicFvMesh dynamicMotionSolverFvMesh;
18
19 motionSolverLibs ( "libfvMotionSolvers.so" );
20
21 motionSolver solidBody;
22
23 cellZone rotor;
24
25 solidBodyMotionFunction rotatingMotion;
26
27
28 origin      (0 0 0);
29 axis        (0 1 0);
30 omega       table (
31               (0.0 0)
32               (0.01 -104.7)
33               (5.0 -104.7)
34             );
```

В файле transportProperties используйте значение кинематической вязкости для воды.

Листинг 76 – Файл transportProperties

```
17 transportModel Newtonian;
18
19 nu              [0 2 -1 0 0 0 0] 1e-06;
```

Укажите модель турбулентности в файле momentumTransport.

Листинг 77 – Файл momentumTransport

```
17 simulationType RAS;
18
19 RAS
20 {
21     model          kOmegaSST;
22
23     turbulence      on;
24
25     printCoeffs     on;
26 }
```

Осталось настроить файлы каталога system. Начнем с controlDict. Время окончания расчета 5 секунд, deltaT будет автоматически подстраиваться в зависимости от хода решения. Записывать результаты будем каждые 0.001 с. Так как используем алгоритм PIMPLE, можем увеличить число Куранта. Установите значение 50.

Листинг 78 – Файл ControlDict

```
17 application      pimpleFoam;
18
19 startFrom         latestTime;
20
21 startTime         0;
22
23 stopAt            endTime;
24
25 endTime           5.0;
26
27 deltaT            0.0001;
28
29 writeControl       runtime;
30
31 writeInterval      0.001;
32
33 purgeWrite         0;
34
35 writeFormat        ascii;
36
37 writePrecision     6;
38
39 writeCompression  off;
40
41 timeFormat         general;
42
43 timePrecision      6;
44
45 runtimeModifiable yes;
46
47 adjustTimeStep     yes;
48
49 maxCo              50;
```

Настройки файла decomposeParDict можете взять из учебных примеров в зависимости от количества ядер процессора. В данной лабораторной работе используется метод hierarchical.

Листинг 79 – Файл decomposeParDict

```
17 numberOfSubdomains 8;
18 method              hierarchical;
19 simpleCoeffs
20 {
21     n                (8 1 1);
22 }
23 hierarchicalCoeffs
24 {
25     n                (4 2 1);
26     order            xyz;
27 }
28 manualCoeffs
29 {
30     dataFile         "";
31 }
32 distributed         no;
33 roots              ( );
```

Установите следующие схемы для файла fvSchemes.

Листинг 80 – Файл fvSchemes

```
17 ddtSchemes
18 {
19     default          Euler;
20 }
21 gradSchemes
22 {
23     default          Gauss linear;
24 }
25 divSchemes
26 {
27     default          none;
28     div(phi,U)       Gauss linearUpwindV grad(U);
29     div(phi,k)       Gauss upwind;
30     div(phi,omega)   Gauss upwind;
31     div((nuEff*dev2(T(grad(U)))) Gauss linear;
32 }
33 laplacianSchemes
34 {
35     default          Gauss linear limited 0.5;
36 }
37 interpolationSchemes
38 {
39     default          linear;
40 }
41 snGradSchemes
42 {
43     default          limited 0.5;
44 }
45 wallDist
46 {
47     method meshWave;
48 }
```

Настройки файла fvSolution показаны ниже.

Листинг 81 – Файл fvSolution

```
17 solvers
18 {
19     p
20     {
21         solver          GAMG;
22         tolerance       1e-08;
23         relTol          0.01;
24         smoother        GaussSeidel;
25         nCellsInCoarsestLevel 20;
26     }
27     pFinal
28     {
29         solver          GAMG;
30         tolerance       1e-08;
31         relTol          0;
32         smoother        GaussSeidel;
33     }
34 }
```

Листинг 82 – Файл fvSolution продолжение

```

37 "pcorr.*"
38 {
39     solver          GAMG;
40     tolerance       1e-2;
41     relTol          0;
42     smoother        DICGaussSeidel;
43     cacheAgglomeration no;
44     maxIter         50;
45 }
46
47
48 U
49 {
50     solver          smoothSolver;
51     smoother        GaussSeidel;
52     nSweeps         2;
53     tolerance       1e-07;
54     relTol          0.001;
55 }
56
57 k
58 {
59     solver          smoothSolver;
60     smoother        GaussSeidel;
61     nSweeps         2;
62     tolerance       1e-07;
63     relTol          0.001;
64 }
65
66 omega
67 {
68     solver          smoothSolver;
69     smoother        GaussSeidel;
70     nSweeps         2;
71     tolerance       1e-07;
72     relTol          0.001;
73 }
74
75
76 "(U|k|omega)Final"
77 {
78     $U;
79     tolerance       1e-08;
80     relTol          0;
81 }
82 }
83
84 PIMPLE
85 {
86     nNonOrthogonalCorrectors 5;
87     nCorrectors             1;
88     nOuterCorrectors        5;
89     pRefCell                0;
90     pRefValue               101.629889;
91
92     residualControl
93     {
94         U 1e-06;
95         p 1e-05;
96     }
97
98 relaxationFactors
99 {
100     fields
101     {
102         p 0.3;
103         pFinal 1;
104     }
105     equations
106     {
107         "U|k|omega" 0.5;

```

Для автоматического изменения типов граничных условий в файле boundary используют словарь changeDictionaryDict. Создайте данный файл. В подсловаре boundary записываются все грани, указывается тип, на который следует заменить тип patch, который устанавливается по умолчанию. Для поверхностей АМІ следует указывать связанную с ней поверхность АМІ – neighbourPatch. Укажите допуск matchTolerance.

Листинг 83 – Файл changeDictionaryDict

```

1  /*-----*- C++ -
   *-----*\
2  =====
3  \ \      / F i e l d      | OpenFOAM: The Open Source CFD Toolbox
4  \ \    / O peration      | Website: https://openfoam.org
5  \ \  / A nd              | Version: 9
6  \ \ / M anipulation      |
7  \*-----*\
   */
8  FoamFile
9  {
10     format      ascii;
11     class        dictionary;
12     object        changeDictionaryDict;
13 }
14 // *****
   // *****
15
16 boundary
17 {
18     AMI_inlet
19     {
20         type      cyclicAMI;
21         matchTolerance 0.0001;
22         neighbourPatch AMI_impeller_inlet;
23     }
24
25     AMI_impeller_inlet
26     {
27         type      cyclicAMI;
28         matchTolerance 0.0001;
29         neighbourPatch AMI_inlet;
30     }
31
32     AMI_impeller_volume
33     {
34         type      cyclicAMI;
35         matchTolerance 0.0001;
36         neighbourPatch AMI_volume;
37     }
38
39     AMI_volume
40     {
41         type      cyclicAMI;
42         matchTolerance 0.0001;
43         neighbourPatch AMI_impeller_volume;
44     }
45
46     "(wall_inlet|wall_volume|wall_impeller)"
47     {
48         type      wall;
49     }
50
51 }

```

Создайте еще один файл topoSetDict для установки поверхностей AMI. AMI1 для входной части и рабочего колеса, AMI2 для рабочего колеса и спирального отвода.

Листинг 84 – Файл topoSetDict

```

1 /*-----*- C++ -
2  =====|
3  \\      / F ield      | OpenFOAM: The Open Source CFD
Toolbox
4  \\      / O peration  | Website: https://openfoam.org
5  \\      / A nd        | Version: 9
6  \\      / M anipulation|
7 /*-----*-
8 FoamFile
9 {
10     format      ascii;
11     class        dictionary;
12     object        topoSetDict;
13 }
14 // *****
***** //
15
16 actions
17 (
18     {
19         name      AMI1;
20         type      faceSet;
21         action    new;
22         source    patchToFace;
23         sourceInfo
24         {
25             name    AMI_impeller_inlet;
26         }
27     }
28     {
29         name      AMI1;
30         type      faceSet;
31         action    add;
32         source    patchToFace;
33         sourceInfo
34         {
35             name    AMI_inlet;
36         }
37     }
38     {
39         name      AMI2;
40         type      faceSet;
41         action    new;
42         source    patchToFace;
43         sourceInfo
44         {
45             name    AMI_impeller_volume;
46         }
47     }
48     {
49         name      AMI2;
50         type      faceSet;
51         action    add;
52         source    patchToFace;
53         sourceInfo
54         {
55             name    AMI_volume;
56         }
57     }
58 );

```

Из каталога volume в терминале выполните команду для изменения типов граничных условий: changeDictionary. Проверьте файл constant/polyMesh/boundary. Выполните следующую команду: topoSet. Вы должны получить ошибку (Рисунок 128).

```
--> FOAM FATAL ERROR:
The distance between the centre of patch AMI_volume and the transformed centre of patch AMI_impeller_volume is 0.000729789.
This is greater than the match tolerance of 9.79914e-06 for the patch.
Check that the patches are geometrically similar and that any transformations defined between them are correct
It might be possible to fix this problem by increasing the "matchTolerance" setting for this patch in the boundary
```

Рисунок 128 – Ошибка при выполнении команды topoSet

Исправить ее можно либо изменив сетку, либо увеличив значение допуска в файле boundary. Измените сразу для всех AMI поверхностей.

Листинг 85 – Изменение допуска

```
21     AMI_volume
22     {
23         type            cyclicAMI;
24         nFaces           7964;
25         startFace        463856;
26         matchTolerance   0.009;
27         neighbourPatch   AMI_impeller_volume;
28     }
54     AMI_impeller_volume
55     {
56         type            cyclicAMI;
57         nFaces           2630;
58         startFace        509248;
59         matchTolerance   0.009;
60         neighbourPatch   AMI_volume;
61     }
77     AMI_inlet
78     {
79         type            cyclicAMI;
80         nFaces           661;
81         startFace        531824;
82         matchTolerance   0.0005;
83         neighbourPatch   AMI_impeller_inlet;
84     }
45     AMI_impeller_inlet
46     {
47         type            cyclicAMI;
48         nFaces           1493;
49         startFace        507755;
50         matchTolerance   0.0005;
51         neighbourPatch   AMI_inlet;
52     }
```

Снова попробуйте выполнить команду `topoSet`. В каталоге `constant/polyMesh/sets` должны появиться `AMI1` и `AMI2`.

Далее выполняйте `decomposePar` и запускайте расчет: `mpirun -np 8 pimpleFoam -parallel`.

Вы должны получить результаты, как на рисунке 129.

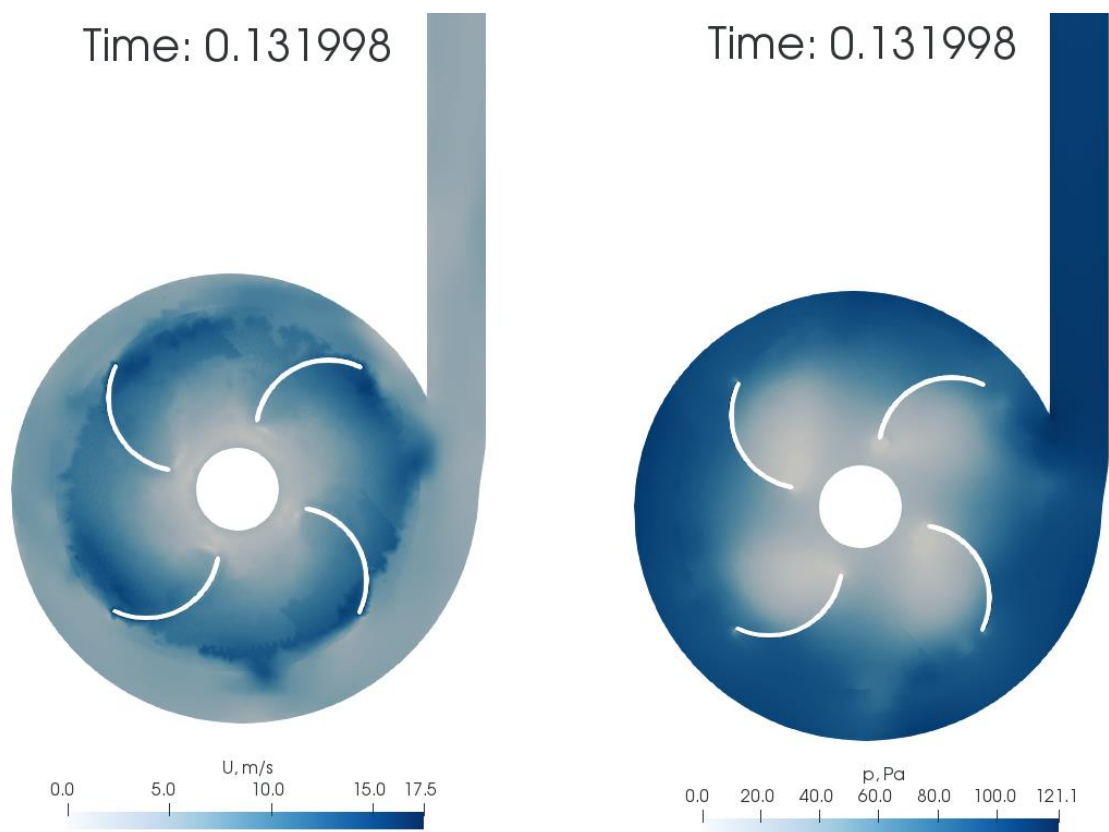


Рисунок 129 – Результаты расчетов

ЗАКЛЮЧЕНИЕ

В ходе данной работы был выполнен расчет центробежного насоса, рассмотрена параметризация построения геометрии в Salome. Сетка была построена как в Salome, так и с использованием утилиты snappyHexMesh. Был рассмотрен новый тип задач в OpenFOAM, новые граничные условия и файлы.

Вопросы для самоконтроля:

1. Опишите принцип работы центробежного насоса?
2. С помощью какого файла OpenFOAM автоматически изменяются типы граничных условий в constant/polyMesh/boundary? Опишите настройки файла.
3. Перечислите основные модели турбулентности. В каких случаях применяют каждую из них?
4. Опишите последовательность действий по настройке вращения ротора?
5. Из каких разделов состоит файл snappyHexMesh? Кратко опишите настройки.